Sapienza
Università di Roma

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

Architettura degli Elaboratori

Appunti a cura di
Lorenzo Ferraro

14 novembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Architettura degli Elaboratori*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1 Sistemi numerici **1**
1.1 Introduzione ai sistemi numerici . . . 1
1.1.1 Convertire i numeri tra i diversi sistemi numerici . . . 1
1.1.2 Numeri col segno . . . 2
1.2 Operazioni elementari . . . 4
1.2.1 Addizioni e sottrazioni . . . 4
1.2.2 Il problema dell'overflow . . . 5
1.2.3 Moltiplicazioni . . . 6
1.2.4 Shift di bit . . . 6
1.3 Numeri con la virgola . . . 7
1.3.1 Rappresentazione a virgola fissa . . . 7
1.3.2 Numeri con la virgola con segno . . . 7
1.3.3 Rappresentazione a virgola mobile . . . 8
1.3.4 Arrotondare numeri a virgola mobile . . . 10
1.3.5 Addizione con i numeri in virgola mobile . . . 11
1.3.6 Moltiplicazione con numeri in virgola mobile . . . 13

2 Circuiti logici **14**
2.1 Funzioni logiche . . . 14
2.1.1 Funzioni logiche a un input . . . 14
2.1.2 Funzioni logiche a più input . . . 15
2.2 Circuiti logici . . . 17
2.3 Equazioni booleane . . . 20
2.3.1 Somma di prodotti (SOP) . . . 20
2.3.2 Prodotto di somme (POS) . . . 21
2.3.3 Assiomi e teoremi . . . 21
2.3.4 Semplificare un'equazione . . . 23
2.3.5 Teorema di DeMorgan . . . 25
2.3.6 Circuiti a più uscite . . . 27
2.4 Mappe di Karnaugh . . . 28
2.4.1 Duale delle mappe di Karnaugh . . . 30
2.4.2 Usare le mappe di Karnaugh per visualizzare i teoremi . . . 30
2.4.3 Algoritmo Quine-McCluskey . . . 31
2.5 Circuiti a più livelli . . . 31
2.5.1 Multiplexer . . . 33
2.5.2 Decoder . . . 37

2.6 Dispositivi logici programmabili . . . 39
2.6.1 PLA . . . 39
2.6.2 FPGA . . . 41

3 Circuiti sequenziali **43**
3.1 Introduzione . . . 43
3.2 Elementi di stato . . . 43
3.2.1 Circuiti bistabili . . . 44
3.2.2 SR Latch . . . 45
3.2.3 D Latch . . . 46
3.2.4 D Flip-Flop . . . 47
3.2.5 Altri Flip-Flop . . . 52
3.3 Progettazione logica sequenziale sincrona . . . 54
3.4 FSM . . . 55
3.4.1 Moore FSM . . . 55
3.4.2 Mealy FSM . . . 60
3.4.3 Esempio di FSM . . . 63
3.4.4 Passare da un circuito di una FSM al diagramma degli stati . . . 70
3.4.5 Contatori . . . 73
3.5 Registri a scorrimento . . . 74
3.5.1 Registri a scorrimento con input paralleli . . . 75
3.5.2 Trasmettitore seriale . . . 76

4 Memorie **77**
4.1 Introduzione ai memory array . . . 77
4.1.1 Interno di un memory array . . . 78
4.2 Memorie RAM . . . 79
4.2.1 Confronto tra le diverse RAM e i Flip-Flop . . . 80
4.2.2 Chip DRAM . . . 81
4.2.3 DDR . . . 82
4.2.4 HBM . . . 84
4.2.5 Memoria multi-porta . . . 85
4.2.6 Costruire una memoria a partire da blocchi . . . 86

Capitolo 1

Sistemi numerici

1.1 Introduzione ai sistemi numerici

Il sistema numerico usato normalmente è il sistema numerico **decimale** (o in base 10). In questo, ogni cifra rappresenta una potenza di **10**: $5374_{10} = 5 \cdot 10^3 + 3 \cdot 10^2 + 7 \cdot 10^1 + 4 \cdot 10^0$. Esistono, oltre a questo sistema, anche molti altri sistemi in basi diverse, tra cui il sistema **binario** e il sistema **esadecimale**.

Definizione 1.1 – Sistema binario

Il sistema **binario** rappresenta ogni numero tramite solo due cifre: **0** e **1**. Ogni cifra rappresenta una potenza di **2**: $1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 1 = 13_{10}$.

Definizione 1.2 – Sistema esadecimale

Il sistema **esadecimale** rappresenta ogni numero tramite tutte le **10 cifre** e tramite le lettere che vanno dalla **A** alla **F** (che rappresentano i numeri da 10 a 15). Ogni cifra rappresenta una potenza di **16**: $4AF_{16} = 15 \cdot 16^0 + 10 \cdot 16^1 + 4 \cdot 16^2 = 15 + 160 + 1024 = 1199_{10}$.

1.1.1 Convertire i numeri tra i diversi sistemi numerici

Abbiamo visto come convertire qualsiasi numero in binario/esadecimale in decimale, ma vediamo come fare il contrario.

- **Decimale** $\longrightarrow$ **Binario**: vi sono due metodi:
  1° Metodo: prendere la più grande potenza di 2 minore o uguale al numero da voler convertire, e sottrarla, e continuare così finché non si arriva a 0. Ad esempio, $68 = 64+4$, dunque $68_{10} = 1000100_2$.
  2° Metodo: dividere il numero per 2 e posizionare il resto appena ottenuto a sinistra della rappresentazione binaria. Ad esempio, $68/2 = 34 + 0 \longrightarrow 34/2 = 17 + 0 \longrightarrow 17/2 = 8 + 1 \longrightarrow 8/2 = 4 + 0 \longrightarrow 4/2 = 2 + 0 \longrightarrow 2/2 = 1 + 0 \longrightarrow 1/2 = 0 + 1$, dunque $68_{10} = 1000100_2$.

- **Decimale ⟶ Esadecimale**: dividere il numero per 16 e posizionare il resto appena ottenuto a sinistra della rappresentazione esadecimale. Ad esempio, $68/16 = 4 + 4 \longrightarrow 4/16 = 0 + 4$, dunque $68_{10} = 44_{16}$.
- **Esadecimale ⟶ Binario**: passare da esadecimale a binario è molto semplice. Bisogna prendere ogni cifra dell'esadecimale e convertirla singolarmente in binario su 4 bit. Ad esempio, $F3A_{16} = 111100111010_2$, dato che $F = 1111$, $3 = 0011$ e $A = 1010$. Questo metodo è valido per passare da una qualsiasi base potenza di 2 al binario: basta convertire ogni cifra in un numero di bit pari all'esponente di tale potenza. Con l'esadecimale ne prendiamo 4 ($16 = 2^4$), con la base 8 invece ne usiamo solo 3 ($8 = 2^3$).
- **Binario ⟶ Esadecimale**: anche passare da binario a esadecimale è molto semplice. Bisogna partire da destra e dividere il numero in gruppi da 4 bit. Si trasformano poi tutti i gruppi nella corrispondente cifra esadecimale e si riscrivono nello stesso ordine. Ad esempio, $100100_2 = 24_{16}$, dato che $0100_2 = 4_{16}$ e $10_2 = 2_{16}$. Questo metodo è valido per passare dal binario a una qualsiasi base potenza di 2: basta dividere il numero in gruppi di lunghezza pari all'esponente di tale potenza. Per l'esadecimale i gruppi sono lunghi 4 ($16 = 2^4$), per la base 32 sono invece lunghi 5 ($32 = 2^5$).

Proprietà 1.1 – Potenze di 2

Conoscendo solo alcune potenze di 2, è possibile approssimare i valori di molte più potenze. Infatti, se si conoscono tutte le potenze da 2^0 a 2^9, e sapendo che $2^{10} \approx 1000, 2^{20} \approx 1000000, 2^{30} \approx 1000000000, \ldots$, si possono approssimare tutte le potenze tra 2^0 e 2^{39}. Infatti, $2^{34} = 2^{30} \cdot 2^4 \approx 1000000000 \cdot 16 = 16000000000$.

Definizione 1.3 – Definizioni sistema binario

Nel sistema binario, chiamiamo **bit** le cifre dei numeri. Un **nibble**, poi, sono 4 bit mentre un **byte** sono 8 bit. Esistono poi il **most significant bit** (msb) e **least significant bit** (lsb), che sono rispettivamente il bit più a sinistra e più a destra, e il **most significant byte** (MSB) e **least significant byte** (LSB), che sono rispettivamente il byte più a sinistra e più a destra.

Ad esempio, nel numero 1010101010, il primo 1 è il msb, mentre l'ultimo 0 è il lsb. Allo stesso modo, nel numero CEB48AE8 (rappresentato in esadecimale, dove ogni cifra rappresenta un nibble), il byte CE è il MSB, mentre il byte E8 è il LSB.

1.1.2 Numeri col segno

In qualsiasi sistema numerico, per rappresentare un numero negativo basta mettere il simbolo − prima del numero. Questo, però, per i computer non è possibile: lavorando solo con dei bit, non si può anteporre ad un numero il simbolo −. Infatti, il computer non ha idea di cosa sia

quel simbolo, conosce solo 0 e 1. Dunque bisogna trovare un'altra soluzione per rappresentare i numeri col segno. Vediamo i due modi più comuni:

- **Segno e Ampiezza**
- **Complemento a 2**

Proprietà 1.2 – Segno e Ampiezza

Segno e Ampiezza è una tecnica usata dai computer per rappresentare i numeri col segno. Si usa il **bit più significativo per rappresentare il segno**: se questo vale **0**, allora il numero è **positivo**, altrimenti è **negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = (-1)^{a_{N-1}} \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 6 tramite il Segno e Ampiezza diventa in binario 0110, mentre -6 diventa 1110. È dunque necessario utilizzare un bit in più del normale per rappresentare i numeri. Con N bit si possono quindi rappresentare tutti i numeri da $-(2^{N-1}-1)$ a $2^{N-1}-1$.

Il problema più grande di questa tecnica è l'addizione: semplicemente, questa non funziona. Se infatti proviamo a sommare -6 (1110) e 6 (0110), troviamo che

$$\begin{array}{cccccc} 1 & 1 & 1 & 0 & + & \\ 0 & 1 & 1 & 0 & = & \\ \hline 1 & 0 & 1 & 0 & 0 & = \quad 20_{10} \neq 0 \end{array}$$

Inoltre, un altro problema è che vi sono due rappresentazioni dello 0: infatti, 0 = 1000 = 0000.

Per questi motivi nasce un'altra tecnica, quella del **complemento a 2**.

Proprietà 1.3 – Complemento a 2

Complemento a 2 è un'altra tecnica usata per rappresentare i numeri col segno. Il **bit più significativo** ha lo stesso valore visto con i numeri senza segno, ma con il **segno negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = a_{N-1}(-2^{N-1}) + \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 7 tramite complemento a 2 diventa in binario 0111, mentre -7 diventa 1001. Con N bit si possono quindi rappresentare tutti i numeri da -2^{N-1} a $2^{N-1}-1$.

Con questa implementazione si risolvono entrambi i problemi del Segno e Ampiezza. Infatti, lo zero è rappresentato solo dalla sequenza 0000, e l'addizione non ha problemi: infatti, se proviamo a sommare -6 (1010) e 6 (0110), troviamo che

$$\begin{array}{cccccc} & 1 & 0 & 1 & 0 & + \\ & 0 & 1 & 1 & 0 & = \\ \hline \not{1} & 0 & 0 & 0 & 0 & = 0 \end{array}$$

dove l'uno iniziale non si conta dato che stiamo parlando di numeri a 4 bit.

Proprietà 1.4 – Convertire un numero nell'opposto tramite il Complemento a 2

Preso un numero in binario possiamo facilmente trasformarlo nell'opposto tramite il Complemento a 2. Infatti, basta **invertire i bit** e **aggiungere 1**. Preso ad esempio $6_{10} = 0110_2$, troviamo che $-6_{10} = 1001_2 + 1 = 1010 = -6_{10}$.

Questa proprietà funziona per un motivo molto semplice: sappiamo che $6+\bar{6} = 0110+1001 = 1111$, dunque aggiungendo un 1 troviamo che $6 + \bar{6} + 1 = 1111 + 1 = 10000$, e dato che il primo uno si elude, troviamo che $6 + \bar{6} + 1 = 0$, ossia $-6 = \bar{6} + 1 \longrightarrow$ **$-0110 = 1001 + 1$**.

1.2 Operazioni elementari

1.2.1 Addizioni e sottrazioni

Le addizioni in binario e in esadecimale funzionano esattamente come in decimale. Ad esempio,

$$\begin{array}{cccc} {\scriptstyle 1} & & & \\ 0 & 1 & 1 & + \\ 0 & 1 & 0 & = \\ \hline 1 & 0 & 1 & \end{array} \qquad \begin{array}{cccc} & {\scriptstyle 1} & & \\ A & 4 & F & + \\ 2 & A & 1 & = \\ \hline C & F & 0 & \end{array}$$

Anche per la sottrazione il metodo rimane lo stesso. Ad esempio,

$$\begin{array}{cccc} & & {\scriptstyle 1(2)} & \\ 0 & \not{1}0 & 0 & - \\ 0 & 0 & 1 & = \\ \hline 0 & 0 & 1 & \end{array} \qquad \begin{array}{cccc} & & {\scriptstyle 1(16)} & \\ 3 & \not{D}C & 9 & - \\ 1 & A & D & = \\ \hline 2 & 2 & C & \end{array}$$

1.2.2 Il problema dell'overflow

Osservazione 1.1 – Overflow

Un problema classico nelle addizioni tra numeri binari in un sistema digitale è l'**overflow**: i computer lavorano su un numero fisso di bit, e con un'addizione è possibile sforare questo numero.

Ad esempio, se lavoriamo con numeri a 4 bit, la somma 1001 + 1000 = 10001 sfora, creando diversi problemi. Infatti, con N bit abbiamo un range di numeri rappresentabili, da 0 a $2^N - 1$ (questo se parliamo di numeri senza segno).
Ci sono due modi per risolvere questo problema, a seconda che il numero sia con o senza il segno:

- **Sign-Extension**
- **Zero-Extension**

Proprietà 1.5 – Sign-Extension

La **sign-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero (in Complemento a 2) così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **ripetere il bit più significativo a sinistra fino ad arrivare al numero di bit desiderato** (noi raddoppieremo il numero di bit, così da esser coperti sia sulle addizioni che sulle moltiplicazioni, che vedremo tra poco).

Ad esempio, $3_{10} = 0011 = 00000011$, e $-5_{10} = 1011 = 11111011$. Questo con gli 0 ovviamente funziona, ma anche con gli 1: infatti, $-5 = 1011 = -8+2+1 = -16+8+2+1 = -32+16+8+2+1 = -64 + 32 + 16 + 8 + 2 + 1 = -128 + 64 + 32 + 16 + 8 + 2 + 1 = 11111011$.

Proprietà 1.6 – Zero-Extension

La **zero-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **aggiungere degli 0 a sinistra fino ad arrivare al numero di bit desiderato** (anche qui raddoppieremo il numero di bit).

Ad esempio, $3_{10} = 0011 = 00000011$ (come nella Sign-Extension). La Zero-Extension però **non funziona con i numeri col segno**. Infatti, $-5_{10} = 1011 \neq 00001011 = 11_{10}$.

1.2.3 Moltiplicazioni

Anche le moltiplicazioni in binario e in esadecimale funzionano come in decimale. Ad esempio,

	0	1	1	×
	0	0	1	=
	0	1	1	+
0	0	0		+
0	0	0		=
0	0	0	1	1

	3	B	7	×
	2	C	4	=
	E	D	C	+
2	C	9	4	+
7	6	E		=
A	4	6	1	C

1.2.4 Shift di bit

Definizione 1.4 – Shift

Uno **shift** è un'operazione sui bit che **sposta tutte le posizioni dei bit** di una sequenza binaria o a destra o a sinistra di un determinato numero di posizioni.

Esistono due tipi di shift:

- **shift aritmetico**, dove nello shift a destra si ripete il bit più significativo seguendo la logica della sign-extension;
- **shift logico**, dove si inseriscono degli 0 a sinistra della sequenza seguendo la logica della zero-extension.

Lo shift a sinistra invece è sempre uguale: sia nell'aritmetico che nel logico vengono inseriti degli 0 a destra della sequenza.

Ad esempio, $-7_{10} = 1001$ **>>** $2 = 0010 = 2_{10}$ è uno **shift logico** (si è infatti perso il segno, cosa che capita quando si usa la zero-extension), mentre $-8_{10} = 111000$ **>>>** $3 = 111111 = -1_{10}$ è uno **shift aritmetico**. Notiamo che, nello shift aritmetico, il nuovo numero si può ottenere prendendo l'originale e dividendolo per 2^N (arrotondando verso il basso), dove N è il valore dello shift. Nello shift logico questo vale solo se leggiamo il numero senza segno: $1001_2 = 9$ e $9/2^2 = 2.25$, cioè 2. Questa è una proprietà degli shift di destra. Allo stesso modo, per gli shift di sinistra, basta moltiplicare il valore per 2^N. Ad esempio, $-8_{10} = 111000$ << $2 = 100000 = -32 = -8 \cdot 2^2$.

Proposizione 1.1 – Calcolare numero decimale dopo uno shift

Preso un numero A_{10}, effettuando uno shift a **sinistra** di N posizioni, troviamo che $\boldsymbol{A_{10} << N = A_{10} \cdot 2^N}$. Se invece si effettua uno shift **aritmetico** a **destra**, troviamo che $\boldsymbol{A >>> N = A/2^N}$.

Se uno shift a destra restituisce in decimale un numero con la virgola, questo verrà approssimato sempre verso il basso. Ad esempio, preso $-13_{10} = 11110011_2$, se effettuiamo uno shift aritmetico di 2 verso destra troviamo che $11110011 >>> 2 = 11111100 = -4$, quando invece $-13/4 = -3.25$.

1.3 Numeri con la virgola

Siamo riusciti a rappresentare numeri interi senza segno e con il segno, ma dobbiamo ora rappresentare i **numeri con la virgola**. Abbiamo due possibili rappresentazioni:

- **Virgola fissa**
- **Virgola mobile**

1.3.1 Rappresentazione a virgola fissa

Proprietà 1.7 – Rappresentazione a virgola fissa

La rappresentazione a **virgola fissa** è una tecnica usata dai computer per rappresentare i numeri con la virgola. Si divide il numero in due sezioni: una rappresenta la parte intera, mentre l'altra rappresenta la **parte decimale**. Il numero di bit per la parte intera e quello per la parte decimale deve essere deciso a priori. Ad ogni cifra della parte frazionaria corrisponde una **potenza di due negativa**, partendo da 2^{-1}.

Esempio

Rappresentare **6.75** utilizzando **4 bit interi e 4 bit frazionari** (ossia per la parte decimale).

Sappiamo già che $6_{10} = 0110_2$. Per la parte decimale, abbiamo 4 cifre che ci permettono di calcolare tutti i valori da 0 a 15/16 con salti di 1/16. 0.75 diventa quindi 1100, dato che $1100 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 0.5 + 0.25 = 0.75$. Dunque troviamo che **6.75 = 01101100**.

In generale, per convertire da decimale a binario un numero con la virgola si possono usare entrambi i metodi già visti in precedenza, con una differenza per il 2° metodo: infatti, non dovremo più dividere per 2, ma dovremo moltiplicare la parte decimale per 2, e se questo numero è minore di uno allora segniamo 0, altrimenti segniamo 1 e sottraiamo 1 al valore ottenuto. Nell'esempio di prima, $0.75 \cdot 2 = 1.5 \geq 1$, dunque segniamo 1 e sottraiamo 1 ottenendo 0.5. Ora, dato che $0.5 \cdot 2 = 1$, segniamo 1 e sottraiamo 1, ottenendo 0, dunque tutte le cifre rimanenti sono 0, e otteniamo quindi il valore 1100.

1.3.2 Numeri con la virgola con segno

Dobbiamo ora combinare la rappresentazione a virgola fissa con il segno e ampiezza e con il complemento a 2.

- Con il **segno e ampiezza** è facilissimo: semplicemente il primo bit della parte intera sarà riservato al segno.
- Con il **complemento a 2** bisogna utilizzare l'algoritmo visto nella Proprietà 1.4.

Ad esempio, il numero -7.5 diventa con il segno e ampiezza **11111000**, mentre con il complemento a 2 bisogna prima calcolare 7.5, che è 01111000, poi invertire i bit e sommare 1, trovando quindi come valore **10001000**. Questo valore ha senso dato che $1 \cdot -2^3 + 1 \cdot 2^{-1} = -8 + 0.5 = -7.5$.

Proposizione 1.2

Utilizzare 4 bit per la parte decimale di un numero significa togliergli 4 bit per la parte intera. È quindi come se lo stessimo **dividendo per $2^4 = 16$**. Dunque un qualsiasi numero a 8 bit con la rappresentazione a virgola fissa è pari a quello stesso numero calcolato con 8 bit di parte intera diviso per 16.

Riprendendo l'esempio di -7.5, 10001000 con 8 bit di parte intera diventa -120, e infatti $-7.5 = -120/16$.

1.3.3 Rappresentazione a virgola mobile

Per studiare la rappresentazione a virgola mobile è necessario conoscere la **notazione scientifica**.

Definizione 1.5 – Notazione scientifica

La **notazione scientifica** è una tecnica usata per rappresentare i numeri nella forma $\pm M \cdot B^E$, dove M è la **mantissa**, B è la base e E è l'esponente.
N.B. In un sistema a base n, $\mathbf{1 \leq M < n}$. Dunque ad esempio nel sistema decimale la mantissa è compresa tra 1 e 10.

Ad esempio, nel nostro sistema decimale scriveremmo $2437 = 2.437 \cdot 10^3$.

Proprietà 1.8 – Rappresentazione a virgola mobile

La rappresentazione a **virgola mobile** è una tecnica usata dai computer per rappresentare i numeri con la virgola che sfrutta la **notazione scientifica**. Si stabilisce un numero di bit a priori per rappresentare un numero, come ad esempio **32 bit**, e si dividono questi bit per rappresentare il **segno** (che richiede un bit), l'**esponente** (che, in una rappresentazione a 32 bit, richiede 8 bit) e la **mantissa** (che si prende il resto dei bit, ossia 23 in una rappresentazione a 32 bit). La base, trovandoci in un sistema binario, è ovviamente **2**.

Per usare questa rappresentazione bisogna:

1° **Convertire il numero da decimale a binario**
2° **Scriverlo nella notazione scientifica binaria** (vedremo ora come farlo)
3° **Scriverlo nella notazione segno-esponente-mantissa**

Vi sono però altre due importanti considerazioni da fare:

- l'1 iniziale della mantissa è **sottinteso**. Questo perché è sempre presente in tutti i numeri dato che $M \geq 1$.
- l'esponente è **polarizzato**, ossia non si usa l'esponente base ma gli si aggiunge un certo **bias**, che nel caso dei numeri a 32 bit vale 127. Questo lo si fa per poter rappresentare anche gli esponenti negativi.

Esempio

Rappresentare il numero $\mathbf{-228}_{10}$ usando la **rappresentazione a virgola mobile con 32 bit**.

Per prima cosa, quindi, convertiamo 228 in binario (il segno non ci interessa, vi è un bit apposta in questa rappresentazione che se ne occupa): $228_{10} = 11100100_2$. A questo punto dobbiamo scriverlo nella notazione scientifica binaria, in un processo che prende il nome di **normalizzazione**: $11100100_2 = 1.11001_2 \cdot 2^7$. Aggiungiamo il bias all'esponente che diventa $127 + 7 = 134$, e riscriviamo tale numero nella sua rappresentazione a 32 bit finale:

$$1|10000110|11001000000000000000000_2$$

il cui valore esadecimale è $0xC3640000$ (dove $0x$ indica l'inizio del valore esadecimale).

Vi sono diverse convenzioni da stabilire per questa rappresentazione, dunque si utilizza uno standard che prende il nome di **IEEE 754**.

Definizione 1.6 – IEEE 754

L'**IEEE 754** è lo standard utilizzato per la rappresentazione a virgola mobile.

Racchiudiamo in questa tabella tutti i casi speciali della rappresentazione a virgola mobile, che sono stati così standardizzati sotto l'IEEE 754.

Number	Sign	Exponent	Fraction
0	Indifferente	00000000	00000000000000000000000
∞	0	11111111	00000000000000000000000
$-\infty$	1	11111111	00000000000000000000000
NaN	Indifferente	11111111	non-zero
Denormalizzati	Indifferente	00000000 (indica −126 e non −127)	$M \neq 0$ $(-1)^s \cdot 2^{-126} \cdot 0.m$

Dunque lo 0 si rappresenta con il codice esadecimale 0x00000000 o il codice 0x80000000 (vi sono anche qui, come nel Segno e Ampiezza, due rappresentazioni per esso).
Quando invece vogliamo rappresentare un'operazione indefinita, come $\infty - \infty$ o $0/0$, usiamo il **NaN** (Not a Number).
Nella tabella è anche rappresentato un altro genere di numeri: i numeri **denormalizzati** (o subnormali). Questi numeri sono numeri la cui mantissa non inizia con 1, perché si vogliono rappresentare numeri ancora più piccoli di 2^{-126} (che sarebbe il minimo consentito dai numeri normalizzati dato che l'esponente 00000000 è riservato allo 0). Infatti, si è pensato che dividendo il valore della mantissa per 2 si potesse diminuire ancora di più. Ad esempio, $0.1 \cdot 2^{-126}$ ha sempre come esponente -126, ma ha come mantissa 0.1 e non 1. In questo caso, però, si deve quindi togliere l'1 sottinteso di cui avevamo parlato precedentemente, e si perde anche precisione: infatti i bit rimanenti per la mantissa vera e propria diminuiscono sempre di più a favore di numeri più piccoli. In questo modo, però, è possibile arrivare fino a 2^{-149} con soli 32 bit.

Proprietà 1.9 – Livello di precisione

In base al numero di bit, si può raggiungere un livello di precisione più o meno alto. Vi sono 3 standard conosciuti:

- **Half-Precision** (16 bit)
- **Single-Precision** (32 bit, che utilizzeremo quasi sempre)
- **Double-Precision** (64 bit)

Per la Half-Precision vengono usati 5 bit per l'esponente e 10 per la mantissa, mentre per la Double-Precision 11 sono riservati all'esponente e 52 alla mantissa (ovviamente in entrambi i casi c'è sempre un bit riservato al segno).

1.3.4 Arrotondare numeri a virgola mobile

Abbiamo già affrontato il problema dell'overflow, ossia quando un numero sfora il numero di bit consentiti (esiste poi anche l'**underflow**, ossia quando un numero è così piccolo da non poterlo rappresentare). Questo problema è risolvibile in parte grazie alla sign-extension o alla zero-extension, ma ci sono numeri che da un sistema binario non sono rappresentabili. Ad esempio, $1/3$ non è rappresentabile: si avrebbe bisogno di un numero di bit infinito. È per questo che in informatica spesso bisogna **arrotondare**.

Proprietà 1.10 – Arrotondare

Esistono quattro modi per arrotondare un numero:

1) per **difetto**
2) per **eccesso**
3) più **vicino allo zero** (ossia se è positivo per difetto, se è negativo per eccesso)
4) al **più vicino possibile** (default, e in caso di parità arrotonda al valore il cui bit meno significativo è pari, ossia 0)

Per capire meglio questi concetti, vediamo come arrotondare 1.100101 (1.578125) a solo 3 bit dopo la virgola con tutti e 4 i modi.

1) **1.100**
2) **1.101**
3) **1.100**
4) **1.101** (dato che la parte scartata 101 è maggiore della metà dell'intervallo 100)

1.3.5 Addizione con i numeri in virgola mobile

L'addizione tra numeri in virgola mobile segue dei passaggi molto specifici:

1° Si estraggono l'esponente e i bit frazionari
2° Si comparano gli esponenti, e si shifta la mantissa più piccola se necessario
3° Si sommano le mantisse
4° Si normalizza la mantissa e si aggiusta l'esponente se necessario
5° Si arrotonda il risultato
6° Si riscrive il numero nel formato a virgola mobile

Può sembrare difficile, ma vediamo un esempio per chiarirci le idee. Sommiamo i numeri $0x3FC00000$ e $0x40500000$.

Per prima cosa dobbiamo quindi estrarre l'esponente e i bit frazionari di entrambi i numeri:

$$3FC00000_{16} = 0|01111111|10000000000000000000000$$
$$40500000_{16} = 0|10000000|10100000000000000000000$$

La mantissa del primo numero è quindi 1.1, mentre per il secondo è 1.101. Gli esponenti sono rispettivamente 127 e 128, ossia 0 e 1 tolto il bias, e dato che $0 - 1 = -1$, dobbiamo fare uno shift verso destra della mantissa del primo numero di 1. Dunque $1.1 >> 1 = 0.11(\cdot 2^1)$. Possiamo ora sommare le due mantisse:

$$\begin{array}{cccccc} & 0. & 1 & 1 & 0 & + \\ & 1. & 1 & 0 & 1 & = \\ \hline 1 & 0. & 0 & 1 & 1 & \end{array}$$

Dobbiamo ora normalizzare la mantissa: $10.011 \cdot 2^1 = 1.0011 \cdot 2^2$. Non serve arrotondare nulla, dato che i bit frazionari sono ben meno di 23, dunque possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000001|00110000000000000000000 = 0x40980000_{16}$$

Vediamo ora cosa succede se invece prendiamo due numeri con segno opposto, come ad esempio $0x40C00000$ e $0xC0200000$.

Per prima cosa estraiamo l'esponente e i bit frazionari:

$$40C00000_{16} = 0|10000001|10000000000000000000000$$
$$C0200000_{16} = 1|10000000|01000000000000000000000$$

La mantissa del primo numero è 1.1, mentre quella del secondo è 1.01. Gli esponenti sono rispettivamente 129 e 128, ossia 2 e 1, e dato che $2 - 1 = 1$, dobbiamo fare uno shift verso destra della mantissa del secondo numero di 1. Dunque $1.01 >> 1 = 0.101$ ($\cdot 2^2$). Dato che però il secondo numero è negativo, ne facciamo il complemento a 2, così da rappresentarlo come negativo e trasformare la sottrazione in una somma. Prendiamo dunque la mantissa (0.101), cambiamo tutti i bit (1.010), e aggiungiamo 0.001 (1.011). Ora possiamo sommare le mantisse:

$$\begin{array}{cccccc} & 1. & 1 & 0 & 0 & + \\ & 1. & 0 & 1 & 1 & = \\ \hline 10. & 1 & 1 & 1 & & \end{array}$$

Dalla mantissa eliminiamo il primo bit perché va in overflow. Dato che la somma ha prodotto questo riporto, il risultato è positivo (se non ci fosse stato riporto, il risultato sarebbe stato negativo e avremmo dovuto rifare il complemento a 2). Possiamo quindi normalizzare la mantissa: $0.111 \cdot 2^2 = 1.11 \cdot 2^1$. Non dobbiamo arrotondare nulla, quindi possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000000|11000000000000000000000 = 0x40600000$$

N.B. L'addizione tra numeri a virgola mobile **non è associativa**.

1.3.6 Moltiplicazione con numeri in virgola mobile

Anche la moltiplicazione tra numeri in virgola mobile è abbastanza schematica:

1° Si estraggono gli esponenti e i bit frazionari
2° Si sommano gli esponenti (togliendo poi un bias, dato che sennò ne avremmo due)
3° Si fa il prodotto tra le mantisse
4° Si calcola il segno (se i segni sono discordi allora è -, altrimenti è +)
5° Si normalizza il tutto se necessario
6° Si riscrive il numero nel formato a virgola mobile

Proviamo a moltiplicare $0x40400000$ e $0x40200000$.

Per prima cosa estraiamo quindi l'esponente e i bit frazionari da entrambi i numeri:

$$40400000_{16} = 0|10000000|10000000000000000000000$$
$$40200000_{16} = 0|10000000|01000000000000000000000$$

La mantissa del primo numero è 1.1, mentre quella del secondo è 1.01. Gli esponenti sono entrambi 128, dunque l'esponente finale è $128 + 128 - 127 = 129$. Eseguiamo ora il prodotto tra le mantisse:

$$\begin{array}{cccccc} & & 1. & 1 & 0 & \times \\ & & 1. & 0 & 1 & = \\ \hline & & 1 & 1 & 0 & + \\ & 0 & 0 & 0 & & + \\ 1 & 1 & 0 & & & = \\ \hline 1. & 1 & 1 & 1 & 0 & \end{array}$$

Non dovendo normalizzare nulla, possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000001|11100000000000000000000 = 0x40F00000$$

Capitolo 2

Circuiti logici

2.1 Funzioni logiche

Definizione 2.1 – Funzione logica

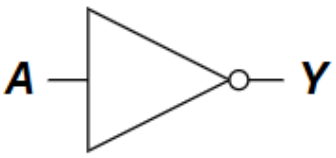
Una **funzione logica** (o booleana) è una funzione matematica definita su variabili binarie (ossia che possono valere solo 0 o 1, dove 0 = Falso e 1 = Vero) che restituisce a sua volta un valore di 0 o 1 in uscita.

Esistono diverse funzioni logiche:

- Funzioni logiche a un input: **`NOT` gate, buffer**
- Funzioni logiche a più input: **`AND`, `OR`, `XOR`, `NAND`, `NOR`, `XNOR`**

Useremo spesso le **tabelle di verità** per comprendere come funzionano queste funzioni logiche.

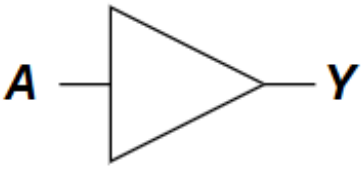
2.1.1 Funzioni logiche a un input



Il **`NOT` gate** si rappresenta tramite un triangolo con un pallino finale (questo pallino lo vedremo con qualsiasi funzione che utilizza l'operatore `NOT` per negare qualcosa). In questo caso abbiamo $Y = \bar{A}$ (indichiamo con una barretta l'inverso di una variabile, dunque se A è 0 $\bar{A} = 1$ e viceversa), che si rappresenta così tramite la tabella di verità:

A	Y
0	1
1	0

Dunque il `NOT` gate **inverte il valore in input**.

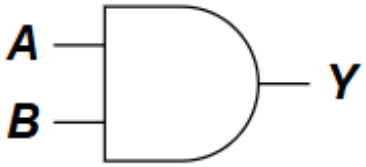


Il **buffer** si rappresenta tramite un triangolo. In questo caso abbiamo $Y = A$, che si rappresenta così tramite la tabella di verità:

A	Y
0	0
1	1

Dunque il buffer **lascia invariato il valore in input**.

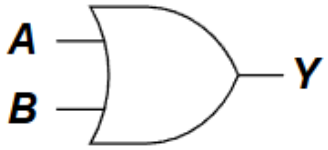
2.1.2 Funzioni logiche a più input



L'**AND** si rappresenta tramite un rettangolo unito a un semicerchio. In questo caso abbiamo $Y = AB$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

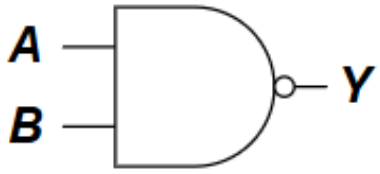
Dunque l'AND ritorna vero se **entrambi i valori sono veri**.



L'**OR** si rappresenta tramite tre archi uniti a formare una specie di triangolo. In questo caso abbiamo $Y = A + B$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

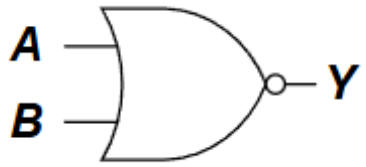
Dunque l'OR ritorna vero se **almeno uno dei due valori è vero**.



Il **NAND** si rappresenta con la stessa figura dell'AND ma termina con un pallino. In questo caso abbiamo $Y = \overline{AB}$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

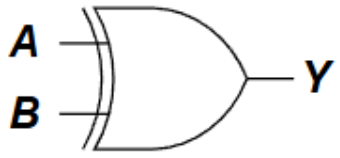
Dunque il NAND ritorna vero se **almeno uno dei due valori è falso** (il contrario dell'AND).



Il **NOR** si rappresenta con la stessa figura dell'OR ma termina con un pallino. In questo caso abbiamo $Y = \overline{A + B}$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

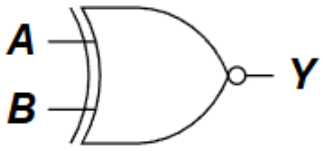
Dunque il NOR ritorna vero se **entrambi i valori sono falsi** (il contrario dell'OR).



Lo **XOR** si rappresenta con la stessa figura dell'OR ma ha un doppio arco alla base. In questo caso abbiamo $Y = A \oplus B$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Dunque lo XOR ritorna vero se **i due valori sono diversi**.



Lo **XNOR** si rappresenta con la stessa figura dello XOR ma termina con un pallino. In questo caso abbiamo $Y = \overline{A \oplus B}$, che si rappresenta così tramite la tabella di verità:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Dunque lo XNOR ritorna vero se **i due valori sono uguali** (il contrario dello XOR).

Proprietà 2.1 – Proprietà dispari dello XOR

Prese tante variabili booleane $X_1, \ldots, X_n$,

$$X_1 \oplus \ldots \oplus X_n = \begin{cases} \mathbf{1} & \textbf{se il numero di 1 è dispari} \\ \mathbf{0} & \textbf{se il numero di 1 è pari} \end{cases}$$

Possiamo quindi dire che ogni 1 aggiunto allo XOR **inverte** il risultato.

Dimostrazione

Dimostreremo questa proprietà solo con 3 variabili, ma vale con uno XOR di n variabili qualsiasi. Prese A, B, C, essendo lo XOR associativo sappiamo che $A \oplus B \oplus C = (A \oplus B) \oplus C$. Poniamo $Y = A \oplus B$. Dunque

$$Y \oplus C = \begin{cases} Y & \text{se } C = 0 \\ \overline{Y} & \text{se } C = 1 \end{cases}$$

Dunque se $C = 0$, allora avremo che $A \oplus B \oplus C = A \oplus B$ che vale 0 se il numero di 1 è pari e 1 se invece è dispari. Se invece $C = 1$, allora avremo che $A \oplus B \oplus C = \overline{A \oplus B}$ che vale 0 se il numero di 1 è dispari e 1 se invece è pari. Dunque ogni 1 che si aggiunge allo XOR inverte il risultato. ■

2.2 Circuiti logici

Definizione 2.2 – Circuito logico

Un **circuito logico** è un circuito elettrico digitale dotato di una serie di input e output che elaborano segnali binari.
N.B. Sintetizzare una funzione booleana f significa creare un circuito logico che rappresenti tale funzione, mentre **analizzare** un circuito significa ricavarne la funzione booleana.

Il comportamento logico è descritto dalla **specifica funzionale**, che descrive cosa fa il circuito. Inoltre, è a volte presente una **specifica temporale**, che ne descrive i ritardi, cioè quanto tempo passa tra la variazione degli input e la risposta dell'output.

Un circuito logico è formato da due elementi fondamentali:

- I **nodi** rappresentano i fili che trasportano il segnale binario, e possono essere ingressi, uscite o nodi interni.
- Gli **elementi del circuito** sono i blocchi logici che formano un circuito, e possono essere una singola porta logica o altri sotto-circuiti.

Esistono due tipi di circuiti logici:

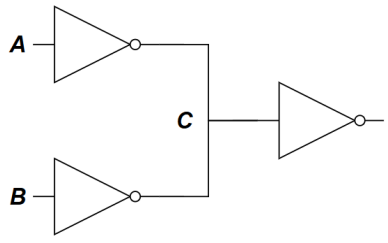
- **Circuiti combinatori**, che non hanno memoria e il cui output è determinato solo dai valori di input correnti
- **Circuiti sequenziali**, che invece hanno memoria e il cui output dunque è determinato non solo dai valori correnti ma anche da quelli passati

Un esempio classico di circuiti sequenziali è la macchinetta delle bevande: se si vuole prendere la bevanda indicata con il numero 34, bisognerà schiacciare prima il 3 e poi il 4. Ma noi non schiacciamo i tasti in contemporanea, ma in modo sequenziale. Dunque abbiamo bisogno di un circuito che si ricordi che abbiamo premuto prima il 3 e poi il 4, e non che dimentichi tutto ogni volta.

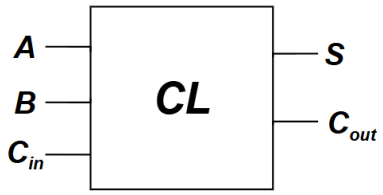
Proprietà 2.2 – Regole dei circuiti combinatori

Ci sono diverse regole da rispettare per creare dei circuiti combinatori funzionanti:

1) ogni elemento è **combinatorio**
2) ogni nodo è o un **input** o è **collegato all'output di un solo componente interno**
3) il circuito **non ha cicli**



Questo ad esempio non è un circuito combinatorio corretto. Infatti, il nodo C si trova in un valore intermedio non determinato: non sappiamo se vale 1 o 0 dato che non vi è nessuna porta logica che prenda in input i due valori $\bar{A}$ e $\bar{B}$ per creare un unico output sensato. Il nodo C è quindi **collegato all'output di due componenti interni**, violando così la regola 2). Parliamo in questo caso di **contention**, e scriviamo $C = X$, che rappresenta un valore ignoto.

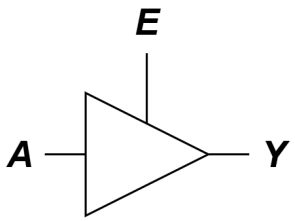


Questo invece è uno dei circuiti combinatori dell'architettura degli elaboratori: si chiama **Full Adder** e serve per eseguire delle somme di bit. In questo caso prendiamo i due bit da sommare A e B e il riporto proveniente da un'eventuale somma precedente C_{in}. L'output è $S = A \oplus B \oplus C_{in}$, che sarebbe la somma tra bit, e C_{out}, l'eventuale riporto generato.

N.B. Il blocco CL centrale indica un circuito logico. Si potrebbe pensare che in questo caso avremmo potuto mettere uno `XOR` a 3 input, ma questo non funziona: infatti, uno `XOR` rilascia solo un output, mentre noi in questo caso abbiamo un circuito che rilascia due output. Dunque creiamo questo blocco CL che rappresenta un circuito logico di cui i dettagli implementativi non ci interessano.

Definizione 2.3 – Alta impedenza

Un nodo si trova nello stato di **alta impedenza** quando si stacca elettricamente dal circuito e non influisce più in alcun circuito logico. Per indicare questo stato di alta impedenza usiamo la lettera **Z**.



Nei circuiti logici per far passare un nodo a uno stato di alta impedenza si usa il **Tristate Buffer** (letteralmente un buffer a tre stati, 0, 1 e Z). Oltre al nodo in input A e all'output Y, vi è un terzo nodo E che serve per attivare e disattivare il buffer. Se infatti $E = 0$, allora il buffer è inattivo e $Y = Z$ sempre, se invece $E = 1$ allora il buffer è attivo e quindi prende il segnale in input ponendo $Y = A$.

Questi tristate buffer sono fondamentali per creare dei circuiti dove non ci sono tante linee di dati, ma ce n'è una sola che collega tutti i diversi componenti. Facciamo un esempio per capire quanto sono importanti.

Abbiamo un circuito diviso in 4 componenti: processore, video, ethernet e memoria. Vogliamo collegare questi quattro componenti in modo che tutti possano comunicare con tutti gli altri, ma solo uno alla volta. Senza un bus condiviso dovremmo creare dei collegamenti tra tutte le possibili coppie di componenti, arrivando a 6 linee totali. Con il bus condiviso invece ne possiamo usare una sola, risparmiando moltissime risorse, ed è possibile anche gestire in modo più efficiente la linea. Infatti, dato che vogliamo che solo un componente alla volta possa parlare, possiamo mettere sulla linea di scrittura di ogni componente un tristate buffer. In questo modo, se un componente vuole parlare con un altro, tutti gli altri componenti verranno mandati in alta impedenza, così da non poter più mandare dati sul bus, rimanendo solo in ascolto.

2.3 Equazioni booleane

Definizione 2.4 – Equazioni booleane

Un'**equazione booleana** è un'uguaglianza matematica tra due espressioni composte da variabili booleane e operatori logici.

Definizione 2.5 – Definizioni dell'algebra di Boole

L'**algebra di Boole** è quella branca dell'algebra che si occupa di studiare le equazioni booleane. Vi sono diversi termini necessari da conoscere per poterla studiare:

- **Complemento**: una variabile con una barra ($\bar{A}$)
- **Literal**: una variabile o il suo complemento (A o $\bar{B}$)
- **Implicante**: un prodotto tra literal (AC o $A\bar{B}$)
- **Mintermine**: un prodotto che include tutte le variabili in input ($AB\bar{C}$, se come input abbiamo A, B, C)
- **Maxtermine**: una somma che include tutte le variabili in input ($A + \bar{B} + C$, se come input abbiamo A, B, C)

2.3.1 Somma di prodotti (SOP)

Tutte le equazioni booleane possono essere scritte come **somma di mintermini**. Vediamo come facendo un esempio con due variabili.

Prese A e B, sappiamo che abbiamo 4 possibili combinazioni di valori con queste due variabili:

A	B
0	0
0	1
1	0
1	1

Ognuna di queste righe ha un proprio **mintermine**, che è **vero solo e soltanto per quella riga**. Abbiamo quindi

A	B	**Mintermine**
0	0	$m_0 = \bar{A}\bar{B}$
0	1	$m_1 = \bar{A}B$
1	0	$m_2 = A\bar{B}$
1	1	$m_3 = AB$

Adesso, ogni equazione booleana Y ha per ogni riga un risultato prestabilito. Ad esempio, $Y = A + B$ vale 1 per ogni riga tranne che per la prima. Dunque possiamo riscrivere Y come **somma di tutti i mintermini delle righe dove $Y = 1$**. Dunque $Y = A + B = \bar{A}B + A\bar{B} + AB = m_1 + m_2 + m_3 = \sum(1, 2, 3)$. Questa forma prende il nome di **forma canonica della SOP**.
N.B. Per convenzione usiamo come indice di ogni mintermine il valore delle variabili della riga che rappresenta trasformato in binario. Ad esempio, quando $A = 0$ e $B = 1$ abbiamo il mintermine $01_2 = 1_{10}$. La stessa convenzione la vedremo anche per i maxtermini.

2.3.2 Prodotto di somme (POS)

Oltre alla somma di mintermini, ogni equazione booleana può essere riscritta anche come **prodotto di maxtermini**. Vediamo anche qui un esempio con due variabili.

Riprendiamo le variabili A e B. Ogni riga della tabella che rappresenta le loro possibili combinazioni di valori ha un proprio **maxtermine**, che è **falso solo e soltanto per quella riga** (dato che i maxtermini danno come output 1 con tre combinazioni su quattro essendo degli `OR`, prendiamo per ogni riga il maxtermine falso e non vero). Abbiamo quindi

A	B	**Maxtermine**
0	0	$M_0 = A + B$
0	1	$M_1 = A + \bar{B}$
1	0	$M_2 = \bar{A} + B$
1	1	$M_3 = \bar{A} + \bar{B}$

Dunque ogni equazione booleana Y sappiamo già che ha per ogni riga un risultato prestabilito (Y con $A = 1$ e $B = 0$ vale sempre o 1 o 0, qualsiasi funzione sia è già scritto). Ad esempio, $Y = AB$ vale 0 per ogni riga tranne che per l'ultima. Dunque possiamo riscrivere Y come **prodotto di tutti i maxtermini delle righe dove $Y = 0$**. Dunque $Y = AB = (A + B)(A + \bar{B})(\bar{A} + B) = M_0M_1M_2 = \prod(0, 1, 2)$. Questa forma prende il nome di **forma canonica della POS**.

2.3.3 Assiomi e teoremi

Per semplificare l'algebra booleana vi è una serie di assiomi e teoremi che bisogna conoscere. Alcuni sono molto scontati e semplici, altri invece sono meno intuitivi e più complessi.

Definizione 2.6 – Dualità dell'algebra booleana

Nell'algebra booleana vi è una **dualità** tra le operazioni `AND` e `OR` e i valori 1 e 0: infatti, è possibile trasformare qualsiasi operazione con questi operatori e questi valori semplicemente scambiando gli `AND` con gli `OR` e gli 1 con gli 0 e viceversa.
N.B. Questa dualità si applica SOLO alle **identità**, e non a tutte le equazioni booleane. Ci sarà quindi molto utile nei teoremi e negli assiomi che vedremo tra poco.

Ad esempio, presa l'identità $A + 1 = 1$ (A `OR` $1 = 1$), se applichiamo le regole di dualità otteniamo l'identità $A \cdot 0 = 0$ (A `AND` $0 = 0$), che è anch'essa vera.

Proprietà 2.3 – Assiomi

Numero	Assioma	Duale
1	$B \neq 1 \implies B = 0$	$B \neq 0 \implies B = 1$
2	$\bar{0} = 1$	$\bar{1} = 0$
3	$0 \cdot 0 = 0$	$1 + 1 = 1$
4	$1 \cdot 1 = 1$	$0 + 0 = 0$
5	$0 \cdot 1 = 1 \cdot 0 = 0$	$1 + 0 = 0 + 1 = 1$

Proprietà 2.4 – Teoremi a una variabile

Numero	Teorema	Duale
1	$B \cdot 1 = B$	$B + 0 = B$
2	$B \cdot 0 = 0$	$B + 1 = 1$
3	$B \cdot B = B$	$B + B = B$
4	$\bar{\bar{B}} = B$	
5	$B \cdot \bar{B} = 0$	$B + \bar{B} = 1$

Per dimostrare questi teoremi abbiamo due metodi:

1) **Induzione perfetta** (o dimostrazione per esaustione), dove in poche parole si provano tutte le combinazioni possibili in input e si dimostra che entrambe le espressioni producono gli stessi output
2) **Usare altri teoremi e assiomi**, così da semplificare l'espressione

Proprietà 2.5 – Teoremi a più variabili

Nome	Teorema	Duale
P. Commutativa	$B \cdot C = C \cdot B$	$B + C = C + B$
P. Associativa	$(B \cdot C) \cdot D = B \cdot (C \cdot D)$	$(B + C) + D = B + (C + D)$
P. Distributiva	$B \cdot (C + D) = (B \cdot C) + (B \cdot D)$	$B + (C \cdot D) = (B + C)(B + D)$
1° T. Assorbimento	$B \cdot (B + C) = B$	$B + (B \cdot C) = B$
2° T. Assorbimento	$(B \cdot C) + (B \cdot \bar{C}) = B$	$(B + C) \cdot (B + \bar{C}) = B$

N.B. Il duale della proprietà distributiva ci dice che $B + (C \cdot D) = (B + C)(B + D)$, che non coincide con le proprietà dell'algebra che conosciamo, dove l'operatore + non si distribuisce.

Dimostrazione

Dimostriamo solo i due teoremi di assorbimento (ricordiamo che il duale NON va dimostrato, perché dimostrando il teorema stiamo dimostrando anche il suo duale).

- 1° Teorema di assorbimento: $B \cdot (B + C) = B$
  Dimostriamolo per induzione perfetta:

B	C	$B(B + C)$
0	0	0
0	1	0
1	0	1
1	1	1

  Dato che B e $B \cdot (B + C)$ per ogni input restituiscono lo stesso output, possiamo concludere che $B \cdot (B + C) = B$. ■
- 2° Teorema di assorbimento: $(B \cdot C) + (B \cdot \bar{C}) = B$
  Dimostriamolo usando altri assiomi e teoremi:

$$\begin{aligned} (B \cdot C) + (B \cdot \bar{C}) &= B \cdot (C + \bar{C}) && \text{P. Distributiva al contrario} \\ &= B \cdot 1 && \text{Teorema 5} \\ &= B && \text{Teorema 1} \end{aligned}$$

  Dunque troviamo che $(B \cdot C) + (B \cdot \bar{C}) = B$. ■

2.3.4 Semplificare un'equazione

Definizione 2.7 – Semplificare un'equazione

Semplificare un'equazione (o minimizzare un'equazione) significa **ridurre** un'equazione al minor numero di implicanti, dove ogni implicante deve avere il minor numero di literal.

Per semplificare un'equazione sono molto utili in particolare la Proprietà distributiva e i due teoremi di assorbimento. Sarà molto utile anche vedere queste proprietà/teoremi al contrario: ad esempio, a volte ci capiterà di semplificare passando da $(B + C)(B + D)$ a $B + CD$ piuttosto che il contrario.
Altre tecniche di semplificazione utili sono l'**espansione** e la **duplicazione**. L'espansione è praticamente il fenomeno inverso dei due teoremi di assorbimento. Ad esempio, a volte sarà utile passare da P a $P \cdot (P + A)$, oppure da P a $PA + P\bar{A}$. La duplicazione è invece banalmente il riscrivere un literal A come $A + A$. Sembrano dei procedimenti inutili, ma possono esserci equazioni molto grandi semplificabili tramite queste tecniche.

È importante capire che esistono diversi modi per semplificare un'equazione: alcuni più veloci, altri più lenti. Il nostro obiettivo è quello di cercare la strada più veloce, così da ridurre il numero di passaggi e quindi di errori.

Esempi

Semplificare l'equazione $\boldsymbol{A(AB + ABC)}$.

Banalmente

$$\begin{aligned} A(AB + ABC) &= A(AB(1 + C)) && \text{P. Distributiva al contrario} \\ &= A(AB \cdot 1) && \text{Duale del teorema 2} \\ &= A \cdot AB && \text{Teorema 1} \\ &= AA \cdot B && \text{P. Associativa} \\ &= \boldsymbol{AB} && \text{Teorema 3} \end{aligned}$$

Semplificare l'equazione $\boldsymbol{(A + BC)(A + DE)}$.

Questa equazione utilizzando il duale della Proprietà Distributiva al contrario si semplifica subito: infatti, se poniamo $X = BC$ e $Y = DE$, troviamo proprio $(A + X)(A + Y) = A + XY = \boldsymbol{A + BCDE}$.

Proposizione 2.1 – Obiettivo della semplificazione

L'obiettivo della semplificazione di un'equazione è quasi sempre quello di ridurci a una **SOP** (somma di prodotti, come abbiamo fatto negli esempi precedenti) o una **POS** (prodotto di somme).

Esempi

Trasformare nella forma **POS** l'equazione $\boldsymbol{AB + C'DE + F}$.

N.B. C' equivale a scrivere $\bar{C}$.

Notiamo come anche in questo esempio non ci resti che eseguire tante volte il duale della Proprietà Distributiva al contrario. Infatti,

$$\begin{aligned} AB + C'DE + F &= (AB + C')(AB + DE) + F \\ &= (A + C')(B + C')(A + DE)(B + DE) + F \\ &= F + (A + C')(B + C')(A + D)(A + E)(B + D)(B + E) \\ &= (F + (A + C')(B + C')(A + D))(F + (A + E)(B + D)(B + E)) \\ &= (F + A + C')(F + (B + C')(A + D))(F + A + E)(F + (B + D)(B + E)) \\ &= \mathbf{(F + A + C')(F + B + C')(F + A + D)(F + A + E)(F + B + D)(F + B + E)} \end{aligned}$$

N.B. Qui ci aspettavamo 6 termini moltiplicativi nell'ultima espressione dato che avevamo 3 implicanti inizialmente, uno con 3 literal, uno con 2 e uno con 1, dunque $3 \cdot 2 \cdot 1 = 6$.

Trasformare nella forma **POS** l'equazione $\boldsymbol{A + B'CDE}$.

Seguendo lo stesso ragionamento visto nell'esempio precedente, troviamo subito che $A+B'CDE = \mathbf{(A + B')(A + C)(A + D)(A + E)}$. Abbiamo anche qui 4 termini moltiplicativi come previsto.

2.3.5 Teorema di DeMorgan

Proprietà 2.6 – Teorema di DeMorgan

Il **teorema di DeMorgan** è uno dei teoremi più importanti dell'algebra di Boole:

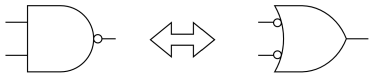
DeMorgan	Duale
$\overline{B_0 \cdot B_1 \cdot B_2 \ldots} = \overline{B_0} + \overline{B_1} + \overline{B_2} + \ldots$	$\overline{B_0 + B_1 + B_2 + \ldots} = \overline{B_0} \cdot \overline{B_1} \cdot \overline{B_2} \cdot \ldots$

In sostanza, il **complemento del prodotto è la somma dei complementi** e il **complemento della somma è il prodotto dei complementi**.

Questo teorema è fondamentale nei circuiti logici, ed è alla base proprio del processo chiamato **Bubble pushing**.

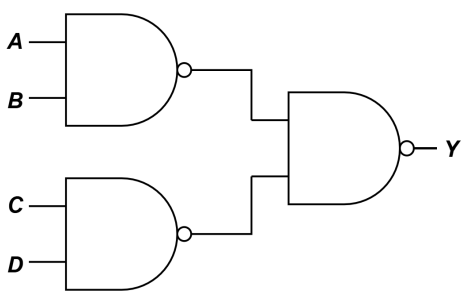
Definizione 2.8 – Bubble pushing

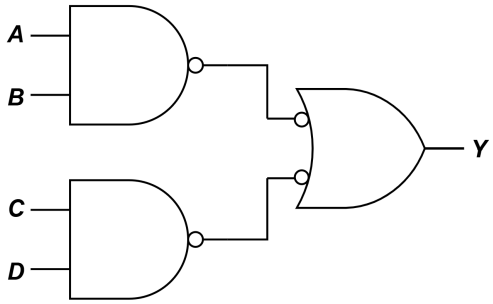
Il **bubble pushing** è una tecnica utilizzata nei circuiti per passare da circuiti `NOR` a `AND` e viceversa o da `NAND` a `OR` e viceversa.



Grazie al teorema di DeMorgan, riusciamo a dimostrare che i due circuiti qui a sinistra sono identici. I pallini in corrispondenza dell'input dell'`OR` indicano un valore `NOT`. Se ad esempio avevamo un `NAND` che collegava A e B, lo avremmo ora trasformato in un `OR` che collega $\bar{A}$ e $\bar{B}$. Lo stesso ragionamento varrebbe anche con un `NOR` che diventerebbe un `AND`.

Dunque possiamo trasformare ogni circuito logico in un altro equivalente usando questa tecnica.

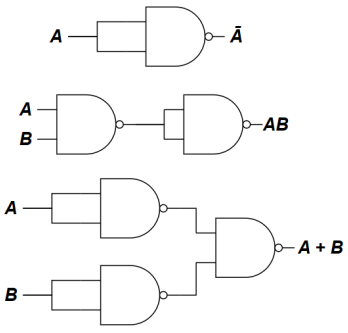




Ad esempio, questi due circuiti rappresentano la stessa identica equazione booleana: $\boldsymbol{Y = \overline{\overline{AB} \cdot \overline{CD}} = AB + CD}$ (sappiamo che è la stessa equazione grazie proprio al teorema di DeMorgan).

Questo ci permette sicuramente di semplificare alcuni circuiti, ma ci permette soprattutto di creare circuiti solo tramite **`NAND`** e **`NOR`**, che prendono il nome di **porte logiche universali**. Questo è molto importante perché a livello fisico è molto più conveniente creare circuiti solo tramite `NAND` e `NOR`, eliminando tutte le altre porte.

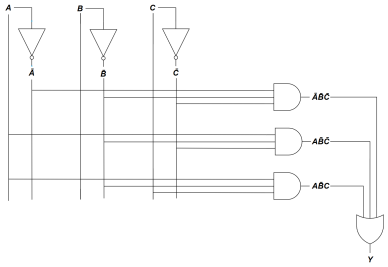
N.B. Al contrario delle porte `AND` e `OR`, le porte `NAND` e `NOR` NON sono associative.



Il `NOT` è rappresentato tramite un `NAND` dove i due valori di entrata sono entrambi `A` ($\overline{AA} = \bar{A}$). L'`AND` è rappresentato da un `NAND` che ha come valori di entrata il risultato di un altro `NAND` dove i valori di entrata sono `A` e `B` ($\overline{\overline{AB} \cdot \overline{AB}} = AB + AB = AB$). L'`OR` infine è rappresentato da un `NAND` che ha come valori di entrata i risultati di due altri `NAND` che hanno come input uno solo `A` e l'altro solo `B` ($\overline{\overline{AA}\overline{BB}} = AA + BB = A + B$). Lo `XOR` e lo `XNOR` non li studiamo ma anch'essi sono rappresentabili tramite `NOR` e `NAND`.

In particolare, grazie a questa tecnica possiamo rappresentare una SOP come una serie di `NAND`, e una POS come una serie di `NOR` (negando due volte la funzione e usando DeMorgan). Ad esempio, se abbiamo $Y = AB + BC$, possiamo riscriverla con una serie di `NAND` semplicemente scrivendo $Y = \bar{\bar{Y}} = \overline{\overline{AB + BC}} = \overline{\overline{AB} \cdot \overline{BC}}$.

Nascono quindi le **reti a due livelli**, dove il primo livello è rappresentato da tanti `AND` e il secondo da un `OR` (o viceversa). In questo modo, è possibile rappresentare qualsiasi SOP come circuiti logici nel modo più chiaro possibile. Tramite il bubble pushing è poi possibile trasformare queste funzioni in funzioni composte solo da tanti `NAND` e da un altro `NAND` finale (o solo da tanti `NOR` e da un altro `NOR` finale, se si parte con tanti `OR` e si finisce con un `AND`).



Questa a sinistra è ad esempio una funzione a due livelli che rappresenta l'equazione $Y = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$. Notiamo come in questi schemi è importante mettere gli input in alto (o a sinistra), e gli output in basso (o a destra), con le porte logiche che vanno da sinistra a destra con linee dritte possibilmente, perché più facili da leggere.

2.3.6 Circuiti a più uscite

Abbiamo per ora visto, fatta eccezione per il Full Adder, solo circuiti con un output. Esistono, però, anche i **circuiti a più uscite**. Tra questi il primo che vediamo è il **Priority Circuit**.

Definizione 2.9 – Priority Circuit

Nel **priority circuit** il numero di ingressi è pari al numero di uscite e l'output assegnato corrisponde all'**input TRUE più significativo** (ossia l'input più a sinistra). I circuiti che restituiscono esattamente un output TRUE per ogni combinazione di input sono detti circuiti **one-hot**: il priority circuit ci si avvicina molto, dato che ha al più un output TRUE (nessuno solo quando tutti gli input sono FALSE).

Preso quindi un priority circuit con 4 input A_3, A_2, A_1 e A_0, questo avrà 4 output Y_3, Y_2, Y_1 e Y_0, e come già accennato per ogni combinazione di input con almeno un 1 si accenderà l'output corrispondente all'input TRUE più a sinistra. Quindi, se ad esempio è acceso l'input A_3 (ossia vale 1), sicuramente sarà acceso l'output Y_3. Se invece è acceso A_2, se A_3 non è acceso allora si accenderà Y_2. Se è acceso A_1, se A_3 e A_2 non sono accesi allora si accenderà Y_1, e infine se A_0 è l'unico acceso allora si accenderà Y_0. Abbiamo quindi un circuito con le uscite **mutualmente esclusive**: con una determinata combinazione di input **al più un output sarà acceso**.

Definizione 2.10 – Don't Care

Quando l'output di un'equazione **non è influenzato da uno dei valori in input**, si può segnare quel valore come **X** per indicare che non è importante. Questa rappresentazione prende il nome di rappresentazione **Don't Care**. Useremo la X anche come valore di **output**: in quel caso indica una combinazione di input per cui il valore dell'output non ci interessa (o che non si verifica mai), e che possiamo quindi scegliere liberamente tra 0 e 1.

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	**1**
0	0	1	X	0	0	**1**	0
0	1	X	X	0	**1**	0	0
1	X	X	X	**1**	0	0	0

Dunque la tabella di verità del priority circuit a 4 ingressi può essere rappresentata in modo compatto tramite la Don't Care in questo modo.

2.4 Mappe di Karnaugh

Definizione 2.11 – Mappe di Karnaugh

Le **Mappe di Karnaugh** (K-Maps) sono rappresentazioni grafiche delle tabelle di verità per minimizzare le espressioni booleane graficamente.

Per capire al meglio le mappe di Karnaugh è meglio vederle con un esempio. Ipotizziamo di voler rappresentare l'equazione booleana della funzione Y che ha la seguente tabella di verità:

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

C \ AB	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Questa è la mappa di Karnaugh corrispondente alla funzione Y che vogliamo rappresentare. Abbiamo quindi diviso i literal in due gruppi: AB e C. Sulle colonne ci sono tutti i possibili valori di A e B, mentre sulle righe i possibili valori di C. Notiamo una cosa molto interessante: le colonne non sono scritte nel classico ordine 00, 01, 10, 11 che ci aspetteremmo, ma nell'ordine 00, 01, 11 e 10. Questo è fatto perché ad ogni colonna viene modificato sempre e solo un bit, mai due insieme. Troviamo quindi che tra la 1° e la 2° cambia B, tra la 2° e la 3° A, tra la 3° e la 4° di nuovo B e tra la 4° e la 1° di nuovo A.

Una volta che abbiamo creato la mappa di Karnaugh per la nostra funzione Y, dobbiamo cerchiare tutti gli 1 presenti nella mappa. Possiamo cerchiare più 1 insieme se questi sono vicini

e formano un **rettangolo** (o un quadrato) con un numero di celle uguale a una potenza di 2 ($1, 2, 4, \ldots$), e dobbiamo fare i cerchi più grandi possibili. Inoltre, se sono presenti dei valori don't care (X), è possibile cerchiarli, ma solo se questi aiutano a creare cerchi più grandi. In questo caso cerchieremo quindi semplicemente i due 1 sulla prima colonna. Notiamo che nell'implicante creato da queste due celle cerchiate, l'unico valore che cambia è C (abbiamo infatti i mintermini $\bar{A}\bar{B}\bar{C}$ e $\bar{A}\bar{B}C$). Per questo motivo, è possibile eliminare la C da questi due implicanti (infatti $\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C = \bar{A}\bar{B}$). Dunque otteniamo che $\boldsymbol{Y = \bar{A}\bar{B}}$.

Definizione 2.12 – Implicante primo

Un **implicante primo** è un raggruppamento di 1 o di X di dimensione massima in una mappa di Karnaugh. Rappresenta un termine prodotto che **non può più essere minimizzato**.

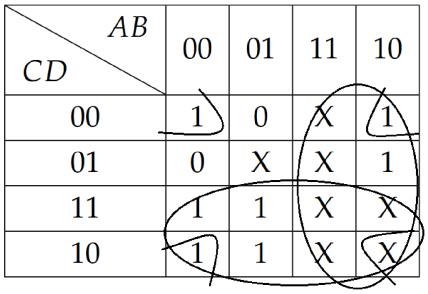
Vediamo un altro esempio un po' più complesso. Proviamo a minimizzare la funzione Y con la seguente tabella di verità tramite la sua mappa di Karnaugh:

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

CD \ AB	00	01	11	10
00	1	0	X	1
01	0	X	X	1
11	1	1	X	X
10	1	1	X	X

N.B. I colori qui ci aiutano a capire come tradurre la tabella di verità nella sua mappa di Karnaugh velocemente: in questo caso, con 4 variabili, le prime due colonne sono i primi 8 valori, ma dato che la terza colonna è invertita con la quarta, invertiamo anche qui gli 8 valori, mettendo gli ultimi 4 alla terza colonna e i penultimi quattro alla quarta colonna. Lo stesso discorso vale anche per la 3° e 4° riga di ogni colonna (in questo caso non si nota essendo quei due valori uguali in tutte le colonne).

Proviamo ora a raggruppare gli 1 e le X. Possiamo prendere i quattro 1 e le quattro X in basso per formare un cerchio da 8, poi possiamo prendere anche i due 1 e le sei X tutte a destra per formare un altro cerchio da 8, e possiamo anche prendere i tre 1 e la X sui quattro angoli (ricordiamo che l'angolo in alto a sinistra è come se toccasse l'angolo in alto a destra e in basso a sinistra, e ciò vale per tutti gli angoli). Arriviamo quindi a una mappa del genere.

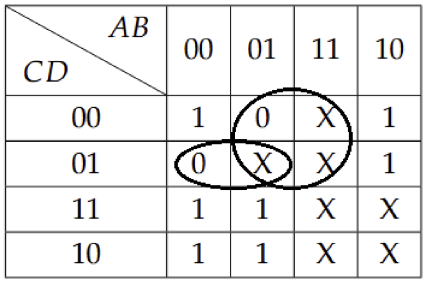


Per il cerchio tutto in basso notiamo che l'unico termine che non cambia è C, per il cerchio più a destra è A, e per il cerchio formato dai quattro angoli sono $\bar{B}$ e $\bar{D}$. Dunque troviamo che $\boldsymbol{Y = C + A + \bar{B}\bar{D}}$. Questa è la **SOP minima** della nostra funzione Y.

2.4.1 Duale delle mappe di Karnaugh

Le mappe di Karnaugh possono essere usate anche per trovare la **POS minima** di una funzione booleana, sfruttando le regole della dualità. Dobbiamo in questo caso prendere gli 0 al posto degli 1, e quando dobbiamo prendere una variabile che non cambia, se questa vale 0 prendiamo la stessa variabile, altrimenti se vale 1 ne prendiamo il complemento. Riprendiamo l'esempio di prima per capire come fare.

Dalla tabella di verità troviamo la seguente mappa di Karnaugh. Dobbiamo quindi raggruppare non gli 1 ma gli 0. Possiamo quindi prendere lo 0 e le tre X per formare un cerchio da 4, e possiamo poi prendere lo 0 a sinistra e la X affianco per formare un cerchio da due.



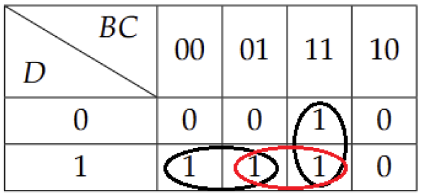
Per il cerchio da 4 notiamo che i termini che non cambiano sono $\bar{B}$ e C, mentre per il cerchio da 2 sono A, C e $\bar{D}$ (ricordiamo che per la dualità quando abbiamo $A = 1$ dobbiamo prendere $\bar{A}$ e quando abbiamo $A = 0$ dobbiamo prendere A). Dunque troviamo che $\boldsymbol{Y = (\bar{B} + C)(A + C + \bar{D})}$. Questa è la **POS minima** della nostra funzione Y (nota che questa funzione algebrica non è equivalente a quella trovata prima come SOP minima: questo è causato dai valori Don't Care, che possono creare funzioni algebriche differenti a partire dalla stessa mappa di Karnaugh).

2.4.2 Usare le mappe di Karnaugh per visualizzare i teoremi

Le mappe di Karnaugh sono molto importanti per capire visivamente i teoremi dell'algebra di Boole. Vedremo in questo caso un teorema che noi non abbiamo visto tra i teoremi da studiare, ossia il *teorema del consenso*, che dice che $BC + \bar{B}D + CD = BC + \bar{B}D$. Non è importante conoscerlo,

ma lo useremo come base per vedere come le mappe di Karnaugh ci aiutano a dimostrare queste identità.

Notiamo come, presa la mappa di Karnaugh per le variabili B, C e D, per scrivere l'espressione $BC + \bar{B}D + CD$ dobbiamo prendere due 1 al centro due volte. Quel cerchio in più serve proprio per rappresentare CD, ma noi sappiamo che è superfluo: infatti, i due 1 che prende erano già presi da altri cerchi, dunque possiamo toglierlo, trovando proprio l'espressione $BC + \bar{B}D$. Ecco dimostrato intuitivamente come $BC + \bar{B}D + CD = BC + \bar{B}D$.



Oltre ad aiutarci a capire il teorema visivamente, le mappe di Karnaugh ci aiutano anche a intuirne la dimostrazione algebrica. Infatti, abbiamo visto come CD nella mappa di Karnaugh sia rappresentato da quei due 1, che rappresentano rispettivamente il mintermine $\bar{B}CD$ e il mintermine BCD. Capiamo quindi che se scriviamo CD come $\bar{B}CD + BCD$ ci verrà più veloce la dimostrazione. Infatti,

$$\begin{aligned} BC + \bar{B}D + CD &= BC + \bar{B}D + \bar{B}CD + BCD \\ &= BC(D + 1) + \bar{B}D(1 + C) \\ &= BC + \bar{B}D \quad \blacksquare \end{aligned}$$

2.4.3 Algoritmo Quine-McCluskey

Le mappe di Karnaugh abbiamo visto come sono molto utili per trovare SOP/POS minima nelle funzioni a poche variabili (solitamente 3 o 4). Per le funzioni con tante variabili, però, iniziano ad essere molto lente. Per questo motivo si introduce un nuovo algoritmo: l'**algoritmo Quine-McCluskey**.

Definizione 2.13 – Algoritmo Quine-McCluskey

L'**algoritmo Quine-McCluskey** (QMC, anche detto metodo degli implicanti primi), è un metodo utilizzato per **minimizzare** le funzioni booleane a n variabili con un solo output che rappresentano reti a due livelli.

2.5 Circuiti a più livelli

Abbiamo parlato brevemente in precedenza delle **reti a due livelli**, ossia circuiti formati da tanti `AND` e da un `OR` finale o viceversa. Questi circuiti rappresentano le funzioni booleane in forma rispettivamente **SOP** e in forma **POS**, e sono molto veloci, ma spesso richiedono una quantità esagerata di hardware. Per questo motivo nascono i **circuiti a più livelli**.

Definizione 2.14 – Circuiti a più livelli

I **circuiti a più livelli** (o reti a più livelli) sono dei circuiti combinatori rappresentati non più da soli due livelli di `AND` e `OR` o di `OR` e di `AND`, ma da più livelli con diverse porte logiche.

Prima di vedere alcuni circuiti a più livelli, capiamo perché sono così fondamentali con un esempio.
Ipotizziamo di voler rappresentare tramite un circuito uno `XOR` a 3 ingressi. Avremo quindi la seguente tabella di verità con la corrispondente mappa di Karnaugh:

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

C \ AB	00	01	11	10
0	0	1	0	1
1	1	0	1	0

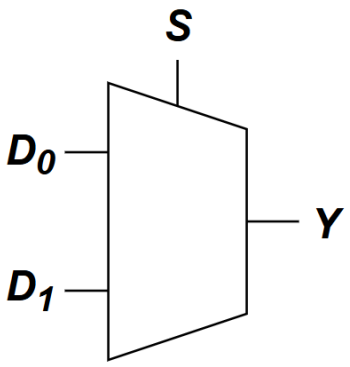
Se volessimo rappresentarla tramite SOP o POS minima avremmo 4 min/maxtermini in entrambi i casi. Questo perché nella mappa di Karnaugh i valori sono alternati, dunque non c'è nemmeno un 1 vicino a un 1 o uno 0 vicino a uno 0 (ciò ha perfettamente senso, dato che nella mappa di Karnaugh i valori vicini cambiano di un solo bit, e sappiamo che nello XOR un cambio di bit indica un cambio nel risultato, che passa da 1 a 0 o da 0 a 1). Dunque se volessimo rappresentare tale circuito con una rete a due livelli avremmo bisogno di 4 `AND` a tre ingressi e di un `OR` a 4 ingressi (o di 4 `OR` e di 1 `AND`, è indifferente). Ma dato che un `AND` a 3 ingressi equivale a due `AND` a due ingressi, e dato che un `OR` a 4 ingressi equivale a tre `OR` a due ingressi, troviamo che il numero totale di porte logiche ammonta a **11**. Se invece volessimo rappresentarlo tramite un circuito a più livelli, potremmo trasformare questo `XOR` a 3 in due `XOR` a 2, e potremmo poi trasformare ogni `XOR` a 2 in un circuito a due livelli. Infatti, uno `XOR` a 2 può essere rappresentato da 2 `AND` e un `OR` (o viceversa), dunque avremmo un totale di **6** porte distribuite su 4 livelli, un netto miglioramento rispetto alle 11 viste prima. Si guadagna quindi a livello hardware ma si perde a livello di velocità, dato che appunto dobbiamo passare su più livelli. Questa differenza, che non sembra nemmeno così importante con uno `XOR` a 3 ingressi, diventa evidente in uno `XOR` a 8 ingressi, dove servono $2^8/2 = 2^7 = 128$ `AND` a 8 ingressi e un `OR` a 128 ingressi per poterlo tradurre in una rete a due livelli (che significano $128 \cdot 7 = 896$ `AND` e 127 `OR` a 2 ingressi), mentre in una rete a più livelli bastano 7 `XOR` a due ingressi, che si traducono in 14 porte `AND` e 7 `OR`. Dunque passiamo da $896 + 127 = 1023$ porte a sole 21.

2.5.1 Multiplexer

Definizione 2.15 – Multiplexer

Un **multiplexer** (MUX) è un circuito combinatorio che **seleziona uno tra i segnali** in ingresso e lo collega in output. Per farlo utilizza dei **selettori**, il cui numero dipende dal numero di input: in un circuito con n input servono $\mathbf{\log_2 n}$ bit-selettori.

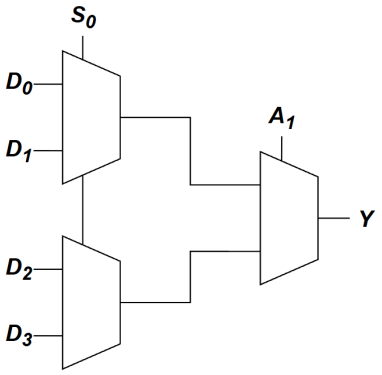
Un multiplexer si raffigura usando un trapezio in verticale, con gli input a sinistra, i selettori in alto e gli output a destra. In questo caso abbiamo un MUX a due input (e con quindi un selettore dato che $\log_2 2 = 1$). La tabella di verità è molto semplice:



S	Y
0	D_0
1	D_1

Notiamo quindi come quando $S = 0$ allora si attiva D_0, mentre quando $S = 1$ si attiva D_1. Dunque $\boldsymbol{Y = \bar{S}D_0 + SD_1}$.

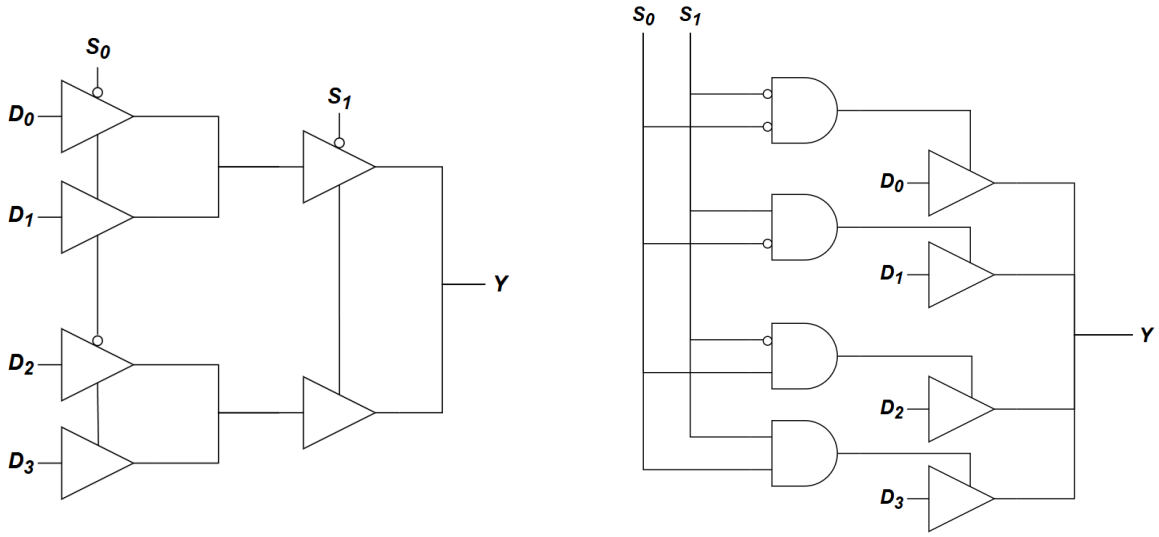
Un MUX a 4 ingressi, oltre alla sua forma canonica con 4 ingressi e 2 selettori nello stesso MUX, può essere riprodotto anche usando 3 MUX a 2 ingressi. Dato che dobbiamo usare solo 2 selettori ($\log_2 4 = 2$), utilizziamo il selettore S_0 per i primi due MUX e S_1 solo per il terzo. Infatti, dato che stiamo cercando di mandare in output uno solo dei 4 valori in input, ci bastano questi due selettori per poterli scegliere tutti senza problemi. Se ad esempio vogliamo $Y = D_2$, basta porre $S_0 = 0$ e $S_1 = 1$.



Troviamo quindi che $\boldsymbol{Y = \bar{S_0}\bar{S_1}D_0 + S_0\bar{S_1}D_1 + \bar{S_0}S_1D_2 + S_0S_1D_3}$. Se volessimo quindi trasformare questo MUX in un circuito a due livelli, avremmo bisogno di 4 `AND` a 3 ingressi e di un `OR` a 4 ingressi, ossia di 8 `AND` a 2 ingressi e di 3 `OR` a 2 ingressi, per un totale di 11 porte logiche. Se invece anche qui, piuttosto che fare un circuito a due livelli ne creiamo uno a 4, trasformando ogni multiplexer in un circuito a due livelli, avremmo bisogno di sole 6 porte `AND` e di 3 porte `OR`, per un totale di 9 porte logiche.

Tristate Buffer al posto del multiplexer

Grazie allo stato di alta impedenza, è possibile riprodurre lo stesso identico comportamento dei multiplexer utilizzando semplicemente dei tristate buffer. Ad esempio, un MUX a 4 input può essere riprodotto in questi due modi usando tristate buffer:



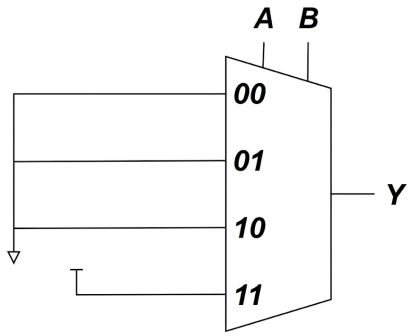
Nella prima immagine usiamo 6 tristate buffer, mentre nella seconda ne usiamo solo 4, ma usiamo 4 porte `AND` per gestire gli ingressi di questi tristate per far partire il segnale solo da un singolo tristate alla volta, così da non creare mai contention.

Usare i multiplexer per rappresentare funzioni booleane

Possiamo usare i multiplexer anche per rappresentare delle funzioni booleane. Ad esempio, presa la funzione Y con la seguente tabella di verità:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

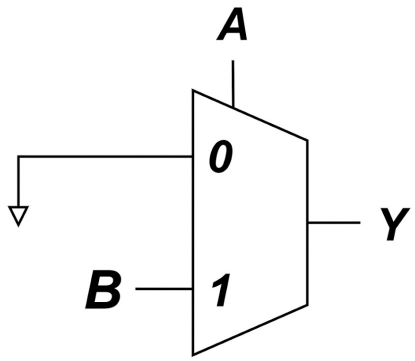
se poniamo i diversi input del nostro multiplexer tutti a 0 tranne che per l'ultimo che vale 1, possiamo rappresentare questa funzione con il seguente MUX:



dove la freccia verso il basso indica un input fisso di 0 (prende il nome di *massa*), mentre la barretta indica un input fisso di 1. Abbiamo quindi rappresentato la funzione $Y = AB$, ossia la porta logica `AND`, tramite un MUX a 4 ingressi. Notiamo però che possiamo migliorare questa rappresentazione utilizzando un MUX a soli due ingressi. Infatti, modificando la tabella di verità in questo modo

A	Y
0	0
1	B

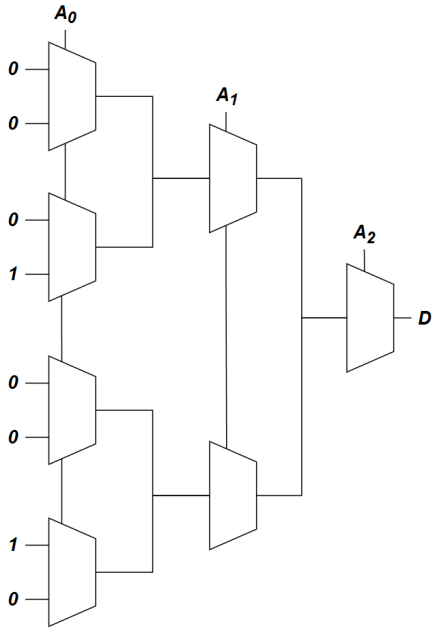
che è totalmente equivalente a quella vista sopra, possiamo rappresentare la funzione Y con questo MUX:



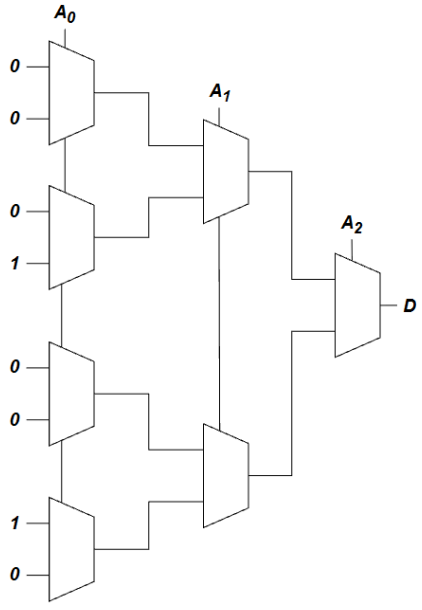
Vediamo anche un esempio più complesso. Prendiamo la seguente funzione D a 3 ingressi:

A_2	A_1	A_0	D
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Possiamo rappresentarla usando un MUX a 8 ingressi, ma è più comodo rappresentarla con 7 MUX a 2 ingressi, in questo modo:



Notiamo però anche qui una cosa: i primi 4 MUX sono praticamente superflui. Infatti, il primo e il terzo valgono sempre 0, indipendentemente da A_0, mentre il secondo vale 0 se $A_0 = 0$, e 1 se $A_0 = 1$, ossia vale A_0, e il quarto allo stesso modo vale invece $\bar{A}_0$. Dunque possiamo rappresentare il circuito di questa funzione usando solamente tre MUX:



Proposizione 2.2 – Teorema di Shannon

Data una funzione booleana f a n variabili, vale sempre l'uguaglianza

$$f(x_1, \ldots, x_n) = x_1 f(1, \ldots, x_n) + \bar{x_1} f(0, \ldots, x_n)$$

Questo teorema è la dimostrazione formale di ciò che abbiamo visto in precedenza, ossia del perché riusciamo a trasformare un MUX a 4 ingressi in 3 MUX a 2 ingressi, ed espande questo concetto a MUX con un qualsiasi numero di ingressi, formalizzandolo tramite le funzioni booleane. Infatti, presa una funzione di n variabili, possiamo realizzarla usando un MUX a 2 ingressi che prende in input 2 funzioni di $n-1$ variabili. Un MUX a 4 ingressi, ad esempio, realizza una funzione di 2 variabili (i due selettori): questa diventa un MUX con 2 funzioni di 1 variabile, che a loro volta diventano due MUX i cui ingressi sono direttamente i valori D_i, con un totale quindi di 3 MUX a 2 ingressi.

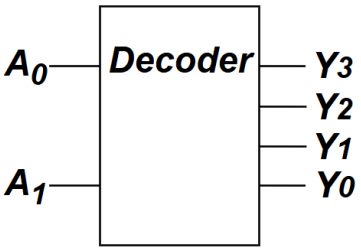
2.5.2 Decoder

Definizione 2.16 – Decoder

Un **decoder** è un circuito combinatorio con n input e 2^n output e che restituisce un solo output TRUE per ogni combinazione di input, ossia è un circuito **one-hot**.

N.B. Per far uscire solo un output TRUE tra tutti gli output il decoder utilizza una tecnica chiamata **mascheramento**: ogni uscita è un `AND` degli ingressi (diretti o negati), cioè un mintermine, che vale 1 solo per la propria combinazione di input così che, per ogni combinazione, tutte le altre uscite valgano 0.

Un decoder si raffigura semplicemente con un rettangolo verticale con la scritta "Decoder" in alto, con gli n ingressi a sinistra (in questo caso 2), e i 2^n output a destra (in questo caso 4).



Possiamo vedere un decoder come una delle due componenti di un MUX.Infatti, un MUX con n input e $\log_2 n$ selettori non è altro che un decoder a $\log_2 n$ input con n output, collegati a una serie di `AND` e a un `OR` finale. Vediamo con i circuiti come si presenta questa idea:

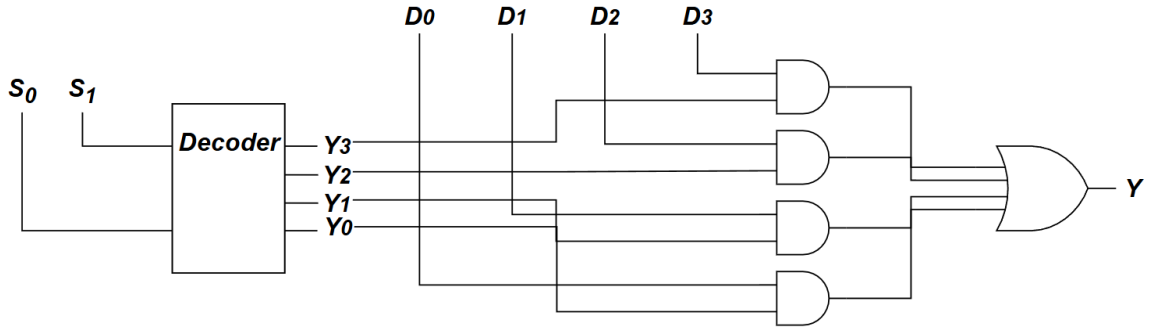
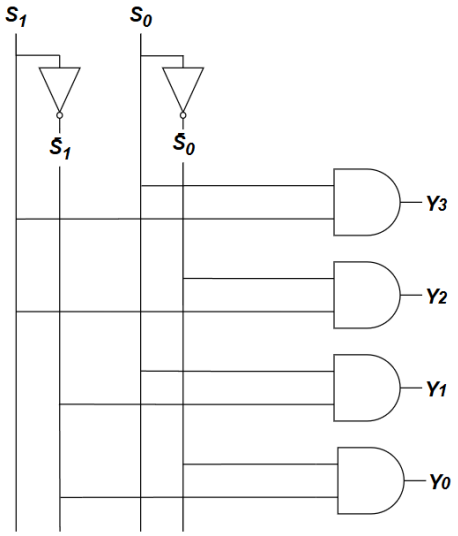


Figura 2.1: MUX creato tramite decoder e `AND/OR`

Capire perché questo altro non è che un MUX a 4 ingressi è molto semplice. Sappiamo che la funzione booleana che descrive un MUX a 4 ingressi è $Y = \bar{S_0}\bar{S_1}D_0 + S_0\bar{S_1}D_1 + \bar{S_0}S_1D_2 + S_0S_1D_3$. In questo decoder abbiamo che $Y_0 = \bar{S_0}\bar{S_1}$, $Y_1 = S_0\bar{S_1}$, $Y_2 = \bar{S_0}S_1$, $Y_3 = S_0S_1$, e dato che la funzione di questo circuito che abbiamo creato è $Y = Y_0D_0 + Y_1D_1 + Y_2D_2 + Y_3D_3$, sostituendo tutti gli Y_i con i loro rispettivi valori troviamo proprio la funzione booleana di un classico MUX a 4 ingressi.

Vediamo ora però com'è fatto al suo interno un decoder. Abbiamo già visto come, in un decoder a 2 ingressi, ogni output corrisponda a un mintermine, nell'ordine $\bar{S_0}\bar{S_1}$, $S_0\bar{S_1}$, $\bar{S_0}S_1$, S_0S_1. Dunque possiamo usare quattro `AND` per formare un decoder, in questo modo:

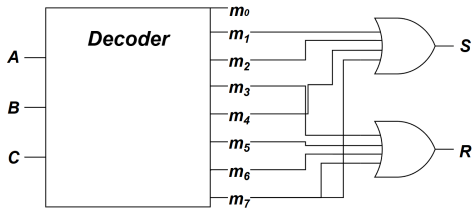


Con i decoder possiamo scrivere qualsiasi funzione booleana a n variabili nella forma SOP. Infatti, basta collegare in un `OR` tutti i mintermini che formano tale funzione. Vediamo ad esempio come rappresentare un full adder a 3 ingressi tramite un decoder. Ricordiamo che un full adder è un circuito combinatorio utilizzato per eseguire somme di bit.

Abbiamo quindi due output da rappresentare: S per la somma e R per il riporto, con la seguente tabella di verità (avremo in questa tabella 0 per S se la somma dei tre bit n è $n\%2 = 0$ e 1 altrimenti, e 0 per R se il riporto della somma dei tre bit è 0 e 1 altrimenti):

A	B	C	S	R	mintermini
0	0	0	0	0	$\bar{A}\bar{B}\bar{C} = m_0$
0	0	1	1	0	$\bar{A}\bar{B}C = m_1$
0	1	0	1	0	$\bar{A}B\bar{C} = m_2$
0	1	1	0	1	$\bar{A}BC = m_3$
1	0	0	1	0	$A\bar{B}\bar{C} = m_4$
1	0	1	0	1	$A\bar{B}C = m_5$
1	1	0	0	1	$AB\bar{C} = m_6$
1	1	1	1	1	$ABC = m_7$

Dunque troviamo che $S = \sum m(1,2,4,7)$ e $R = \sum m(3,5,6,7)$, che si traduce in questo circuito combinatorio:



2.6 Dispositivi logici programmabili

Passiamo ora allo studio di dispositivi più complessi, i cui collegamenti interni non sono sempre stabiliti a priori ma possono in alcuni casi essere **configurabili dall'utente**. Questi dispositivi prendono il nome di **dispositivi logici programmabili**. Vedremo due tipologie:

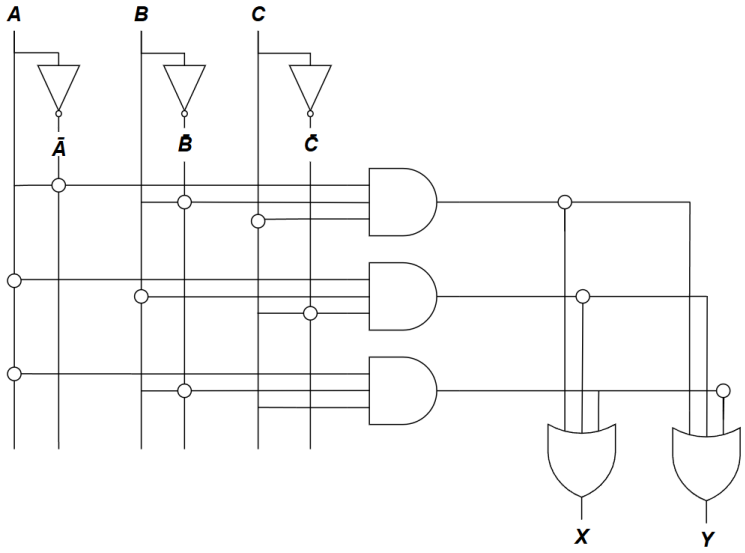
- **PLA**
- **FPGA** (l'evoluzione dei PLA)

2.6.1 PLA

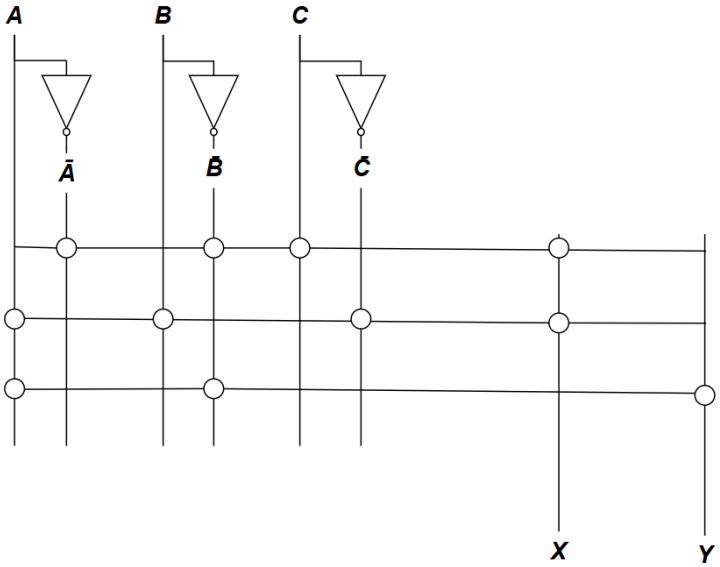
Definizione 2.17 – PLA

I **PLA** (Programmable Logic Arrays) sono dispositivi logici programmabili che utilizzano un array di `AND` seguito da un array di `OR` per formare dei circuiti combinatori: questa struttura interna è fissa, mentre si programmano i collegamenti all'interno dei due array.

Un PLA ha M input, P output e N implicanti rappresentabili (dunque non rappresentiamo tutti i 2^M **mintermini** possibili, ma solo una certa porzione). Vediamo ad esempio un PLA che ha 3 input A, B e C e ha come output $X = \bar{A}\bar{B}C + AB\bar{C}$ e $Y = A\bar{B}$:



Esiste un altro modo più semplice per rappresentare un PLA che prende il nome di **dot notation**:



Osservazione 2.1

Nei PLA non conviene sempre **minimizzare** le SOP.

Capiamo con un esempio perché. Prendiamo due funzioni a 3 input X e Y, con $X = \sum m(3,5,6)$ e $Y = \sum m(3,5,6,7)$ e studiamone le rispettive mappe di Karnaugh:

X

C \ AB	00	01	11	10
0	0	0	1	0
1	0	1	0	1

Y

C \ AB	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Notiamo che possiamo minimizzare Y da $\bar{A}BC + AB\bar{C} + ABC + A\bar{B}C$ a $BC + AC + AB$, lasciando invece X invariato come $\bar{A}BC + AB\bar{C} + A\bar{B}C$. Minimizzando Y, però, stiamo creando 3 nuovi implicanti da rappresentare, quando prima invece ne avevamo già 3 in comune con X. In questo modo, al posto di rappresentare X e Y con un totale di 4 implicanti li staremmo rappresentando con un totale di 6 implicanti, peggiorando di fatto la situazione.

2.6.2 FPGA

Definizione 2.18 – FPGA

I **FPGA** (Field Programmable Gate Array) sono dispositivi logici programmabili che utilizzano array di elementi logici (**LE**) per formare dei circuiti combinatori e sequenziali con connessioni interne programmabili. Questi dispositivi permettono di collegare gli elementi logici con blocchi di input e output, creando così dei nuovi circuiti configurabili dall'utente.

I LE (ossia gli elementi logici) sono composti da:

- un **LUT**, che si occupa della logica combinatoria
- un **Flip-Flop**, che si occupa della logica sequenziale (che vedremo più avanti quando parleremo nel concreto di circuiti sequenziali)
- un **MUX**, che collega i LUT ai flip-flop

Un LUT altro non è che un circuito combinatorio a 4 ingressi che rappresenta una funzione. Ad esempio, se vogliamo rappresentare la funzione $X = \bar{A}\bar{B}C + AB\bar{C}$, il LUT che useremo avrà una tabella di verità di questo tipo:

Data 1 (A)	Data 2 (B)	Data 3 (C)	Data 4	LUT(X)
0	0	0	X	0
0	0	1	X	1
0	1	0	X	0
0	1	1	X	0
1	0	0	X	0
1	0	1	X	0
1	1	0	X	1
1	1	1	X	0

dove Data 4 indica il quarto ingresso non utilizzato: per la funzione è un don't care, ma dato che nel circuito deve comunque avere un valore, viene fissato a 0.

Capitolo 3

Circuiti sequenziali

3.1 Introduzione

Abbiamo parlato fino ad ora di circuiti combinatori, che non avevano memoria. È arrivato ora il momento di iniziare a parlare del secondo tipo di circuiti logici, ossia i **circuiti sequenziali**.

Sappiamo già che i circuiti sequenziali sono dei circuiti logici che si differenziano dai circuiti combinatori per due motivi principali:

- hanno **memoria**
- il loro output dipende anche dagli **input passati**, e non solo da quelli correnti

Prima di poter parlare concretamente di circuiti sequenziali è importante però dare delle definizioni importanti:

- lo **stato** di un circuito sequenziale sono tutte le informazioni riguardo tale circuito necessarie per spiegarne il comportamento presente e futuro
- i **latch** e i **flip-flop** sono degli elementi di stato che salvano un bit di stato
- i circuiti **sequenziali sincroni** sono circuiti combinatori seguiti da un banco di flip-flop

3.2 Elementi di stato

Definizione 3.1 – Elemento di stato

Un **elemento di stato** è un elemento di un circuito che salva uno stato al suo interno.

Vedremo quattro elementi di stato:

- **Circuiti bistabili**
- **SR Latch**
- **D Latch**
- **D Flip-Flop**

3.2.1 Circuiti bistabili

Definizione 3.2 – Circuito bistabile

Un **circuito bistabile** è un circuito con **due stati stabili** ($Q = 0$ oppure $Q = 1$) che, nella forma base, ha due output e nessun input.

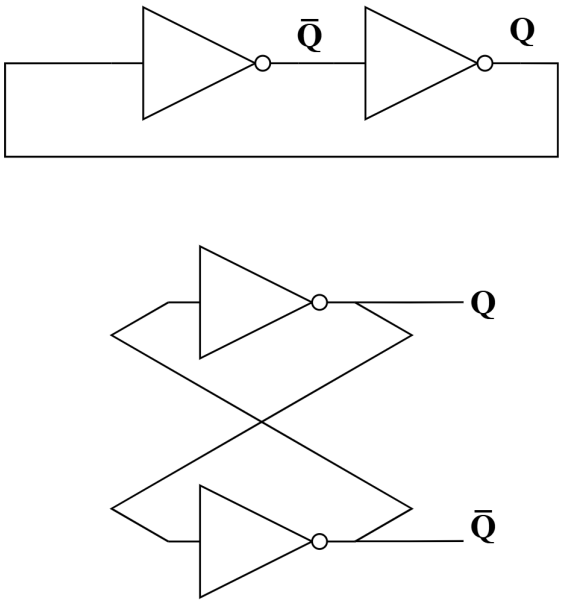
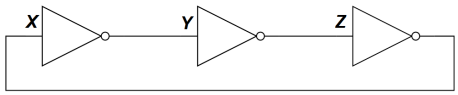


Figura 3.1: Due rappresentazioni interne di un circuito bistabile

Questo circuito è stabile perché il valore di Q non può cambiare. Infatti, se in qualche modo riuscissimo a modificare il valore di Q e metterlo a 0 ad esempio, questo rimarrebbe sempre a 0 e non cambierebbe mai, e stessa cosa se lo mettessimo a 1. Un circuito che invece non è stabile è ad esempio il circuito **astabile**:



Qui il valore di ciascun nodo (ad esempio X) cambia ad ogni **ciclo**: se inizia con 0, alla fine del primo ciclo avrà come valore 1, e solo alla fine del secondo ciclo avrà di nuovo come valore 0.

Tornando al circuito bistabile, c'è però un grosso problema: non è possibile modificare il valore salvato all'interno. Per questo motivo introduciamo dei nuovi circuiti chiamati **SR Latch**.

3.2.2 SR Latch

Definizione 3.3 – SR Latch

Un **SR Latch** (Set/Reset Latch) è un circuito bistabile che permette di modificare il valore di Q attraverso due input, S (set) e R (reset).

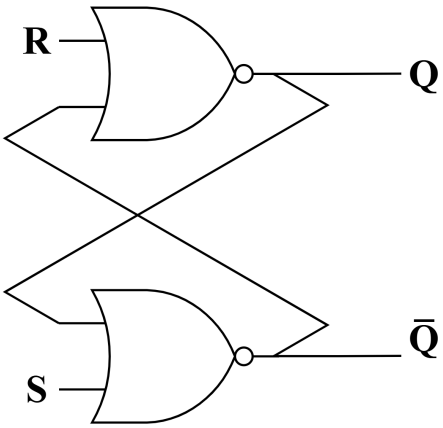


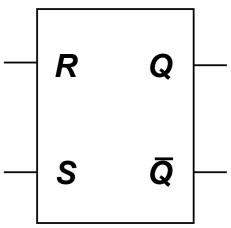
Figura 3.2: Rappresentazione interna di un SR Latch base

Avendo 2 variabili in input, abbiamo 4 casi possibili:

- Se $S = 0$ e $R = 0$ allora l'SR Latch si comporta come un banale circuito bistabile: se Q aveva già un valore, questo non cambia, ossia $Q_{prev} = Q$. Diciamo in questo caso che il latch è in **hold**.
- Se $S = 1$ e $R = 0$ allora $Q = 1$ e $\bar{Q} = 0$
- Se $S = 0$ e $R = 1$ allora $Q = 0$ e $\bar{Q} = 1$
- Se $S = 1$ e $R = 1$ allora $Q = \bar{Q} = 0$. Essendo questo impossibile, quando $S = 1$ e $R = 1$ diciamo che il circuito si trova in uno stato invalido.

Dunque quando $S = 1$ e $R = 0$ stiamo settando Q a 1; quando invece $S = 0$ e $R = 1$ lo stiamo resettando a 0; quando poi $S = 0$ e $R = 0$ lo stiamo tenendo in memoria. L'unico caso da evitare è quando $S = 1$ e $R = 1$, dato che il circuito in quel caso non sa quanto vale Q e quindi finisce in uno stato invalido. Riassumendo, $Q = S\bar{R} + \bar{R}Q_{prev}$.

Rappresentiamo un SR Latch tramite un rettangolo con a sinistra i due input S e R e a destra l'output Q e il suo negato $\bar{Q}$.



3.2.3 D Latch

Per superare il problema dello stato indefinito $S = 1$ e $R = 1$ nasce un nuovo elemento di stato, che prende il nome di **D Latch**.

Definizione 3.4 – D Latch

Un **D Latch** è un elemento di stato con due input, $\boldsymbol{D}$ e $\boldsymbol{CLK}$, e due output Q e $\bar{Q}$.

Il D Latch è molto semplice: quando $CLK = 0$ si ha sempre $Q = Q_{\text{prev}}$ (si dice che il circuito è *opaco*), mentre quando $CLK = 1$ si ha $Q = D$ (si dice che il circuito è *trasparente*).

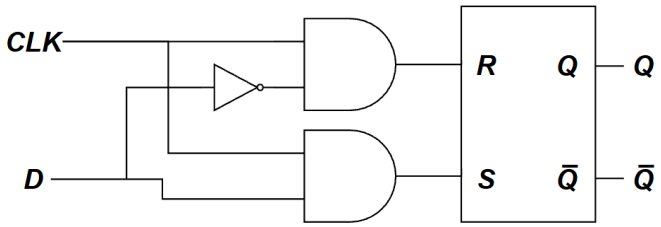


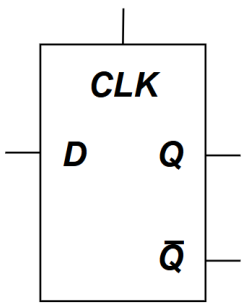
Figura 3.3: Rappresentazione interna di un D Latch base

Un D Latch base si rappresenta quindi tramite un SR Latch. Studiamone ora la tabella di verità:

CLK	D	S	R	Q
0	X	0	0	Q_{prev}
1	0	0	1	0
1	1	1	0	1

Notiamo una cosa fondamentale: in nessun caso si verifica lo stato indefinito $S = 1$ e $R = 1$, dunque questo latch non ha nessun caso particolare da evitare, cosa che invece era presente nei SR Latch. Dunque $\boldsymbol{Q = \overline{CLK} Q_{\text{prev}} + CLKD}$.

Rappresentiamo un D Latch tramite un rettangolo con a sinistra l'input D, in alto l'input CLK e a destra l'output Q e il suo negato $\bar{Q}$.



Dei D Latch studiamo anche la **forma d'onda**: in poche parole, la forma d'onda di un D Latch è una rappresentazione grafica del latch, tramite delle linee che rappresentano i valori dei vari input/output. Una linea in alto a una variabile indica che il valore di quella variabile in quell'istante di tempo è 1, in basso indica che è 0, e se è sia in alto che in basso indica che il valore non si conosce.

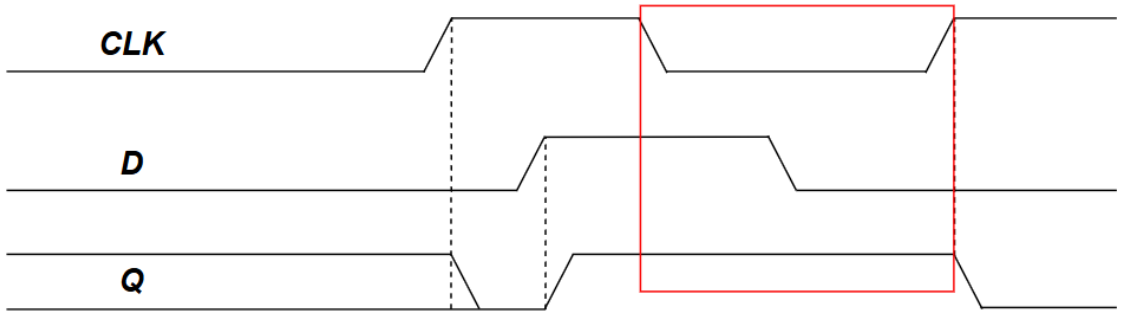


Figura 3.4: Forma d'onda di un D Latch

Inizialmente, quando $CLK = 0$, il valore Q è un valore sconosciuto. Quando poi CLK diventa 1, con un po' di ritardo (ricordiamo che questi circuiti nella realtà non possono essere istantanei, ma hanno tutti un minimo di delay) Q assume un valore definito, pari a D, ossia in questo caso 0. Quando D diventa pari a 1, anche Q cambia, ma non appena CLK torna a essere 0, Q entra nella fase di *hold*, rappresentata in figura dal quadrato rosso, nella quale non modifica più il proprio valore, anche se D dovesse cambiare. Solo quando CLK tornerà a valere 1 allora Q cambierà nuovamente il proprio valore, che diventerà pari a D ossia 0.

3.2.4 D Flip-Flop

Definizione 3.5 – D Flip-Flop

Un **D Flip-Flop** è un D Latch che modifica il valore di Q solo quando CLK passa da 0 a 1 o viceversa. I Flip-Flop che modificano Q in corrispondenza di una salita (ossia quando CLK passa da 0 a 1) sono anche detti **positive edge triggered**, mentre quelli che modificano Q in corrispondenza di una discesa sono anche detti **negative edge triggered**. Quando parleremo di Flip-Flop, se non esplicitamente dichiarato faremo sempre riferimento a un positive edge triggered.

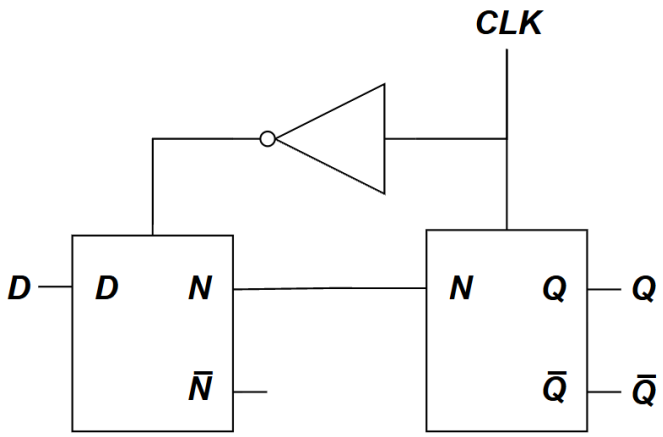


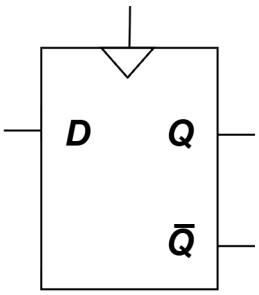
Figura 3.5: Rappresentazione interna di un D Flip-Flop base

N.B. N è il nome che diamo all'uscita del primo D Latch (il master), per non confonderla con l'uscita Q del Flip-Flop.
Il primo D Latch viene detto *master* (o L1), mentre il secondo *slave* (o L2). Per capire perché funziona, guardiamone al volo la tabella di verità:

CLK	D	N	Q
0	X	D	Q_{prev}
1	X	$N_{\text{prev}} = D_{\text{prev}}$	$N = D_{\text{prev}}$

Dunque troviamo che se $CLK = 0$ allora Q rimane uguale, in stato di hold, mentre se $CLK = 1$ allora Q diventa uguale a D_{prev}, che altro non è che il valore di D nel momento in cui CLK passa da 0 a 1. Dunque $\boldsymbol{Q = \overline{CLK}Q_{\text{prev}} + CLKD_{\text{prev}}}$.
N.B. Se avessimo voluto rappresentare un Flip-Flop negative edge triggered avremmo usato CLK nel primo D Latch e $\overline{CLK}$ nel secondo D Latch.

Rappresentiamo un D Flip-Flop come un D Latch ma con un triangolo in alto al posto della scritta CLK.



Anche dei D Flip-Flop studiamo le forme d'onda. Vediamo un esempio come abbiamo fatto con i D Latch:

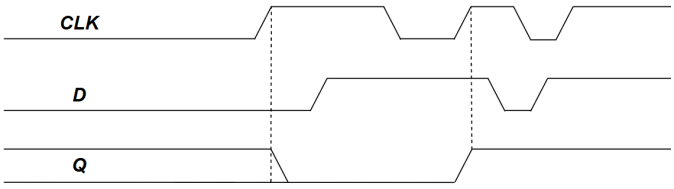


Figura 3.6: Forma d'onda di un D Flip-Flop

In questo caso, Q modifica il suo valore quando per la prima volta CLK passa da 0 a 1, diventando 0 come D, poi quando nuovamente CLK ripassa da 0 a 1 diventa 1 come D, e quando per l'ultima volta CLK sale, Q rimane uguale dato che vale già quanto D in quel momento, ossia 1.

Definizione 3.6 – Registro

Un **registro** è un gruppo di D Flip-Flop collegati in parallelo e sincronizzati da uno stesso segnale CLK.

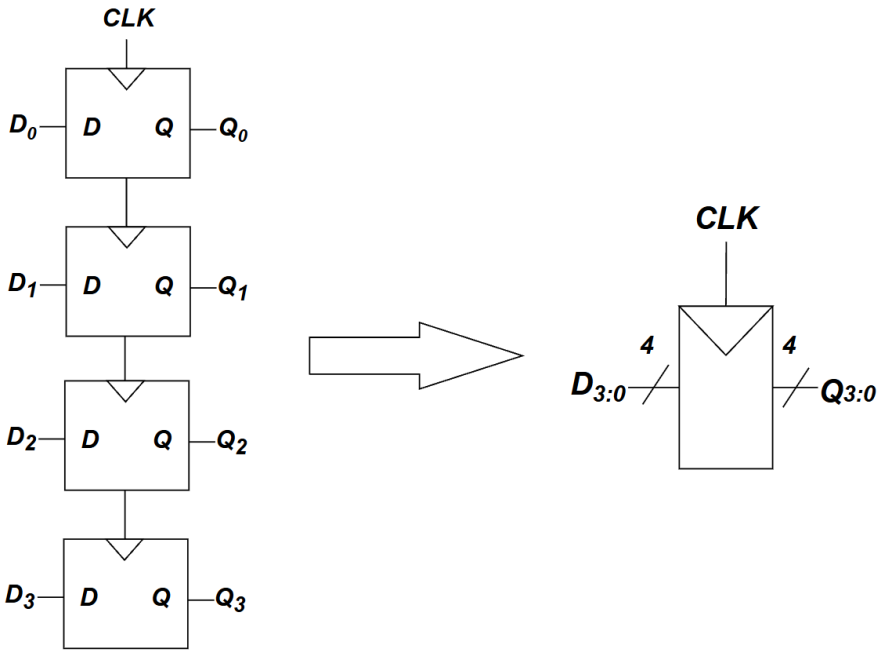


Figura 3.7: Un registro, a sinistra rappresentato con 4 Flip-Flop in parallelo, a destra come un unico blocco

Esistono infine anche i D Flip-Flop **con enable** (e reset), che non sono altro che dei Flip-Flop con due ingressi in input in più: ***EN*** e ***RESET***. L'input EN serve a decidere semplicemente se il Flip-Flop è attivo: se $EN = 0$, il Flip-Flop non funziona, e in ogni caso Q rimane uguale

a Q_{prev}, a prescindere da D e da CLK. Se invece $EN = 1$, il Flip-Flop funziona normalmente. L'input $RESET$, invece, quando vale 1 vuole forzare il Flip-Flop a uno stato noto, ponendo $Q = 0$. Esistono due modi per fare ciò, e per questo distinguiamo reset **sincrono** da reset **asincrono**:

- Il reset **sincrono** non porta subito $Q = 0$, ma lo fa solo in corrispondenza della **prossima salita** del CLK
- Il reset **asincrono** invece azzera **subito** Q, portandolo a 0 ignorando completamente il CLK.

Per quanto riguarda l'implementazione, l'input EN è molto semplice: basta mettere in `AND` questo input e il CLK. In questo modo, se $E = 0$ avremo sempre CLK `AND` $E = 0$, mentre se $E = 1$ avremo CLK `AND` $E = CLK$, trovandoci quindi nella situazione di un Flip-Flop normale. Oltre all'`AND`, esiste un altro modo per implementare l'enable, più consigliato perché non va a toccare il CLK: un MUX. Banalmente, basta usare EN come bit di selezione, e avere in input Q_{prev} quando $EN = 0$ e D quando $EN = 1$. Troviamo quindi il seguente circuito:

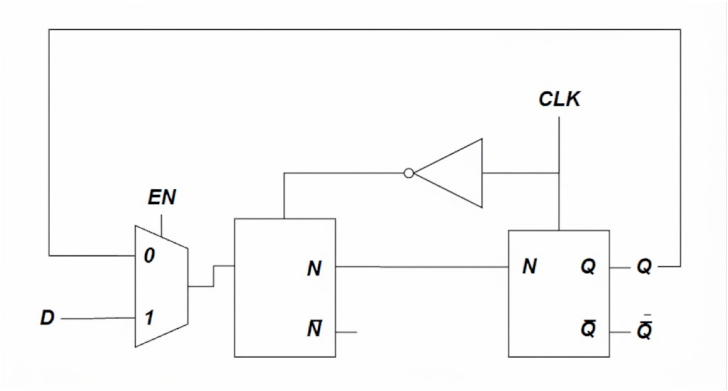


Figura 3.8: Flip-Flop con enable

Per quanto riguarda l'input $RESET$ abbiamo sempre due casi. Per un reset sincrono, bisogna prendere D e metterlo in `AND` con $\overline{RESET}$. In questo modo, se $RESET = 0$ avremo D`AND`$1 = D$, che è corretto dato che $RESET = 0$ non deve modificare in alcun modo il circuito, mentre se abbiamo $RESET = 1$ avremo D`AND`$0 = 0$, che è corretto dato che con $RESET = 1$ vogliamo che, alla prossima salita di CLK, Q diventi 0. Troviamo quindi il seguente circuito:

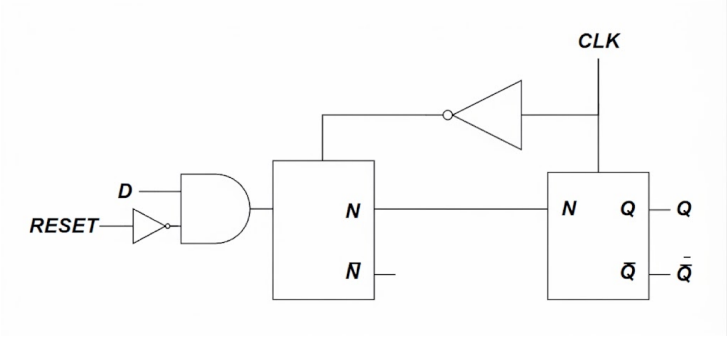
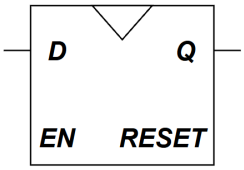


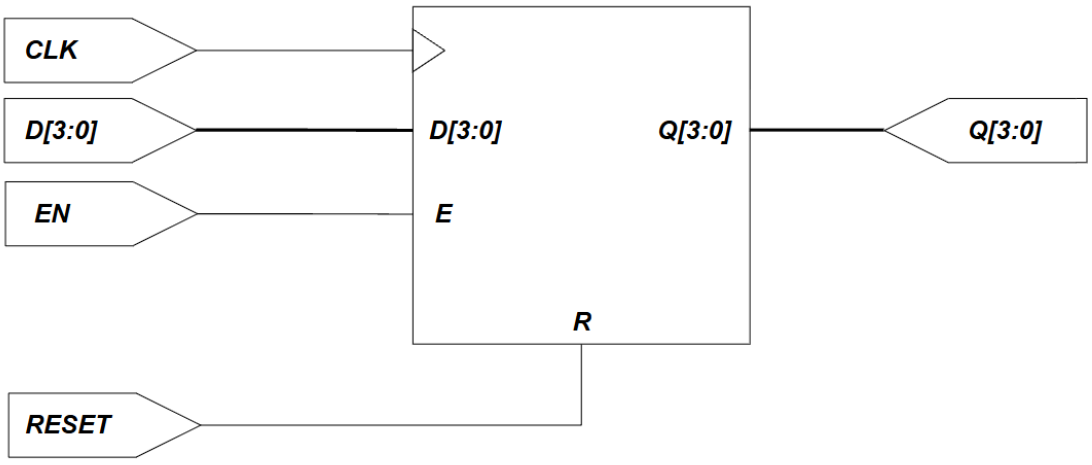
Figura 3.9: Flip-Flop con reset sincrono

Il caso del reset asincrono, invece, non lo vediamo.

Rappresentiamo un Flip-Flop con enable e reset come un Flip-Flop con la scritta EN e RESET (o anche solo R) in basso e senza evidenziare la variabile $\bar{Q}$ (per rappresentare un Flip-Flop con solo enable o solo reset basta scrivere solo uno dei due nomi).



I Flip-Flop con enable sono fondamentali e molto utilizzati per i registri. Infatti, in un sistema più complesso con decine di registri che lavorano sullo stesso clock globale, avere un segnale di enable e uno di reset è fondamentale per poter selezionare quali registri far lavorare e quali mantenere in hold. Ad esempio, il circuito



rappresenta un registro con enable e reset (**N.B.** le linee più spesse rappresentano Flip-Flop in parallelo).

Oltre ai D Flip-Flop con reset esistono anche i Flip-Flop con **set**. Questo lavora all'opposto del reset, dunque quando $SET = 1$ non abbiamo $Q = 0$ ma $Q = 1$. In un Flip-Flop si può usare un reset o un set: entrambi servono a portare Q a uno stato noto, ma il reset lo porta a 0 e il set a 1, quindi si sceglie quello che porta allo stato iniziale desiderato.

3.2.5 Altri Flip-Flop

Oltre ai D Flip-Flop esistono altri due tipi di Flip-Flop:

- **T Flip-Flop** (Toggle Flip-Flop)
- **JK Flip-Flop**
- **SR Flip-Flop**

Definizione 3.7 – T Flip-Flop

Un **T Flip-Flop** è un D Flip-Flop che, ad ogni salita del CLK, se $T = 1$ passa da Q a $\bar{Q}$.

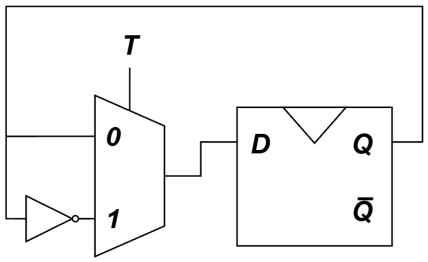
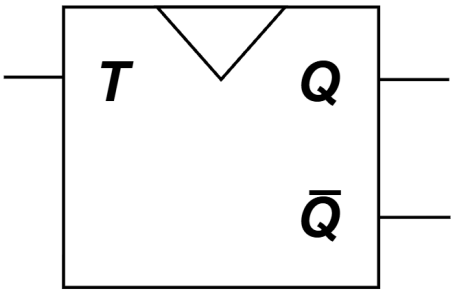


Figura 3.10: Rappresentazione interna di un T Flip-Flop base utilizzando un D Flip-Flop

Rappresentiamo un T Flip-Flop come un D Flip-Flop ma con la T al posto della D.



Definizione 3.8 – JK Flip-Flop

Un **JK Flip-Flop** è un D Flip-Flop che ha, oltre al CLK, altri due input: J e K. Ci sono quattro possibili casi:

- Se $J = 0$ e $K = 0$ allora $Q = Q_{\text{prev}}$
- Se $J = 0$ e $K = 1$ allora $Q = 0$
- Se $J = 1$ e $K = 0$ allora $Q = 1$
- Se $J = 1$ e $K = 1$ allora $Q = \bar{Q}_{\text{prev}}$

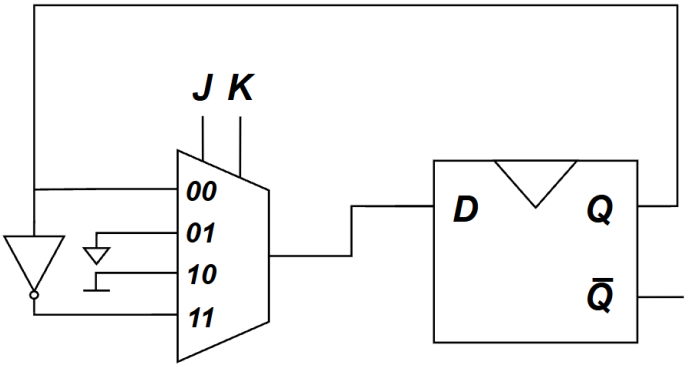
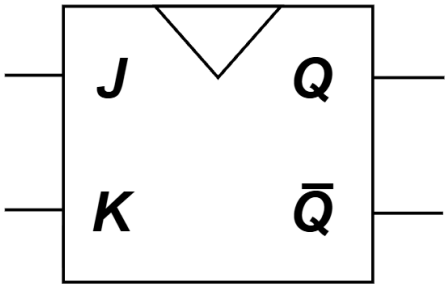


Figura 3.11: Rappresentazione interna di un JK Flip-Flop base utilizzando un D Flip-Flop

Rappresentiamo un JK Flip-Flop come un D Flip-Flop ma con in ingresso J e K.

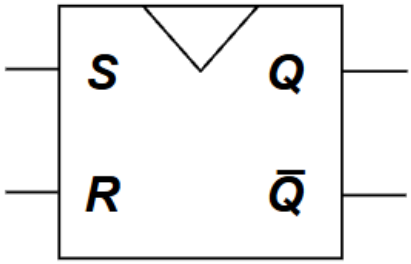


Definizione 3.9 – SR Flip-Flop

Un **SR Flip-Flop** è un JK Flip-Flop che però non ammette la configurazione $S = 1$ e $R = 1$, come gli SR Latch.

La rappresentazione interna di un SR Flip-Flop altro non è che un JK Flip-Flop dove però la combinazione 11 non è collegata a nulla, dato che non è ammessa.

Rappresentiamo un SR Flip-Flop come un D Flip-Flop ma con in ingresso S e R.



Comunque, quando parleremo di Flip-Flop senza specificare ci riferiremo sempre a un D Flip-Flop.

3.3 Progettazione logica sequenziale sincrona

Definizione 3.10 – Progettazione logica sequenziale sincrona

La **progettazione logica sequenziale sincrona** è una metodologia di costruzione dei circuiti sequenziali.

Proprietà 3.1 – Regole della progettazione logica sequenziale sincrona

1) Ogni elemento del circuito è un **registro** o un **circuito combinatorio**
2) Almeno un elemento deve essere un registro
3) Tutti i registri sono collegati allo stesso clock (CLK)
4) Ogni ciclo presente deve essere interrotto da almeno un registro

Esistono due tipi di circuiti sequenziali sincroni:

- **FSM**
- **Pipeline**

Noi vedremo nel dettaglio il primo tipo, le FSM.

3.4 FSM

Definizione 3.11 – FSM

Una **FSM** (Finite State Machine, o Macchina a Stati Finiti) è un circuito sequenziale sincrono con due circuiti combinatori:

- Il primo circuito combinatorio si occupa di generare il **prossimo stato** del registro, e prende il nome di **Next State Logic**
- Il secondo circuito combinatorio si occupa di generare gli **output** del circuito sequenziale, e prende il nome di **Output Logic**

Il prossimo stato è sempre determinato dagli input e dal Current State (stato corrente), mentre, in base a come sono determinati gli output, distinguiamo due tipi di FSM:

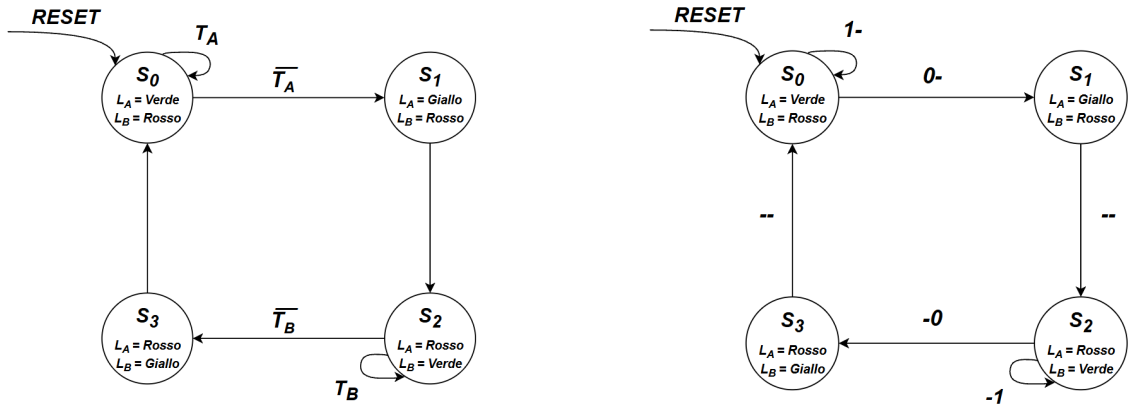
- **Moore FSM**, dove l'output è determinato solo dallo **stato corrente**
- **Mealy FSM**, dove l'output è determinato sia dallo **stato corrente** che dagli **input**

Per capire bene questi circuiti sequenziali sincroni la cosa migliore è fare qualche esempio.

3.4.1 Moore FSM

Ipotizziamo quindi di voler rappresentare un semaforo all'incrocio tra due strade: A e B. Prendiamo quindi in input T_A e T_B, che valgono 1 se c'è traffico nella loro strada e 0 se non c'è, e mandiamo in output L_A e L_B, che rappresentano la luce del semaforo sulla rispettiva strada (vedremo dopo come rappresentare i vari colori in binario). Vogliamo che, quando non c'è più traffico sulla strada con la luce attuale verde, i semafori si invertano, passando prima dal verde al giallo e poi dal giallo al rosso e dal rosso al verde. Ipotizziamo poi di avere un $RESET$ che ci riporti allo stato noto L_A = Verde e L_B = Rosso.

Creiamo per questo esempio una Moore FSM. Per farla, utilizziamo dei cerchi, che rappresentano i vari stati del circuito, e degli archi, che indicano le transizioni da uno stato a un altro. In questo caso avremo una FSM di questo tipo:



N.B. Abbiamo due rappresentazioni del tutto equivalenti qui, ma per comodità useremo spesso la 1°, con T_A e T_B scritti in modo esplicito.

Vediamo ora la **tabella di transizione degli stati**, ossia la tabella che, preso uno stato di partenza e degli input, ci dice quale sarà il prossimo stato (stiamo quindi scrivendo in tabella la logica del circuito combinatorio next state).

Stato corrente	**Input**		**Stato successivo**
S	T_A	T_B	S'
S_0	0	X	S_1
S_0	1	X	S_0
S_1	X	X	S_2
S_2	X	0	S_3
S_2	X	1	S_2
S_3	X	X	S_0

N.B. S' in questo caso NON indica $\bar{S}$ ma indica lo stato successivo di S (usiamo questa notazione per non dover scrivere sempre S_{prev} o S_{succ}).

Dobbiamo ora però tradurre i nostri quattro possibili stati in 0 e 1. Possiamo farlo usando due codifiche:

- Classico binario, usando due bit
- **One-hot**, una codifica che abbiamo già accennato in passato e che fa utilizzo di 4 bit per rappresentare tutti gli stati: in questo caso ogni sequenza assegnata a uno stato avrà tutti 0 tranne uno dei 4 bit che sarà 1

Utilizzando la codifica binaria possiamo tradurre ogni stato in questo modo:

Stato	Codifica
S_0	00
S_1	01
S_2	10
S_3	11

Mentre utilizzando la codifica one-hot possiamo farlo in quest'altro modo:

Stato	Codifica
S_0	0001
S_1	0010
S_2	0100
S_3	1000

Per questo esempio scegliamo di usare la codifica binaria. Riscriviamo dunque la tabella di transizione degli stati sostituendo ogni stato con la sua corrispettiva codifica:

Stato corrente		**Input**		**Stato successivo**	
Q_1	Q_0	T_A	T_B	D_1	D_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

N.B. Abbiamo usato in questo caso Q_i e D_i al posto di S_i e S'_i, ma è possibile scriverlo un po' come si vuole. Ad esempio, avremmo anche potuto scrivere Q_i e Q'_i, ma l'importante è che lo schema rimanga chiaro.

Troviamo quindi che $D_1 = \bar{Q}_1 Q_0 + Q_1 \bar{Q}_0 \bar{T}_B + Q_1 \bar{Q}_0 T_B = \bar{Q}_1 Q_0 + Q_1 \bar{Q}_0 = Q_1 \oplus Q_0$, e che $D_0 = \bar{Q}_1 \bar{Q}_0 \bar{T}_A + Q_1 \bar{Q}_0 \bar{T}_B$.

Vediamo ora invece la **tabella degli output**:

Stato corrente		**Output**	
Q_1	Q_0	L_A	L_B
0	0	Verde	Rosso
0	1	Giallo	Rosso
1	0	Rosso	Verde
1	1	Rosso	Giallo

Da questa tabella è evidente che questa FSM è una FSM di Moore. Infatti qui non sono presenti gli input T_A e T_B, dunque possiamo dedurre che gli output NON dipendono dagli input.

Anche qui dobbiamo scegliere una codifica per le diverse luci. Scegliamo la one-hot in questo caso:

Luce	**Codifica**
Verde	001
Giallo	010
Rosso	100

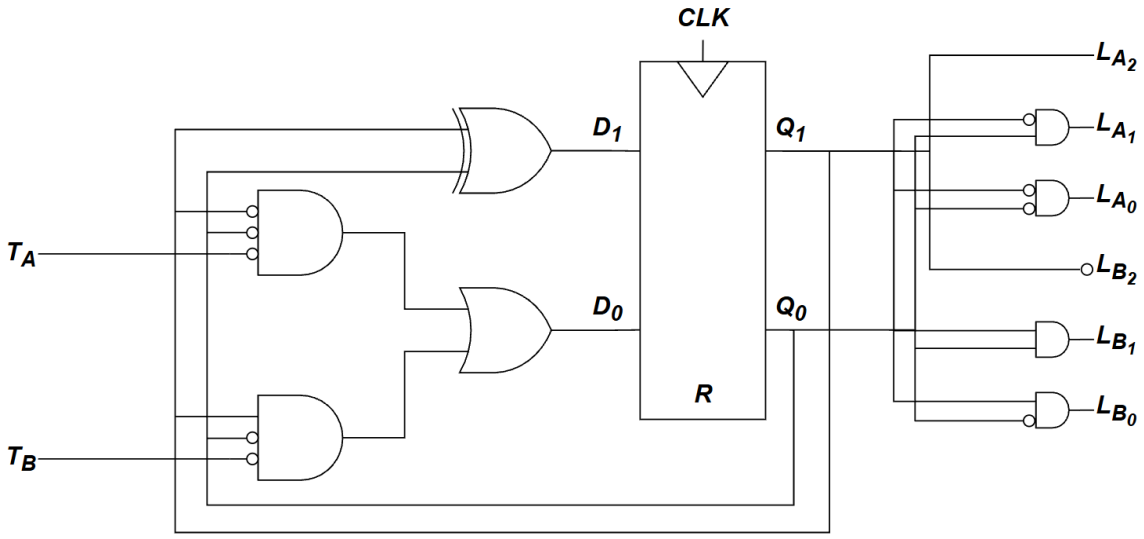
Dunque troviamo questa come tabella degli output finale:

Stato corrente		**Output**					
Q_1	Q_0	L_{A_2}	L_{A_1}	L_{A_0}	L_{B_2}	L_{B_1}	L_{B_0}
0	0	0	0	1	1	0	0
0	1	0	1	0	1	0	0
1	0	1	0	0	0	0	1
1	1	1	0	0	0	1	0

Dunque troviamo:

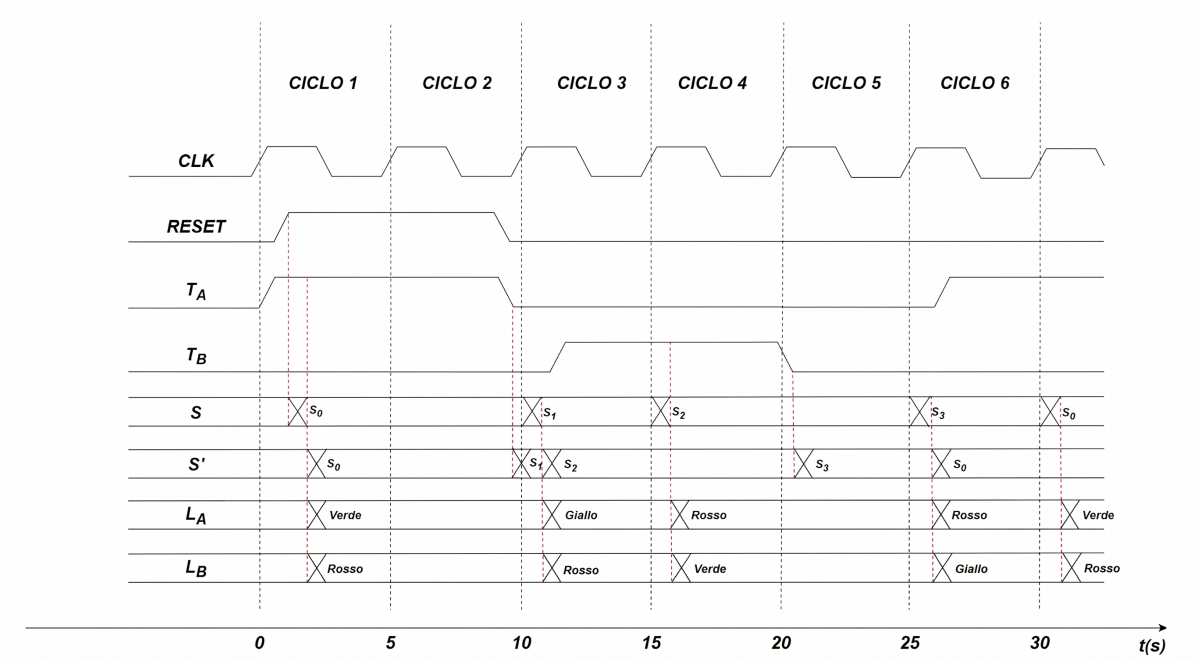
- $L_{A_2} = Q_1\bar{Q}_0 + Q_1Q_0 = Q_1$
- $L_{A_1} = \bar{Q}_1Q_0$
- $L_{A_0} = \bar{Q}_1\bar{Q}_0$
- $L_{B_2} = \bar{Q}_1\bar{Q}_0 + \bar{Q}_1Q_0 = \bar{Q}_1$
- $L_{B_1} = Q_1Q_0$
- $L_{B_0} = Q_1\bar{Q}_0$

Possiamo dunque tradurre questo circuito sequenziale sincrono con questo schema:



A sinistra del registro abbiamo la NSL (Next State Logic), mentre a destra la OL (Output Logic).

Ipotizzando ora di avere un CLK con un periodo di 5 secondi, vediamo come si muove il nostro circuito:



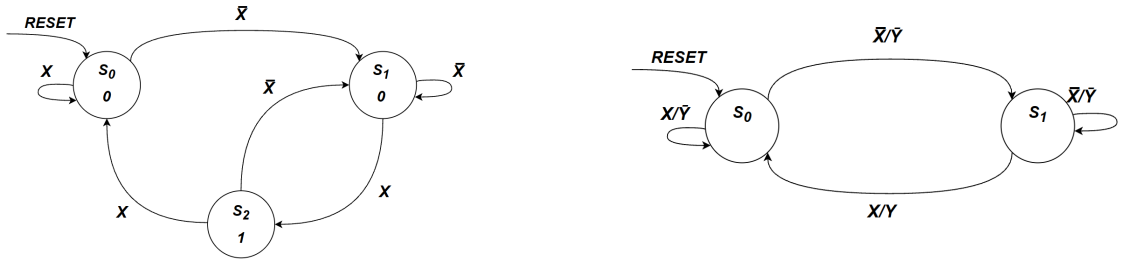
Notiamo come S cambi solo in corrispondenza di un $RESET = 1$ o di una salita del CLK, mentre S' cambia subito in base agli input. Questo ha perfettamente senso: infatti S' è l'output del Next State Logic, ossia di un circuito combinatorio, mentre S è l'output del registro, ossia di tanti D

Flip-Flop. L_A e L_B, invece, cambiano appena S cambia, in modo istantaneo senza aspettare il CLK, dato che anche loro fanno parte di un circuito combinatorio, ossia l'Output Logic.

3.4.2 Mealy FSM

Vediamo ora un altro esempio. Ipotizziamo di avere una sequenza lunghissima di 1 e 0, e di voler restituire in output 1 se gli ultimi due numeri letti sono, nell'ordine, 01, e restituire 0 se invece non lo sono.

Creiamo per questo esempio una Mealy FSM. Per farla, utilizziamo sempre dei cerchi per i vari stati e degli archi per le varie transizioni. La differenza maggiore con una Moore FSM è che qui ogni stato non ha un output prestabilito, dato che questo dipende anche dagli input. Dunque, su ogni arco, dobbiamo scrivere non solo gli input che ci portano a percorrere quell'arco ma anche l'output che ne fuoriesce. Tutto ciò si traduce quindi in questo schema:

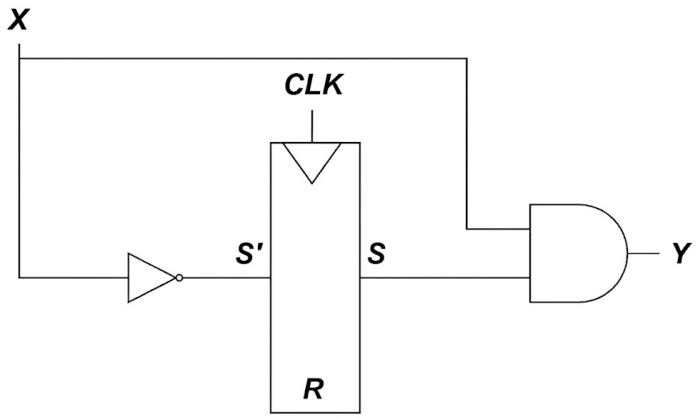


Anche qui abbiamo due rappresentazioni del tutto equivalenti: a sinistra della sbarra c'è sempre l'input, mentre a destra c'è l'output corrispettivo.

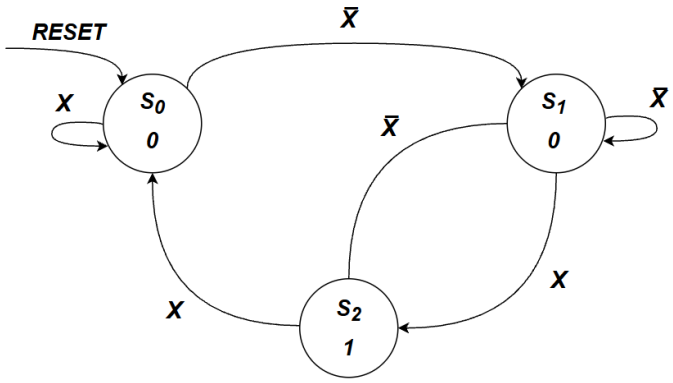
Vediamo ora la tabella di transizione degli stati e degli output (le vediamo insieme, dato che sia gli input che gli stati sono importanti per l'output finale), codificando semplicemente S_0 a 0 e S_1 a 1:

Stato corrente	**Input**	**Stato successivo**	**Output**
S	X	S'	Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

Dunque troviamo $S' = \bar{S}\bar{X} + S\bar{X} = \bar{X}$ e $Y = SX$, che si traduce in questo circuito molto semplice:



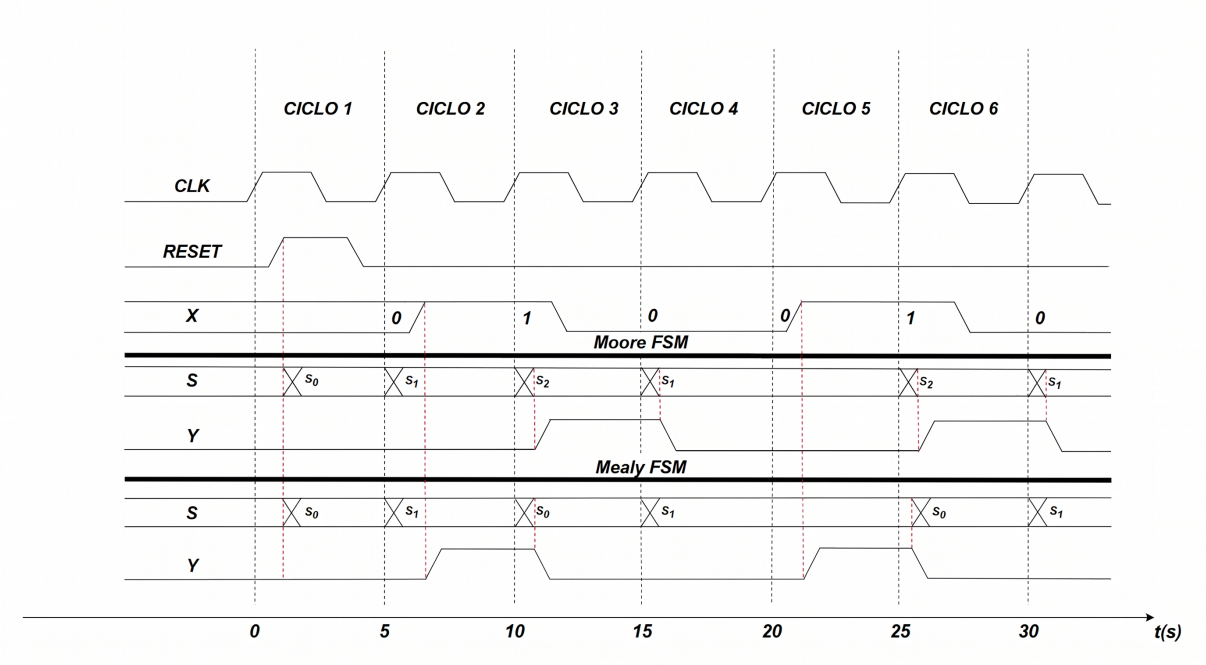
Questo stesso esempio avremmo potuto risolverlo con una Moore FSM, in questo modo:



Proposizione 3.1

Un circuito sequenziale sincrono può sempre essere rappresentato con una FSM Moore o Mealy, ma quella Mealy avrà sempre un numero di stati **minore o uguale**.

Ipotizzando di avere sempre un CLK con un periodo di 5 secondi, vediamo come si muovono i due circuiti Moore e Mealy:

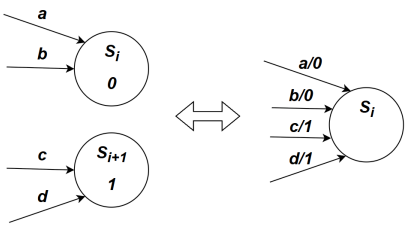


Notiamo una cosa importante: in Moore, Y cambia solo quando abbiamo una salita nel CLK, mentre in Mealy cambia subito, non appena l'input passa da 0 a 1. Per questo motivo, a volte si potrebbe preferire usare uno o l'altro: se magari si vuole un sistema più controllato, Moore è la scelta corretta, mentre se si vuole precisione quasi istantanea meglio Mealy.
N.B. È comunque sempre possibile avere anche in un circuito Mealy delle uscite registrate (ossia uscite che passano per un Flip-Flop e quindi cambiano solo in corrispondenza della salita del CLK) tramite l'utilizzo di un ulteriore Flip-Flop.

Passare da Moore a Mealy e da Mealy a Moore

Come già accenato nella Proposizione 3.1, è sempre possibile, presa una macchina di Moore, passare a una macchina di Mealy, e viceversa. Vediamo ora come farlo.

Per passare da Moore a Mealy basta spostare gli output dagli stati alle transizioni, mentre per passare da Mealy a Moore il procedimento è l'opposto: si spostano gli output dalle transizioni agli stati, sdoppiando uno stato quando le sue transizioni entranti hanno output diversi. Abbiamo qui anche una dimostrazione visiva della Proposizione 3.1: infatti notiamo come, passando da Moore a Mealy, sia possibile accorpare due stati in uno solo.



3.4.3 Esempio di FSM

Prima di vedere un altro esempio di FSM, diamo una definizione che ci tornerà utile tra poco.

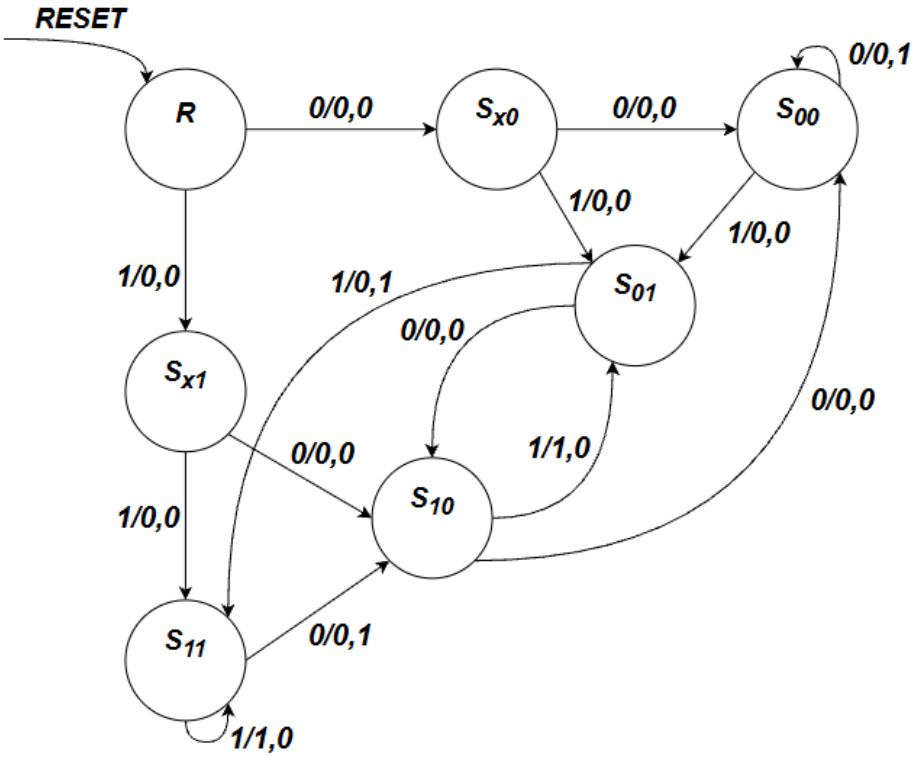
Definizione 3.12 – Stati irraggiungibili

Uno stato di una FSM è detto **irraggiungibile** se questo non è raggiungibile attraverso nessuna sequenza di input.

Ipotizziamo ora di voler progettare un circuito sequenziale con un ingresso X e due uscite Z_1 e Z_0. Consideriamo s la sequenza costituita dagli ultimi tre bit in input di X. L'uscita Z_1 deve valere 1 se s, considerato come un valore in Ca2 (complemento a 2), è un valore negativo dispari, mentre l'uscita Z_0 deve valere 1 se s, considerato come un valore in base 2, è un multiplo di 3. Consideriamo anche eventuali sovrapposizioni. Cerchiamo quindi di creare una macchina di Mealy (utilizzeremo quasi sempre Mealy se non è esplicitamente richiesto di usare Moore) che risolva questo problema.

Per prima cosa, dobbiamo capire quali sequenze di bit facciano valere $Z_1 = 1$ e quali $Z_0 = 1$. Concentriamoci prima su Z_1. Tutti i numeri negativi in complemento a 2 hanno il primo bit pari a 1, dunque non ci resta che verificare quali numeri tra $100, 101, 110$ e 111 siano dispari. Troviamo quindi che $Z_1 = 1 \iff s = 101$ o $s = 111$. Passando ora a Z_0, dato che i multipli di 3 minori di 8 sono solo $0, 3, 6$ troviamo che $Z_0 = 1 \iff s = 000$ o $s = 011$ o $s = 110$.

Ora che abbiamo trovato quando $Z_1 = 1$ e quando $Z_0 = 1$, possiamo realizzare la corrispondente Mealy FSM (l'ordine degli output è ovviamente Z_1, Z_0):



Abbiamo in totale 7 stati: 1 di reset, quando s è vuota, 2 quando la sequenza ha solo un bit, e 4 quando la sequenza ha 2 bit. I primi 3 stati servono solo per arrivare agli ultimi 4, i quali ci forniranno tutti i valori fino a quando la sequenza non finirà. Utilizziamo per rappresentare questi stati una codifica binaria:

Stato	Codifica
R	001
S_{x0}	010
S_{x1}	011
S_{00}	100
S_{01}	101
S_{10}	110
S_{11}	111

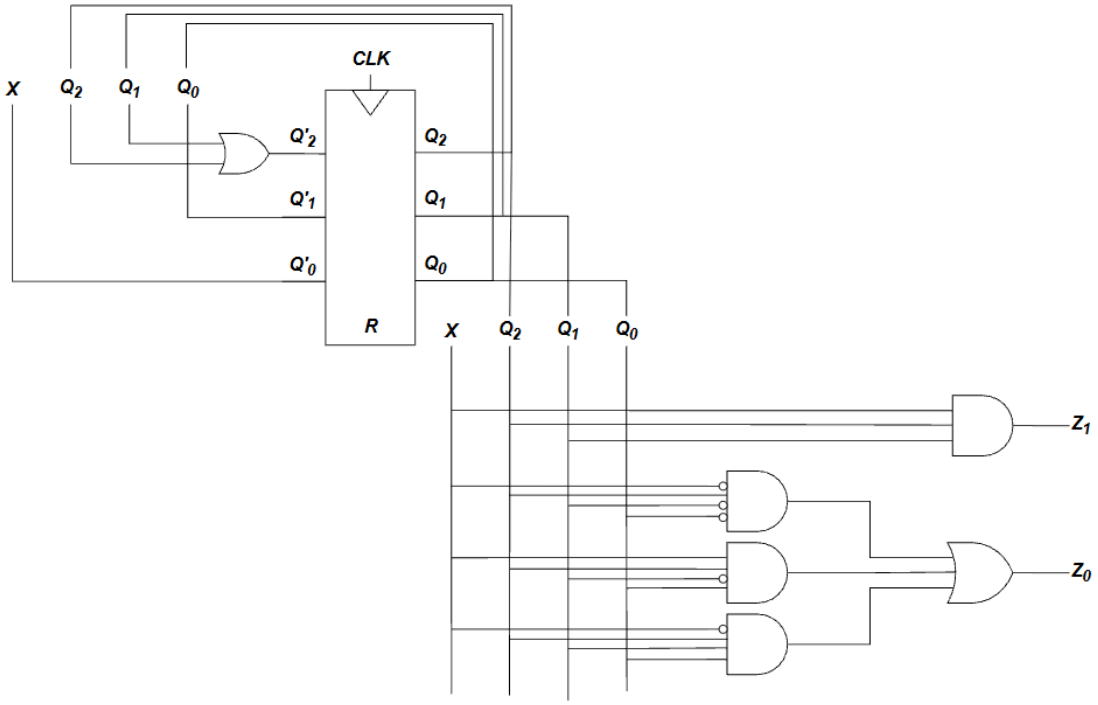
Partiamo da 001 e non da 000 per un motivo molto semplice: in questo modo, gli ultimi 4 stati partono tutti con un 1, e i loro ultimi 2 bit equivalgono agli ultimi due bit della sequenza quando ci troviamo in quello stato.
Vediamo ora la tabella degli stati e degli output:

Stato corrente				Input	Stato successivo				Output	
Stato	Q_2	Q_1	Q_0	X	**Stato**	Q_2'	Q_1'	Q_0'	Z_1	Z_0
R	0	0	1	0	S_{x0}	0	1	0	0	0
R	0	0	1	1	S_{x1}	0	1	1	0	0
S_{x0}	0	1	0	0	S_{00}	1	0	0	0	0
S_{x0}	0	1	0	1	S_{01}	1	0	1	0	0
S_{x1}	0	1	1	0	S_{10}	1	1	0	0	0
S_{x1}	0	1	1	1	S_{11}	1	1	1	0	0
S_{00}	1	0	0	0	S_{00}	1	0	0	0	1
S_{00}	1	0	0	1	S_{01}	1	0	1	0	0
S_{01}	1	0	1	0	S_{10}	1	1	0	0	0
S_{01}	1	0	1	1	S_{11}	1	1	1	0	1
S_{10}	1	1	0	0	S_{00}	1	0	0	0	0
S_{10}	1	1	0	1	S_{01}	1	0	1	1	0
S_{11}	1	1	1	0	S_{10}	1	1	0	0	1
S_{11}	1	1	1	1	S_{11}	1	1	1	1	0

Dunque troviamo:

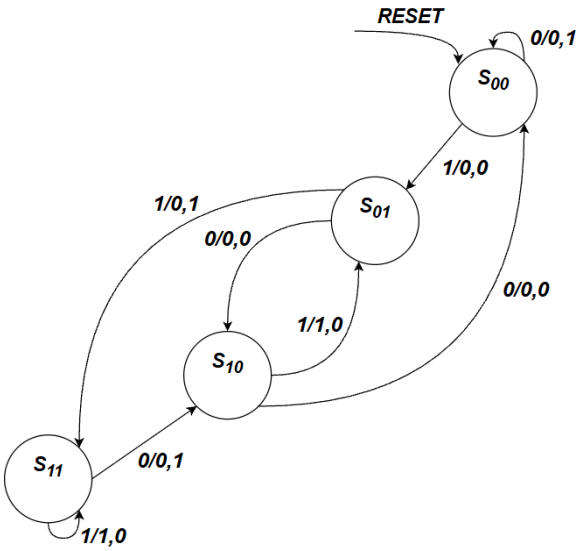
- $Q_0' = X$ (stiamo scorrendo il primo bit)
- $Q_1' = Q_0$ (stiamo scorrendo il secondo bit)
- $Q_2' = Q_2 + Q_1$ (stiamo scorrendo il terzo bit, ma controlliamo anche Q_2 perché se $Q_2 = 1$ allora anche $Q_2' = 1$, dato che $Q_2 = 1$ ci dice che siamo fuori dalla fase iniziale)
- $Z_1 = Q_2Q_1\bar{Q}_0X + Q_2Q_1Q_0X = Q_2Q_1X$ (101 o 111 e $Q_2 = 1$ che ci dice che siamo fuori dalla fase iniziale)
- $Z_0 = Q_2\bar{Q}_1\bar{Q}_0\bar{X} + Q_2\bar{Q}_1Q_0X + Q_2Q_1Q_0\bar{X}$ (000 o 011 o 110 e $Q_2 = 1$ che ci dice che siamo fuori dalla fase iniziale)

Il circuito risultante sarà quindi il seguente:



Riprendiamo ora questo esempio e modifichiamolo leggermente. Vogliamo ora trascurare i primi due output, ossia quelli che restituiscono sempre 0 perché s non è ancora formata da 3 bit.

Al posto di rifare tutto il circuito di Mealy da capo, notiamo che ci basta prendere il circuito precedente ed eliminargli i primi 3 stati, in questo modo:



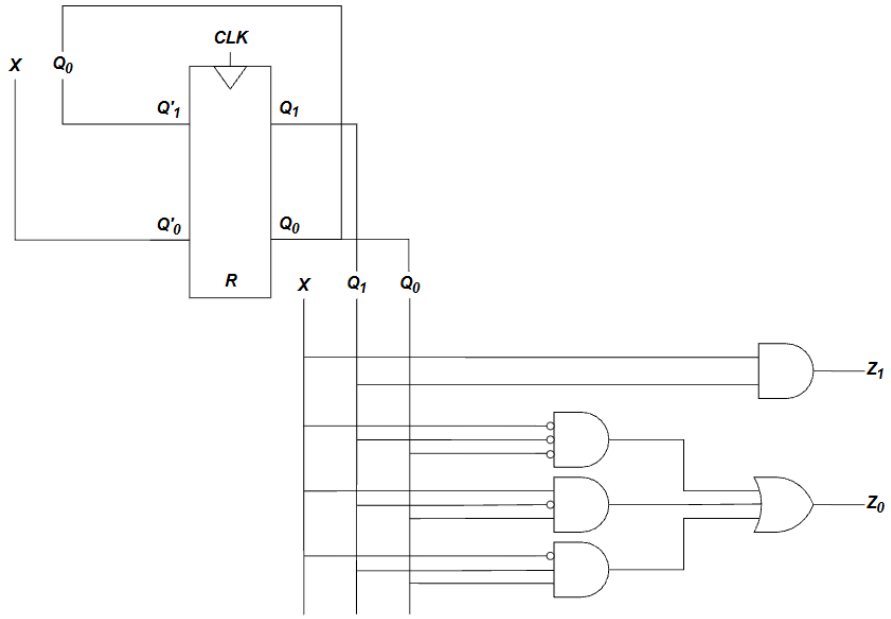
Abbiamo quindi reso gli stati R, S_{x0} e S_{x1} **irraggiungibili**. A questo punto anche le codifiche e la tabella degli stati e degli output possiamo riprenderle da quelle già fatte, eliminando di fatto un bit per la codifica portandola a una codifica a soli due bit:

Stato corrente			**Input**	**Stato successivo**			**Output**	
Stato	Q_1	Q_0	X	**Stato**	Q_1'	Q_0'	Z_1	Z_0
S_{00}	0	0	0	S_{00}	0	0	0	1
S_{00}	0	0	1	S_{01}	0	1	0	0
S_{01}	0	1	0	S_{10}	1	0	0	0
S_{01}	0	1	1	S_{11}	1	1	0	1
S_{10}	1	0	0	S_{00}	0	0	0	0
S_{10}	1	0	1	S_{01}	0	1	1	0
S_{11}	1	1	0	S_{10}	1	0	0	1
S_{11}	1	1	1	S_{11}	1	1	1	0

Anche i valori dei bit di stato e gli output sono identici a prima, ma senza Q_2:

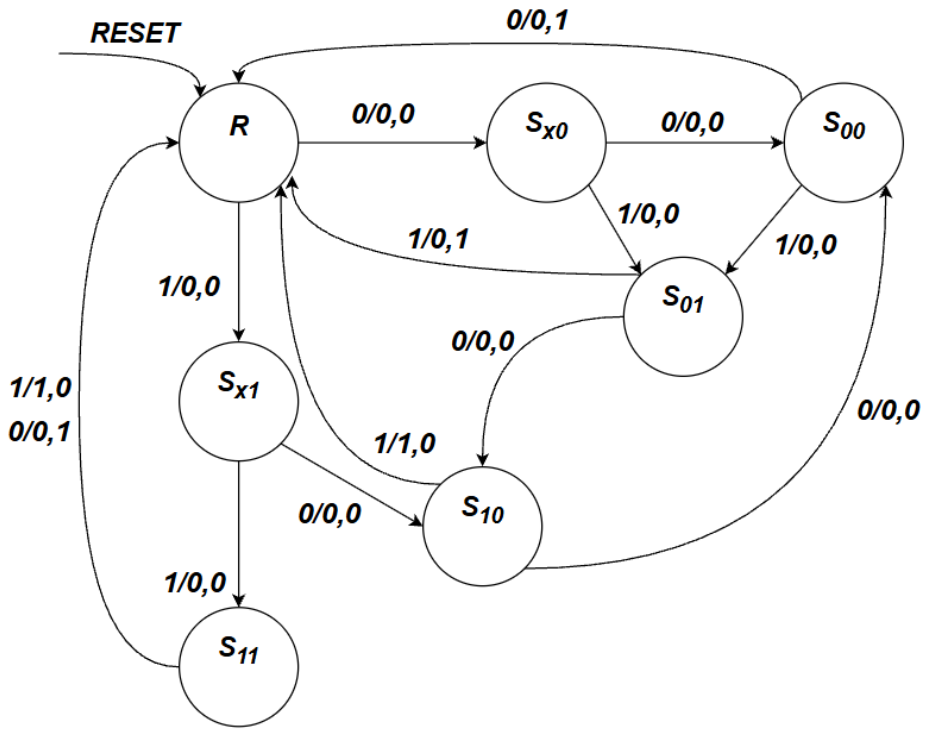
- $Q_0' = X$
- $Q_1' = Q_0$
- $Z_1 = Q_1\bar{Q}_0X + Q_1Q_0X = Q_1X$ (101 o 111)
- $Z_0 = \bar{Q}_1\bar{Q}_0\bar{X} + \bar{Q}_1Q_0X + Q_1Q_0\bar{X}$ (000 o 011 o 110)

Troviamo quindi il seguente circuito:



Consideriamo ora il caso in cui i primi due output non sono trascurati ma le sovrapposizioni non sono ammesse. Ciò significa che i simboli che hanno completato una sequenza non possono essere riutilizzati come inizio della sequenza successiva. In pratica, ogni volta che viene riconosciuta una sequenza (cioè l'uscita diventa $Z_1 = 1$ oppure $Z_0 = 1$), la macchina deve ripartire da zero, tornando allo stato iniziale R, come se non avesse letto nulla.

Possiamo anche qui riprendere il primo schema Mealy e mandare tutte le transizioni che hanno un 1 tra gli output allo stato R, in questo modo:



Anche per la tabella degli stati e degli output possiamo prendere quella già fatta all'inizio, modificando lo stato successivo di ogni riga che ha un output pari a 1 con il valore 001, che si

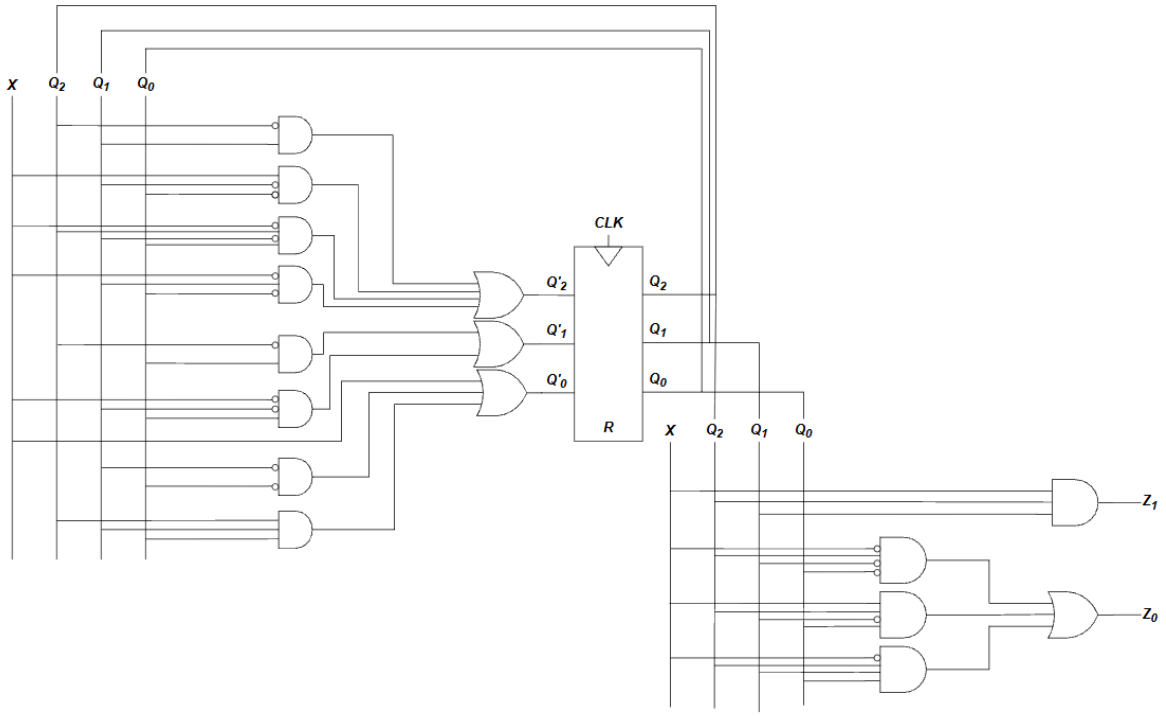
riferisce proprio allo stato R:

Stato corrente				**Input**	**Stato successivo**				**Output**	
Stato	Q_2	Q_1	Q_0	X	**Stato**	Q_2'	Q_1'	Q_0'	Z_1	Z_0
R	0	0	1	0	S_{x0}	0	1	0	0	0
R	0	0	1	1	S_{x1}	0	1	1	0	0
S_{x0}	0	1	0	0	S_{00}	1	0	0	0	0
S_{x0}	0	1	0	1	S_{01}	1	0	1	0	0
S_{x1}	0	1	1	0	S_{10}	1	1	0	0	0
S_{x1}	0	1	1	1	S_{11}	1	1	1	0	0
S_{00}	1	0	0	0	R	0	0	1	0	1
S_{00}	1	0	0	1	S_{01}	1	0	1	0	0
S_{01}	1	0	1	0	S_{10}	1	1	0	0	0
S_{01}	1	0	1	1	R	0	0	1	0	1
S_{10}	1	1	0	0	S_{00}	1	0	0	0	0
S_{10}	1	1	0	1	R	0	0	1	1	0
S_{11}	1	1	1	0	R	0	0	1	0	1
S_{11}	1	1	1	1	R	0	0	1	1	0

I valori dei bit di stato cambiano leggermente, mentre gli output rimangono ovviamente identici:

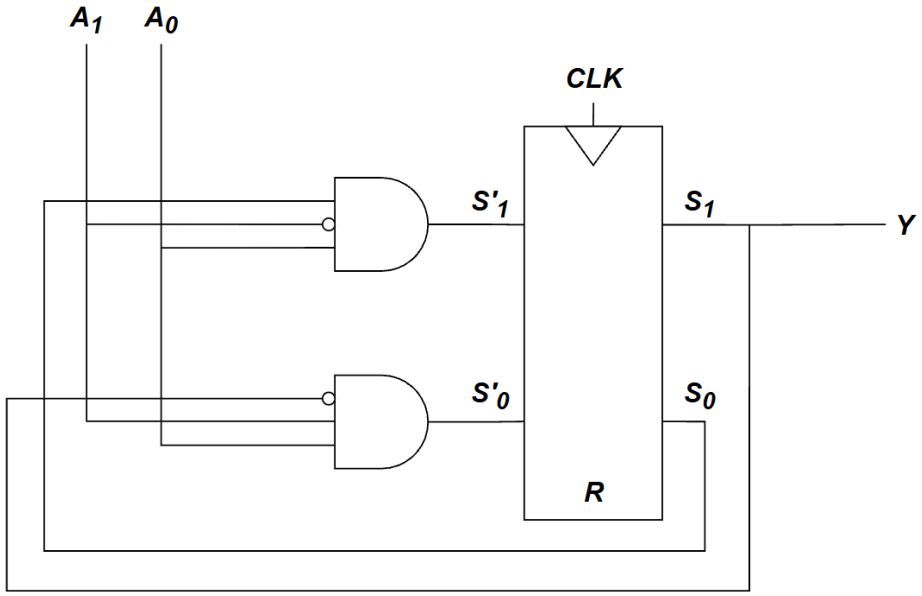
- $Q_0' = X + \bar{Q}_1\bar{Q}_0 + Q_2Q_1Q_0$
- $Q_1' = \bar{Q}_2Q_0 + \bar{Q}_1Q_0\bar{X}$
- $Q_2' = \bar{Q}_2Q_1 + \bar{Q}_1\bar{Q}_0X + Q_2\bar{Q}_1Q_0\bar{X} + Q_1\bar{Q}_0\bar{X}$
- $Z_1 = Q_2Q_1X$
- $Z_0 = Q_2\bar{Q}_1\bar{Q}_0\bar{X} + Q_2\bar{Q}_1Q_0X + Q_2Q_1Q_0\bar{X}$

Troviamo quindi il seguente circuito:



3.4.4 Passare da un circuito di una FSM al diagramma degli stati

Fino ad ora abbbiamo visto in una FSM come passare dal diagramma degli stati al circuito finale. Vediamo ora un esempio dove proviamo invece a fare l'opposto. Prendiamo il seguente circuito e cerchiamo di ricavarne il diagramma degli stati da cui esso è stato generato:



Da questo circuito troviamo che:

- $S_1' = S_0 \bar{A_1} A_0$
- $S_0' = \bar{S_1} A_1 A_0$
- $Y = S_1$

Dato che l'output non dipende dagli input, concludiamo subito che questa è una Moore FSM, dunque dobbiamo ricavarci due tabelle distinte: una per i prossimi stati e una per gli output:

Tabella degli stati

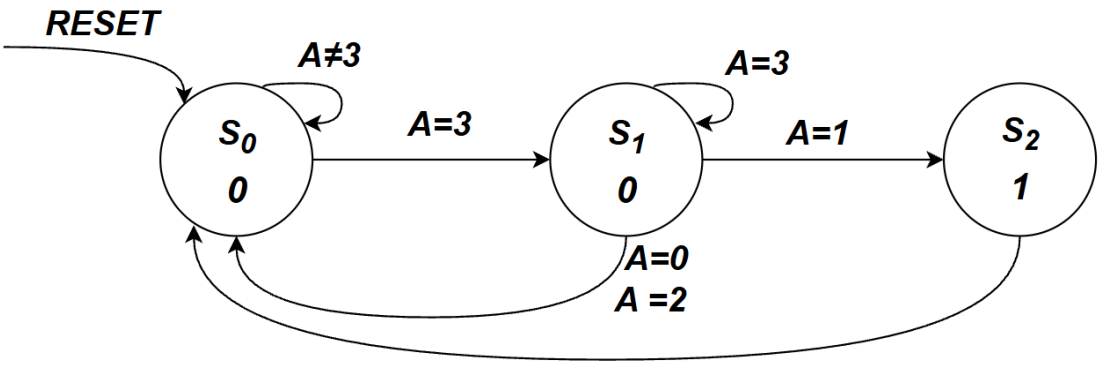
Stato corrente			**Input**		**Stato successivo**		
Stato	S_1	S_0	A_1	A_0	**Stato**	S_1'	S_0'
S_0	0	0	0	0	S_0	0	0
S_0	0	0	0	1	S_0	0	0
S_0	0	0	1	0	S_0	0	0
S_0	0	0	1	1	S_1	0	1
S_1	0	1	0	0	S_0	0	0
S_1	0	1	0	1	S_2	1	0
S_1	0	1	1	0	S_0	0	0
S_1	0	1	1	1	S_1	0	1
S_2	1	0	0	0	S_0	0	0
S_2	1	0	0	1	S_0	0	0
S_2	1	0	1	0	S_0	0	0
S_2	1	0	1	1	S_0	0	0
S_3	1	1	0	0	S_0	0	0
S_3	1	1	0	1	S_2	1	0
S_3	1	1	1	0	S_0	0	0
S_3	1	1	1	1	S_0	0	0

Tabella degli output

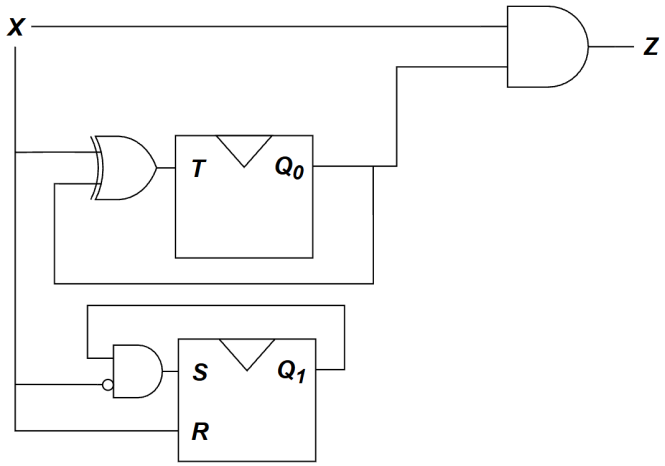
Stato	S_1	S_0	Y
S_0	0	0	0
S_1	0	1	0
S_2	1	0	1
S_3	1	1	1

N.B. Usiamo 2 bit per gli stati perché abbiamo 2 Flip-Flop nel nostro registro, dunque avremo in totale 4 stati da analizzare.

Dalla tabella degli stati notiamo una cosa molto interessante: lo stato S_3, dato che il reset punta allo stato S_0, **non è raggiungibile**. Infatti, non c'è nessuna combinazione di input che da S_0 ci porti a S_3, dunque possiamo anche non rappresentarlo nel nostro diagramma degli stati:



Vediamo ora un altro circuito da analizzare:



Questo esempio è interessante per un motivo molto semplice. Se infatti lo guardiamo attentamente, notiamo subito una cosa: il secondo Flip-Flop (l'SR Flip-Flop) non è collegato a nulla! Questo significa che non influisce nell'FSM e che quindi è come se non esistesse. Il nostro output Z non è collegato in alcun modo a questo Flip-Flop. Troviamo quindi le seguenti equazioni:

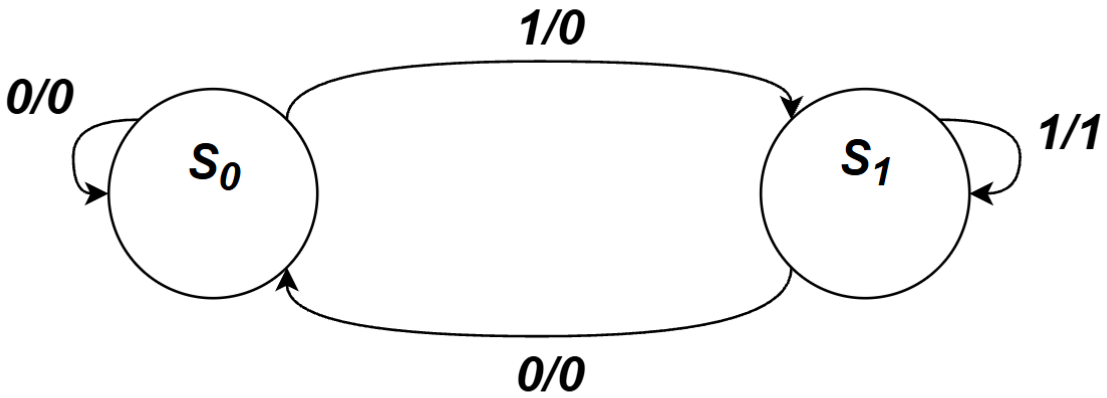
- $Q_0 = T\bar{Q}_0 + \bar{T}Q_0 = T \oplus Q_0 = X \oplus Q_0 \oplus Q_0 = X \oplus 0 = X$
- $Z = XQ_0$

Dato che l'output dipende dall'input X ci troviamo davanti un FSM Mealy. Rappresentiamo quindi la tabella degli stati e degli output in un'unica tabella:

Tabella degli stati e degli output

Stato corrente		Input	Stato successivo		Output
Stato	S	X	**Stato**	S'	Z
S_0	0	0	S_0	0	0
S_0	0	1	S_1	1	0
S_1	1	0	S_0	0	0
S_1	1	1	S_1	1	1

Troviamo quindi il seguente diagramma degli stati:



3.4.5 Contatori

Definizione 3.13 – Contatore

Un **contatore** è una Moore FSM che ad ogni ciclo di clock incrementa il proprio valore di un bit. Ad esempio, con un contatore a 3 bit avremo in totale 9 possibili valori che si ripetono in sequenza ogni 9 cicli di clock.

Questo viene utilizzato negli orologi digitali, per contare i secondi/minuti, e per il Program Counter (PC) del processore, che tiene traccia dell'istruzione che si sta eseguendo.

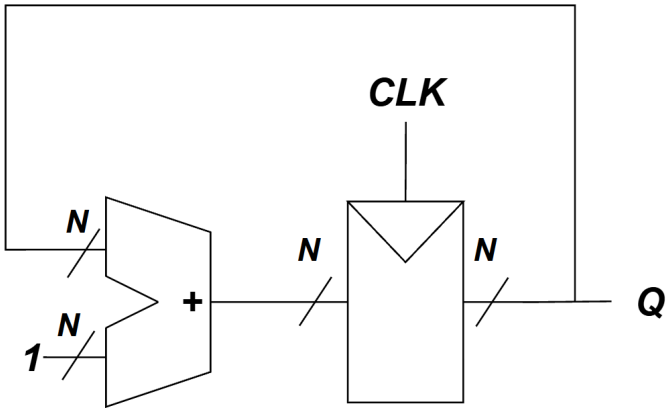
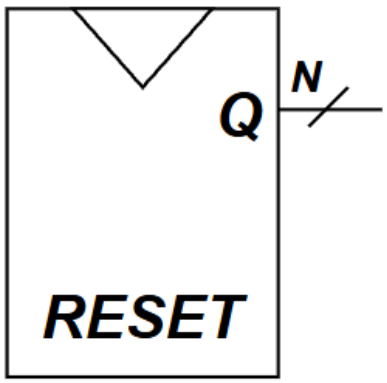


Figura 3.12: Rappresentazione interna di un contatore

N.B. L'elemento a sinistra del registro è un **sommatore a N bit**: è un costrutto che vedremo più avanti, ma per ora ci basta sapere che somma i due valori che prende in input, ossia Q e 1_{10} (1 decimale, ossia $0 \ldots 01$ con $N-1$ zeri).

Rappresentiamo un contatore come un Flip-Flop senza input ma con in output una Q formata da N bit.



3.5 Registri a scorrimento

Definizione 3.14 – Registro a scorrimento

Un **registro a scorrimento** (shift register) a N bit è una catena di N flip-flop collegati allo stesso CLK, dove l'uscita di ciascun flip-flop è collegata all'ingresso del successivo. A ogni ciclo del clock:

- un nuovo bit entra da S_{in} nel primo flip-flop
- ogni bit si sposta di una posizione lungo la catena
- il bit contenuto nell'ultimo flip-flop esce da S_{out}

Le uscite di tutti i flip-flop nei registri a scorrimento sono disponibili contemporaneamente, quindi il registro a scorrimento funziona da **convertitore seriale-parallelo**: i bit entrano uno alla volta ed escono tutti insieme in parallelo.

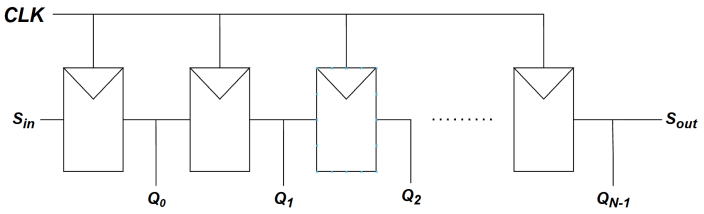
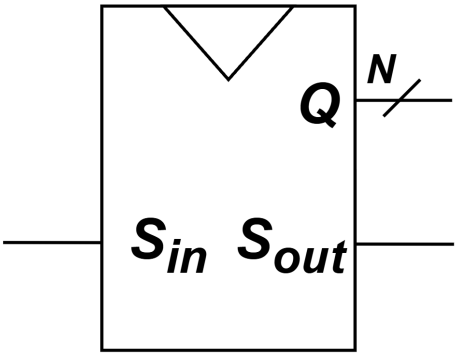


Figura 3.13: Rappresentazione interna di un registro a scorrimento

Rappresentiamo un registro a scorrimento come un contatore ma con in input S_{in} e in output, oltre a Q, anche S_{out}.



3.5.1 Registri a scorrimento con input paralleli

Esiste un'altra versione dei registri a scorrimento, nella quale è possibile caricare tutti i flip flop in parallelo. Questi registri prendono il nome di **registri a scorrimento con input paralleli**.

Definizione 3.15 – Registro a scorrimento con input paralleli

Un **registro a scorrimento con input paralleli** è un registro a scorrimento dei nuovi input: $LOAD$, che se vale 0 fa lavorare questo registro come un normale registro a scorrimento, mentre se vale 1 lo fa lavorare come un normale registro a N bit, inserendo nei Flip-Flop N valori in input in contemporanea ($D_0, \ldots, D_{N-1}$).

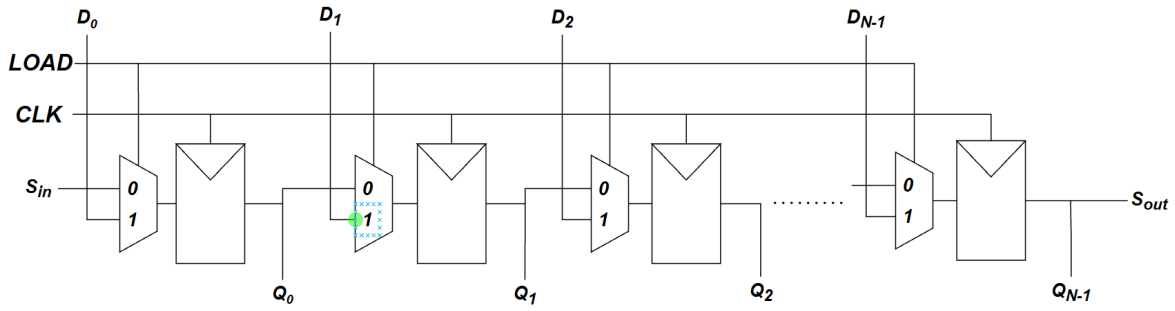


Figura 3.14: Rappresentazione interna di un registro a scorrimento con input paralleli

3.5.2 Trasmettitore seriale

Grazie a questi due nuovi registri che abbiamo appena studiato possiamo creare un costrutto fondamentale, che prende il nome di **trasmettitore seriale**.

Definizione 3.16 – Trasmettitore seriale

Un **trasmettitore seriale** è un circuito che invia N bit facendoli passare uno alla volta su un unico filo. Si realizza con due registri a scorrimento (uno con input paralleli, l'altro senza) collegati allo stesso CLK:

- Il primo registro SR_1 è un registro a scorrimento con input paralleli: con $LOAD = 1$ carica in un ciclo gli N bit e poi, con $LOAD = 0$, li fa scorrere uno alla volta sul filo.
- Il secondo registro SR_2 è un registro a scorrimento che riceve i bit emessi da SR_1 e, dopo N cicli, li rende disponibili in parallelo su D_{out}.

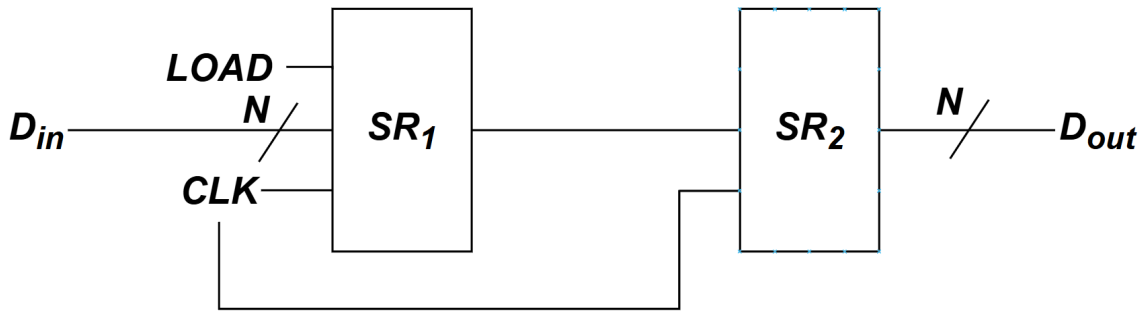


Figura 3.15: Rappresentazione interna di un trasmettitore seriale

Capitolo 4

Memorie

4.1 Introduzione ai memory array

Fino ad ora abbiamo usato dei Flip-Flop per memorizzare un singolo bit e dei registri per memorizzare più bit insieme. Queste strutture sono ottime per memorizzare piccole quantità di dati, ma per memorie più grandi sono troppo costose. Per questo motivo introduciamo una nuova struttura: i **memory array**.

Definizione 4.1 – Memory array

Un **memory array** è una struttura che memorizza in modo efficiente grandi quantità di dati. Questo altro non è che un **array bidimensionale di celle di bit**. Ogni riga di questo array bidimensionale rappresenta una **parola**, ed a ogni accesso è possibile leggere/scrivere il contenuto di una riga intera specificandone l'indirizzo. Dunque, dato che le righe sono distinte dall'indirizzo, con N bit di indirizzo avremo 2^N possibili righe e quindi parole, che sono lunghe M bit. Avremo quindi una matrice con 2^N righe e M colonne. Il numero di righe prende il nome di **lunghezza**, mentre il numero di colonne rappresenta la **larghezza** del memory array. Infine, la **dimensione** del memory array è la lunghezza $\times$ la larghezza, ossia $2^N \times M$ bit.
Esistono tre tipi di memory array:

- **DRAM** (Dynamic Random Access Memory)
- **SRAM** (Static Random Access Memory)
- **ROM** (Read Only Memory)

Le RAM sono memorie **volatili**, mentre la ROM è **non volatile**. Questo significa che una volta tolta la corrente le RAM vengono completamente cancellatate e tutti i loro dati sono eliminati, mentre ciò non accade per la ROM.
N.B. La parola *random* nell'acronimo RAM non significa che queste memorie prendono dati casuali, ma piuttosto che è possibile trovare un dato in una qualunque posizione senza alcuna sequenzialità necessaria.

Ad esempio, una memoria da 32Kb (ossia 4KB) può essere rappresentata con 1024 parole da 32 bit. In questo caso avremo quindi $N = 10$ bit di indirizzo, dato che $2^{10} = 1024$, e $M = 32$ bit di

dato, per un totale proprio di 32768 bit, ossia 32Kb.

Dato che sulle memorie ROM non è possibile scrivere (o comunque il processo risulta molto lento), non è necessario nessun bit che ci dica se stiamo scrivendo o leggendo. Questo bit è invece presente nelle memorie RAM, ed è salvato in una variabile chiamata ***WE*** (Write Enable). Quando WE vale 1 stiamo scrivendo in memoria, mentre quando WE vale 0 stiamo leggendo dalla memoria.

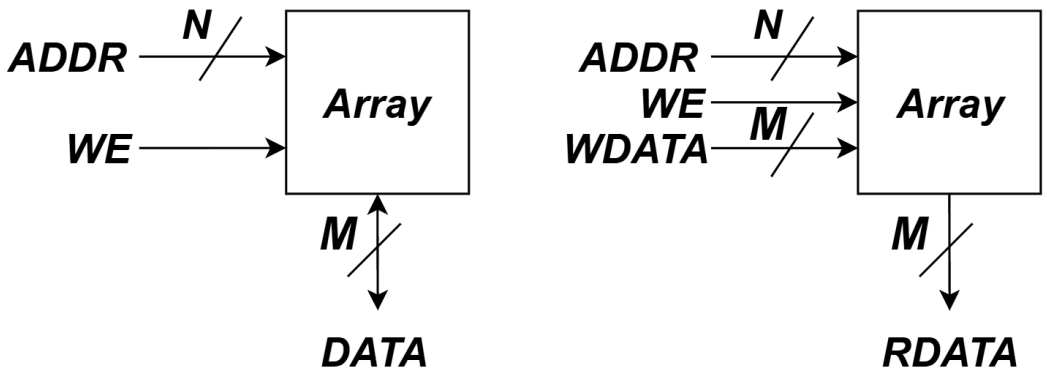


Figura 4.1: Rappresentazione schematica della memoria RAM

Una ROM viene rappresentata come una RAM nella 1° rappresentazione ma senza l'ingresso WE e con la freccia che va solo dall'array a $DATA$ e non viceversa.

4.1.1 Interno di un memory array

Abbiamo visto come un memory array sia fatto internamente di sole celle di bit. Queste devono fare solo due cose: conservare i bit e, nel caso vengano selezionate, comunicare il bit all'esterno. Esistono per questa motivazione due diversi fili che si collegano ad ogni cella:

- La **wordline** è un filo orizzontale comune a tutte le celle di una riga/parola, che vale 0 quando la riga non è stata selezionata per la lettura/scrittura, mentre vale 1 quando invece lo è.
- La **bitline** è un filo verticale comune a tutte le celle di una colonna sul quale passa il valore da mandare dentro la cella selezionata se si sta scrivendo o da mandare fuori dalla cella selezionata se si sta leggendo.

Per implementare la wordline in modo che solo una riga alla volta venga selezionata si utilizza un costrutto che abbiamo già visto in passato: un **decoder**, utilizzato in questo modo:

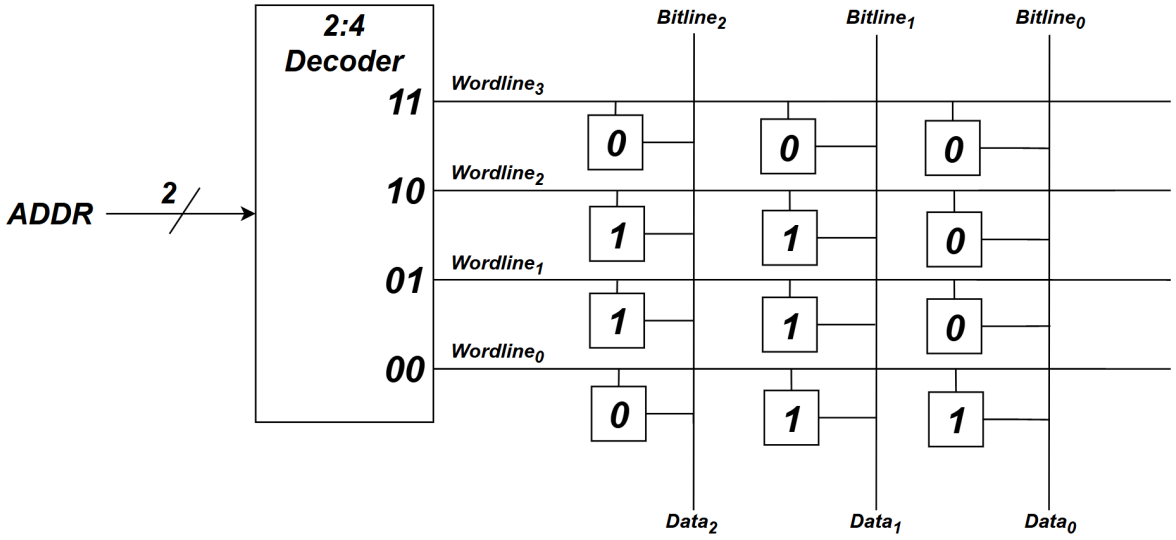


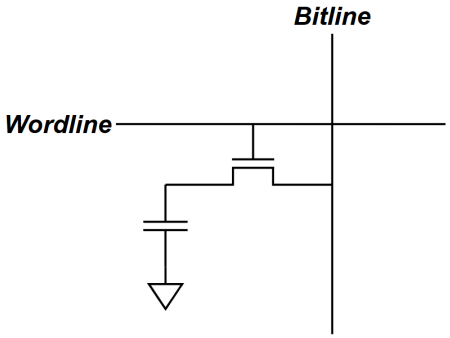
Figura 4.2: Rappresentazione interna di un memory array 4×3

Ovviamente in base alla modalità questo memory array cambia direzione: se ci troviamo in modalità lettura, i dati vengono letti e salvati nelle variabili $Data_2$, $Data_1$ e $Data_0$, mentre se ci troviamo in modalità scrittura i dati vengono presi da queste stesse variabili e sovrascritti nelle celle della word selezionata dall'indirizzo.

4.2 Memorie RAM

Lo schema che abbiamo appena visto è valido sia per le memorie ROM che per quelle RAM. Di quest'utlime, però, abbiamo visto che ne esistono due tipi: DRAM e SRAM, che differiscono nel **modo in cui queste salvano i dati**.

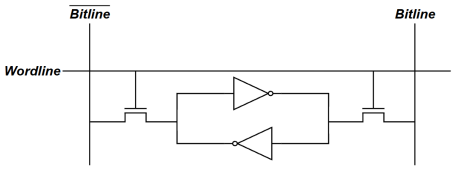
Le DRAM utilizzano dei **condensatori** per memorizzare i bit, e un **transistor**, una specie di interruttore tra la bitline e il bit che viene attivato e fa passare la corrente solo quando la wordline vale 1.



Ma come fa la bitline a leggere il valore della cella? Semplicemente, la bitline parte con una carica X, che, quando si collega al condensatore, varia. In particolare, se il condensatore è carico X aumenta, mentre se è scarico diminuisce. Questa gestione ha, però, un problema molto grande: ad ogni lettura, il valore del bit viene perso, ossia **la lettura è distruttiva**. Questo significa che, ad ogni lettura, bisogna riscrivere il valore nella cella. Questa riscrittura, poi, non va effettuata

solo quando c'è una lettura, ma anche **periodicamente** per salvaguardare il valore della cella. Infatti, l'isolamento di questi sistemi non è perfetto, dunque la carica dopo pochi millisecondi si va a perdere. Per questo motivo, ogni circa $32/64ms$ bisogna riscrivere la cella per mantenerne il valore intatto. Il nome *dinamica* deriva proprio dal fatto che il valore non resta mai fermo, ma tende a cambiare e a essere riscritto.

Le SRAM utilizzano invece un **bistabile** per memorizzare i bit, collegato a non una bitline ma a due: una negata e una normale. In totale utilizzano 6 transistor (dato che ogni `NOT` ne utilizza internamente 2).



La bitline in questo caso legge/scrive i nodi con la stessa idea della DRAM, ma in questo caso non vi è alcuna distruzione perché i bit sono sempre salvi nel bistabile. Il nome *stabile* deriva proprio dal fatto che il valore, se nessuno lo tocca, resta fermo e costante nel tempo. Sorge però qui un'altra domanda: come si fa a sovrascrivere il dato all'interno del bistabile per effettuare una scrittura? Semplicemente, le bitline vengono forzate dall'esterno così da costringere il circuito a sovrascrivere i dati al proprio

4.2.1 Confronto tra le diverse RAM e i Flip-Flop

Tipo	Transistori per bit	Velocità
Flip-flop	Circa 20	Alta
SRAM	6	Media
DRAM	1	Bassa

Dunque la DRAM è sicuramente la memoria più economica, ma è anche la più lenta tra le 3. Questo per due motivi:

- Il valore va sempre riscritto ad ogni lettura
- La carica sulla bitline cambia di pochissimo: è necessario un amplificatore che la renda leggibile, e questo richiede tempo

Le SRAM invece, essendo collegate alla corrente, riempiono la bitline più velocemente rispetto alle DRAM, ma non raggiungono la velocità dei Flip-Flop, dato che questi restituiscono direttamente il bit su un semplice filo senza alcun passaggio intermedio.

Le DRAM, essendo economiche, vengono quindi usate per la memoria principale di un computer (la classica memoria RAM di cui sentiamo spesso parlare è proprio DRAM)

4.2.2 Chip DRAM

Lo schema visto fino ad ora, che utilizza una riga per ogni singola parola, funziona per memorie piccole, ma inizia ad essere problematico quando ci spostiamo su memorie più grandi. Già una memoria con 1024 parole da 32 bit avrebbe 1024 wordline e solo 32 bitline, creando una striscia molto alta ma molto stretta, che rallenta notevolmente la lettura. Per questo introduciamo ora un nuovo sistema che si basa su un presupposto molto semplice: **mettiamo sulla stessa wordline più parole**. Mettendo più parole su una stessa riga il decoder non è più sufficiente, ma è necessario un oggetto che ci permetta di scegliere tra le diverse parole selezionate. Questo oggetto non è altri che un **mux**. Abbiamo quindi ora un decoder che sceglie quale riga selezionare, e un mux che invece sceglie quale colonna selezionare. Il nostro indirizzo sarà quindi diviso in 2 parti, una per dire quale riga selezionare e una per dire quale colonna selezionare.
Riprendendo l'esempio di prima delle 1024 parole da 32 bit, se utilizziamo righe da 32 parole possiamo trovare un **chip quadrato** (in realtà sono 32 righe e 1024 colonne, ma dato che le colonne sono divise in gruppi di 32, troviamo 32 righe e 32 "gruppi di colonne"). Dunque, con un indirizzo di 10 bit, ne utilizzeremo 5 per decidere quale riga selezionare ($A_9, \ldots, A_5$), e 5 per decidere quale colonna selezionare ($A_4, \ldots, A_0$).

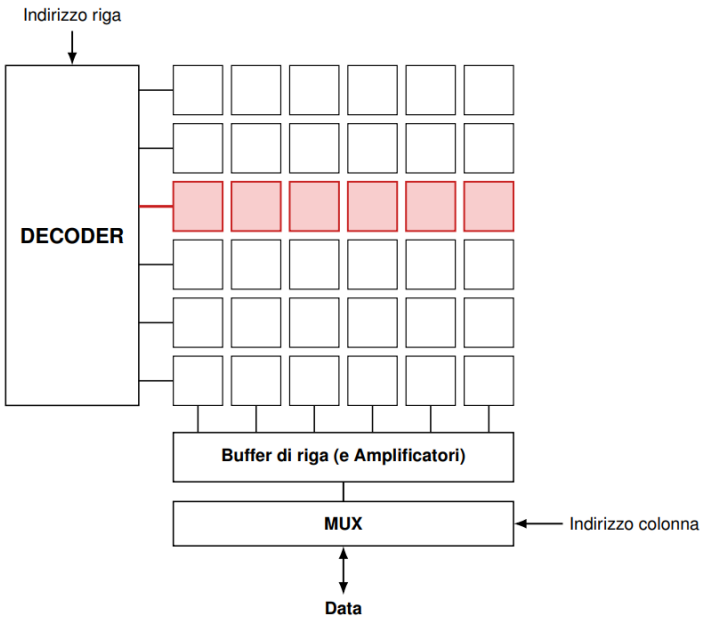


Figura 4.3: Chip interno di una DRAM

I chip delle DRAM utilizzano proprio il principio che abbiamo appena visto, mettendo quindi più word sulla stessa riga . Esistono poi tre comandi che gestiscono questi chip:

- **`ACT`**, che attiva una wordline e copia l'intera riga nel **buffer di riga**, dove questa viene momentaneamente salvata

- **`RD/WR`** (read/write), che sceglie una delle parole all'interno del buffer di riga per leggerla o sovrascriverla
- **`PRE`**, che riscrive l'intera riga nel memory array dato che questa viene distrutta durante la lettura eseguita con `ACT` e prepara la bitline per la prossima `ACT`

Quando vogliamo leggere una parola su una nuova riga, dobbiamo effettuare tutte e tre queste operazioni, nell'ordine `PRE`, `ACT` e `RD`. Se però subito dopo vogliamo leggere una parola su questa stessa riga, non è necessario rieffettuare tutte e tre queste operazioni, perché la riga è già salvata nel buffer di riga, dunque ci basta effettuare semplicemente una `RD`. Dato che ogni comando ha un costo di $14/17ns$, se effettuiamo un cosidetto **row hit**, ossia se troviamo la riga che vogliamo già nel buffer di riga, riusciamo a risparmiare $28/34ns$, altrimenti, se effettuaimo un **row miss**, ossia se non troviamo la riga che vogliamo nel buffer di riga, dovremo impiegare $42/51ns$ per effettuare tutte e 3 le operazioni.

Definizione 4.2 – Banchi

Ogni chip non è composto da un'unica matrice, ma da più matrici indipendenti, che prendono il nome di **banchi** (banks). Ogni banco ha un proprio decoder e un proprio mux per poter lavorare in modo indipendente e soprattutto in modo **parallelo**, così da ridurre ancora di più i tempi di lettura/scrittura.

Con più banchi è possibile poi avere più righe salvate nel buffer di riga, così da aumentare ancora di più i row hit rispetto ai row miss.

4.2.3 DDR

Abbiamo già visto come le DRAM siano delle memorie comuqnue abbastanza lente. Possiamo provare a migliorare la loro efficienza in due modi:

- Introducendo le **SDRAM** (Syncrhonous DRAM), ossia DRAM dove comandi e dati vengono sincronizzati da un clock, così da rendere più prevedibile e schematica la gestione degli accessi alla RAM, aumentando i dati trasferiti al secondo senza star effettivamente aumentando la velocità della singola lettura
- Utilizzando la **DDR** (Double Data Rate) al posto della SDR (Single Data Rate), ossia facendo muovere i dati non solo durante la salita del clock, ma anche durante la discesa, raddoppiando il numero di trasferimenti al secondo mantenendo una frequenza di clock identica.

Tramite questi accorgimenti, riusciamo a migliorare la quantità di dati che trasporta una singola cella senza però modificarne la velocità, che resta uguale, andando a modificare ad esempio il **prefetch**, ossia il numero di bit che un chip riesce a prelevare contemporaneamente per ogni

pin. È importante inoltre capire che le parole, anche se lette in parallelo, escono comunque serializzate, ossia una alla volta, un po' come succede nei trasmettitori seriali che abbiamo visto lo scorso capitolo. La parallelizzazione è però fondamentale perché dà il tempo al chip di prepararsi per l'accesso successivo mentre le parole escono una alla volta.

Generazione	Anno	Data rate (MT/s)	Prefetch	Banda massima con un canale
DDR	2000	200-400	2n	3.2 GB/s
DDR2	2003	400-1066	4n	8.5 GB/s
DDR3	2007	800-2133	8n	17.1 GB/s
DDR4	2014	1600-3200	8n	25.6 GB/s
DDR5	2020	3200-6400	16n	51.2 GB/s

Tabella 4.1: Generazioni di DDR

L'ultima colonna ci dice il valore massimo della **banda** (bandwidth, BW), che rappresenta il numero di dati che possono passare al secondo. Il calcolo è semplice:

$$\text{Banda} = \text{canali} \times 8B \times \text{Data rate}$$

dove i **canali** sono le diverse strade che collegano il processore alla RAM (se ho una sola strada tutti i dati passano da lì, mentre con più di una strada i dati possono viaggiare su più strade contemporaneamente). In generale, una CPU su un PC domestico ha 2 canali, mentre una CPU che gestisce un server ne ha fino a 12, dato che deve gestire un traffico di dati molto elevato. Invece, per quanto riguarda gli 8 byte nella formula, questi semplicemente rappresentano la larghezza di ogni canale, ossia 64 bit.

Ci sono due valori che però restano sempre costanti o quasi durante le varie generazioni:

- La **frequenza** (della matrice), che è passata solo da 200 MHz a 400 MHz
- La **latenza**, che è fissa a $14/17ns$

La frequenza è aumentata molto poco durante questi anni, ma questa viene compensata dal prefetch che invece è aumentato di 8 volte dal 2000 al 2020. Questi due valori insieme ci danno il data rate totale (ossia il numero di megatransfer al secondo), dato che data rate $\approx$ prefetch $\times$ frequenza, dunque è importante con ogni generazione cercare di migliorare o uno o l'altro per avere dei veri miglioramenti anche nella banda.
La latenza è invece rimasta sempre fissa perché dipende dalla fisica della cella e non dal clock: per leggere un dato bisogna comunque aprire la riga, aspettare il piccolo segnale prodotto dai condensatori e amplificarlo, e nessuna di queste operazioni diventa più rapida aumentando la frequenza dei pin.

4.2.4 HBM

Abbiamo visto come la DDR aumenti la banda facendo correre pochi fili ma più in fretta, aumentando ad esempio il prefetch. È possibile però aumentare la banda in un altro modo, ossia tenendo i fili a velocità normale ma aggiugnendone moltissimi. Questa idea da vita a un nuovo sistema, che prende il nome di **HBM** (High Bandwidth Memory), ad oggi sempre più importante perché fa parte delle GPU usate per l'IA.

Far passare però 1024 fili da un chip di memroia al processore attraverso una scheda madre è un'impresa impossibile, dunque l'HBM avvicina direttamente la memoria al processore così che questi si trovino direttamente nello stesso contenitore:

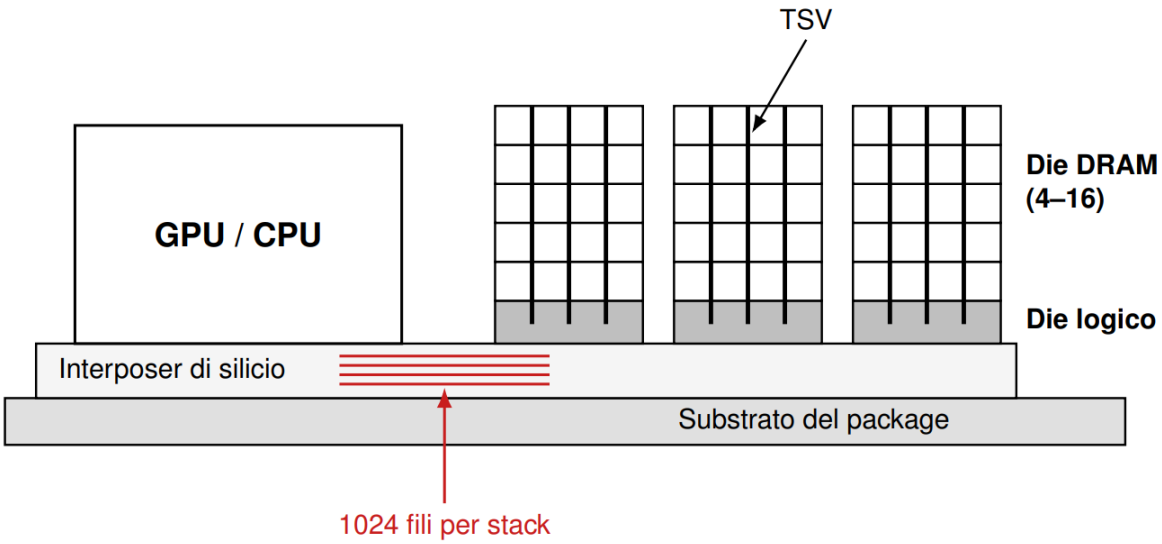


Figura 4.4: Contenitore con all'interno HBM e GPU/CPU insieme

Abbiamo quindi più DRAM impliati sopra un die logico (ossia pezzetti di silicio), che formano dei veri e propri *stack*, uniti da delle TSV, ossia dei collegamenti verticali che attraversano il silicio e permettono di collegare i diversi die DRAM senza passare dall'esterno. Vi è poi un interposer di silicio dove si trovano i fili di collegamento a distanza di millimetri.

Questo sistema ci permette di scambiare 1024 bit alla volta contro i 64 di un singolo canale di DDR, ma con un costo di implementazione decisamente più elevato.

Generazione	Pin	Gb/s per pin	Banda per stack	GB per stack
HBM2	1024	2	256 GB/s	8
HBM2E	1024	3.6	461 GB/s	16
HBM3	1024	6.4	819 GB/s	24
HBM3E	1024	9.6	1.2 TB/s	36
HBM4	2048	8	2 TB/s	Fino a 64

Tabella 4.2: Generazioni di HBM

Abbiamo quindi una banda che arriva fino a 2 TB/s per stack, contro i 51.2 GB/s di un canale DDR5. Questo è fondamentale per l'IA: infatti, un modello da 7 miliardi di parametri in FP16 (14 GB) può generare al massimo circa 146 token al secondo con uno stack HBM4, contro i 3.7 di un canale DDR5-6400 (una DDR5 con data rate da 6400 MT/s).

4.2.5 Memoria multi-porta

Le memorie che abbiamo visto fino ad ora ci permettono di accedere ad una sola parola per volta. Questo però non è sufficiente. Se infatti volessimo fare una somma tra due operandi, dovremmo avere la possibilità di farlo in un unico ciclo di clock, senza attendere la lettura di due parole in modo sequenziale. Nascono così le **memorie multi-porta**.

Definizione 4.3 – Memorie multi-porta

Una **memoria multi-porta** è una memoria che ha più **porte** (ossia coppie di indirizzo/dato), che ci permette di leggere/scrivere più dati contemporaneamente.

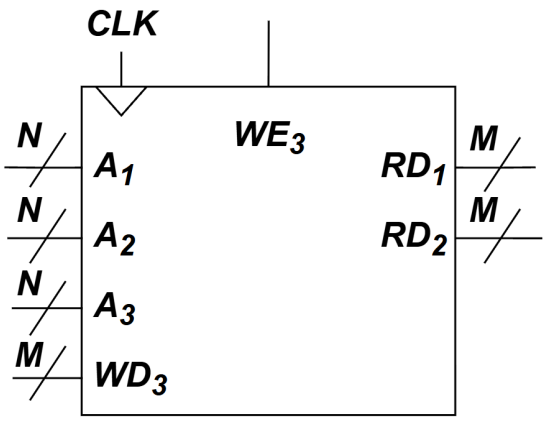


Figura 4.5: Memoria multi-porta

Una memoria multi-porta ha di solito diverse porte per la lettura e una (o poche più) porte per la scrittura, con un solo bit WE per decidere se la scrittura è abilitata o meno.

4.2.6 Costruire una memoria a partire da blocchi

Nella pratica non costruiamo mai una memoria da zero: si prendono invece dei **blocchi** di memorie più piccole di una certa forma per comporli in un certo modo così da ottenere il risultato voluto. Ci sono in generale tre casi in cui ci possiamo ritrovare:

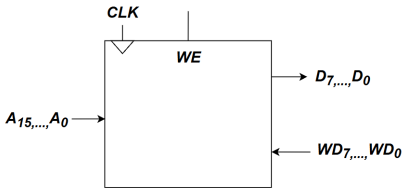
1) Il blocco è **più stretto** della memoria richiesta (ossia restituisce una parola più piccola di quella voluta): in questo caso ne affianchiamo diversi per allargare la parola del nostro blocco finale
2) Il blocco è **più corto** della memoria richiesta (ossia si utilizzano indirizzi più piccoli di quelli voluti): in questo caso se ne impilano diversi così da formare più parole
3) Il blocco è **più largo** della memoria richiesta (ossia restituisce una parola pià grande di quella voluta): in questo caso si utilizza solo una parte della parola alla volta

Vediamo tre esempi, uno per caso, per capire veramente come risolvere questi problemi.

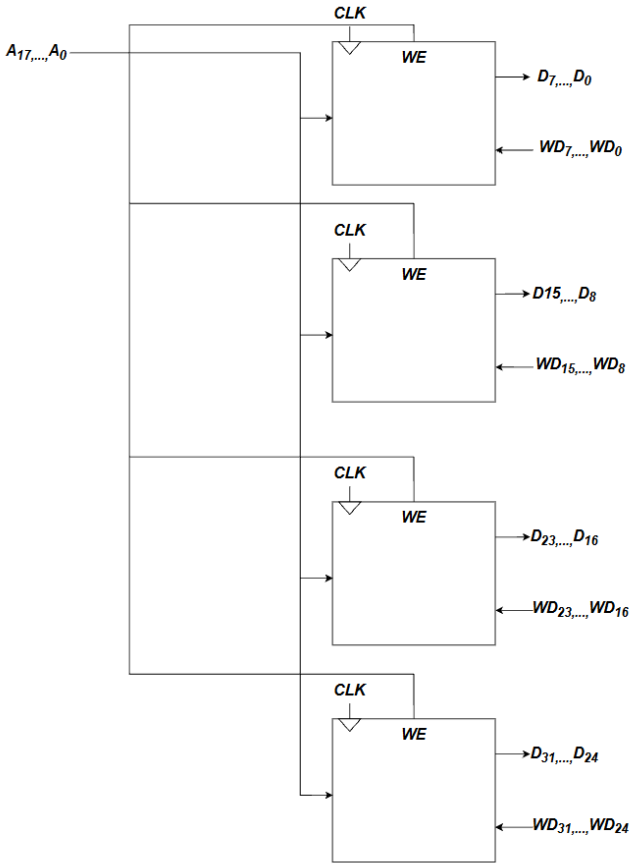
Blocco più stretto

Costruiamo una memoria di 1 MB con parole da 32 bit a partire da blocchi di memoria di dimensioni $2^{18} \times 8$ (ossia di $2^{18} \times 8$ bit con parole da 8 bit l'una).

Abbiamo quindi blocchi di questo tipo, con indirizzi da 18 bit e parole in output/input da 8 bit l'una.



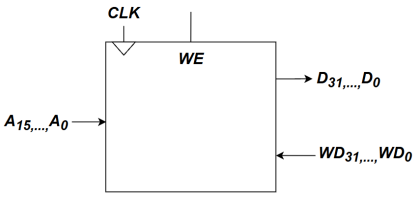
Dato che vogliamo arrivare a parole di 32 bit, ci basta semplicemente unire 4 di questi blocchi insieme. In questo modo arriveremo poi a una capacità totale di $2^{18} \times 8 \times 4 = 2^{23}$ bit, ossia 2^{20} byte, ossia 1MB, proprio come volevamo. Dunque avremo come soluzione a questo esempio la seguente memoria:



Blocco più corto

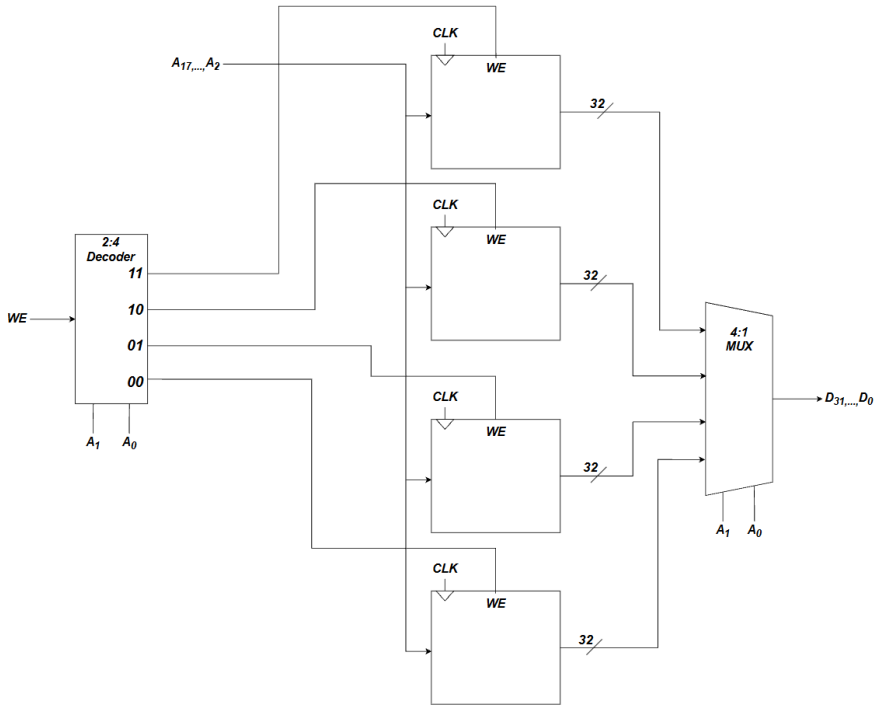
Costruiamo una memoria di 1 MB con parole da 32 bit a partire da blocchi di memoria di dimensioni $2^{16} \times 32$.

Abbiamo quindi blocchi di questo tipo, con indirizzi da 16 bit e parole in output/input da 32 bit l'una.



Dato che vogliamo rappresentare non 2^{16} parole ma 2^{18} possiamo anche qui prendere quattro di questi blocchi, così da avere $2^{16} \times 4 = 2^{18}$ parole totali. Per fare ciò, però, avremo bisogno di un mux che ci dice quale delle quattro parole all'indirizzo $A_{17}, \ldots, A_2$ selezionare. Questo mux avrà quindi 2 bit di selezione, che sono proprio i due bit dell'indirizzo A_1 e A_0 che mancavano tra il blocco e la memoria che volevamo creare. Oltre al mux, dovremo però aggiungere anche un decoder: questo è necessario per quanto riguarda la parte di scrittura. Infatti, se vogliamo scrivere una parola in memoria a un certo indirizzo dobbiamo farlo solo su uno dei quattro blocchi, dunque dobbiamo abilitare il bit WE solo su uno di questi quattro blocchi, e il decoder è

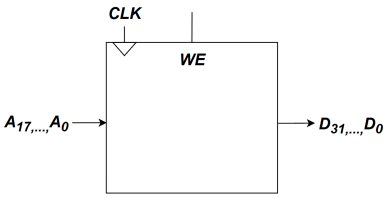
proprio il costrutto perfetto che ci permette di fare ciò. Dunque avremo come soluzione a questo esempio la seguente memoria:



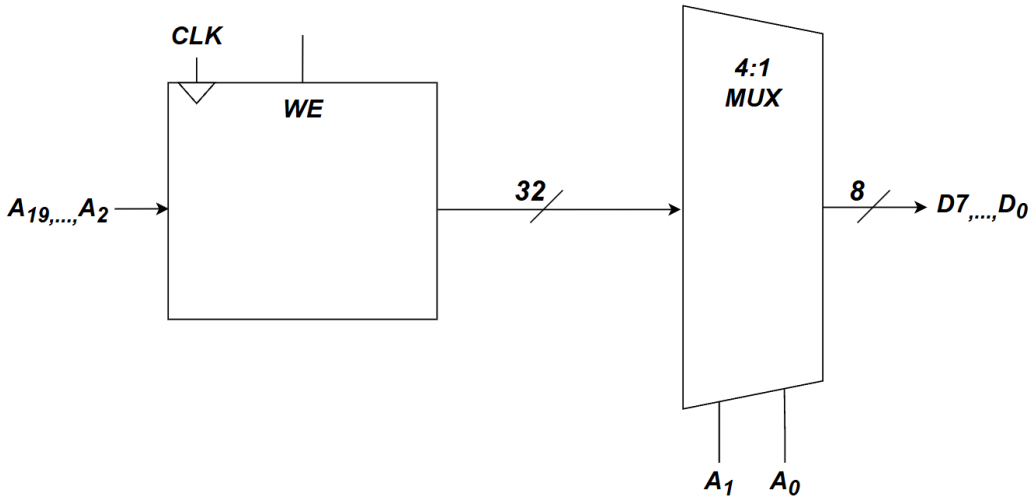
Blocco più largo

Costruiamo una memoria di 1 MB con parole da 8 bit in modalità solo lettura a partire da blocchi di memoria di dimensioni $2^{18} \times 32$.

Abbiamo quindi blocchi di questo tipo, con indirizzi da 16 bit e parole solo in input da 32 bit l'una.



Dato che vogliamo rappresentare parole non da 32 bit ma solo da 8, possiamo usare 8 MUX che prendano in ingresso 4 bit e ne restituiscano uno solo, così da avere in totale parole da soli 8 bit. Useremmo quindi un indirizzo da 18 bit per il blocco di memoria che abbiamo, e i due bit che avanzano dell'indirizzo della nostra memoria (che ne ha 20 dato che $2^{32}/8 = 2^{20}$) verranno usati come bit di selezione per gli 8 mux. Dunque avremo come soluzione a questo esempio la seguente memoria:


CLK
WE
A19,...,A2
32
4:1
MUX
8
D7,...,D0
A1
A0